# Build Federation

A list of frequently asked questions about our build processes.  Feel free to update.



Some of processes involving change, like modifying or copying Jenkins pipelines in certain use cases, will be implemented by `sbx manage-jenkins-jobs`. Run the command to find out what it can do.

## Process

### What is the development process ?

The development process can best be described using a slide :-

1. Developer checks out the source code
2. Required dependencies are resolved and downloaded to the developer desktop
3. Developer performs his/her job, including building and running unit test cases locally
4. Once finished with the task, source code changes are checked back in
5. At some point, the build scheduler determines that a build is due - this could be from time-of-day, change noticed in the source code, dependency updated or other trigger
6. The build scheduler send a command to the build server to request a build.  Note that the scheduler does not do any builds itself
7. The build server checks out from source code control
8. The build server resolves required dependencies and downloads them
9. The build server runs the compile / test cycle
10. If the test cases are successful, the binaries are deployed to the repository - these are then available for use by developers and dependent builds
11. Optionally, static analysis is performed on the code and the results sent to quality analysis system.  This can include results of code coverage runs
12. QA uses the deployed artefacts for any additional manual tests and execute the release process

### What is the ep-build-group@tibco.com address?

The address `ep-build-group` comprises the individuals in the build group. It's also the return address for Jenkins mail. Mail for SSL certificate maintenance, certain Jenkins configuration changes and changes to certain Git repositories is also addressed to the group.

### What tools do we use ?

1. Jenkins to schedule builds - see http://jenkins-bos.na.tibco.com:8080/
2. Artifactory to store built artifacts - see http://ep-artifacts-us-east.tibco.com and http://artifacts.tibco.com/artifactory/webapp/#/home
3. The SonarQube instance can be found here http://jenkins.tibco.com:9000/

### What is a Jenkins build job ?

A single Jenkins build job is responsible for building a single codeline (Git repository).  A typical build job consists of :

1. Checkout the source code
2. Build
3. Run test cases
4. If test cases pass, deploy artifacts to artifactory
5. Build site documentation
6. Deploy site documentation

Jenkins jobs are usually scheduled within a Jenkins pipeline to produce a milestone build. We also have new multibranch pipeline that build a single codeline.

There are also special case or utility jenkins jobs for administration ( such as helping with releases and branching ).

### Why Can't I Do Anything Interesting In Jenkins?

You aren't logged in. Log with your TIBCO credentials.

### What is a milestone ?

Milestone builds are a consistent set of dependencies tagged with a milestone identifier. They are used to keep a set of snapshot artifacts consistent across a sequence of builds. Each build will deploy its artifacts when it succeeds, but a milestone tag allows different deployments of those builds (like `parent dtm sbrt`) to be found from the same sequence. Milestone identifiers also incorporate a platform; otherwise there is no way of separate the Linux snapshot runtime artifacts from Windows snapshot runtime artifacts.

Milestone identifiers consists of <<product>>-<<version>>-<<platform>>-<<unique number>>.  For example, streambase-110-linux-6

1. *branch* can be master, or a minor release without the dot, like `106`.
2. *platform* is `linux`, `macos` (`osx` for older EventProcessing jobs), or `windows`

# What is a Jenkins pipeline ?

Generally, a Jenkins pipeline ( see https://jenkins.io/doc/book/pipeline/ ) is script written in a Groovy-based language that controls a build. Scripted pipelines use steps of Groovy code to their work. Declarative pipelines, for multi-branch pipelines especially, allow nearly all of the characteristics of a build to be defined in the script, instead of being part of the XML-based configuration of the pipeline as stored in Jenkins.

Most of our pipelines use code in https://github.com/tibco/ep-jenkins-workflow to hide details and abstract away useful operations. The classes under `src/` are Groovy script classes, while `var/epDeclarative.groovy` builds on top of those classes to reduce the amount of repetition in our declarative pipelines.

## Jenkins Scripted Pipelines

We use Jenkins *scripted* pipelines to orchestrate building a milestone, even if the milestone invokes just one worker job. The source for such pipelines is kept under source control, under the characteristic repository of the codeline used for that milestone pipeline.

Pipeline jobs co-ordinate a complete milestone build so they are required to :

- Generate the milestone identifier, or query for last good upstream milestone
- Manage global parameters. Defaults should be computed using functions in the workflowLibs in ep-jenkins-workflow.
- Select a build server using labels based on the target platform and the job's needs
- Schedule all the build jobs in the right order, stopping on failure
- Notification to the team with the results

## Jenkins Declarative Pipelines For Multibranch Pipelines

We use Jenkins declarative pipelines for building single repositories on one platform. There are not connected to milestones, although they can request a milestone for their dependencies. The features are particularly useful with GitHub. For each such job, any branch can be built on Jenkins without the branch creator having to do anything extra.

### Basics

- For side branches (not master, main, or release-work), Maven builds run but **do not deploy**. Such convenience encourages development on branches, as you can see how the code actually behaves in Jenkins and our lab, where surprises may occur. Work can then be merged manually or reviewed as a GitHub pull request.
- The  Maven SNAPSHOT version is checked so that only release-work branches can deploy releases.
- It's easy to specify a lot of common behaviors by defining them in the map passed to `epDeclarative.buildInit()`.
- Changes in the build itself can be experimented on branches as well.
- Maven sites are properly deployed to a version-specific location with a little cooperation from the top-level pom and passing in a `siteProduct:` to `epDeclarative.buildInit()`.
- There are no resume options yet; if something fails, try to fix it and let it run again. Having such pipelines orchestrate builds on multiple platforms is a goal for the future.

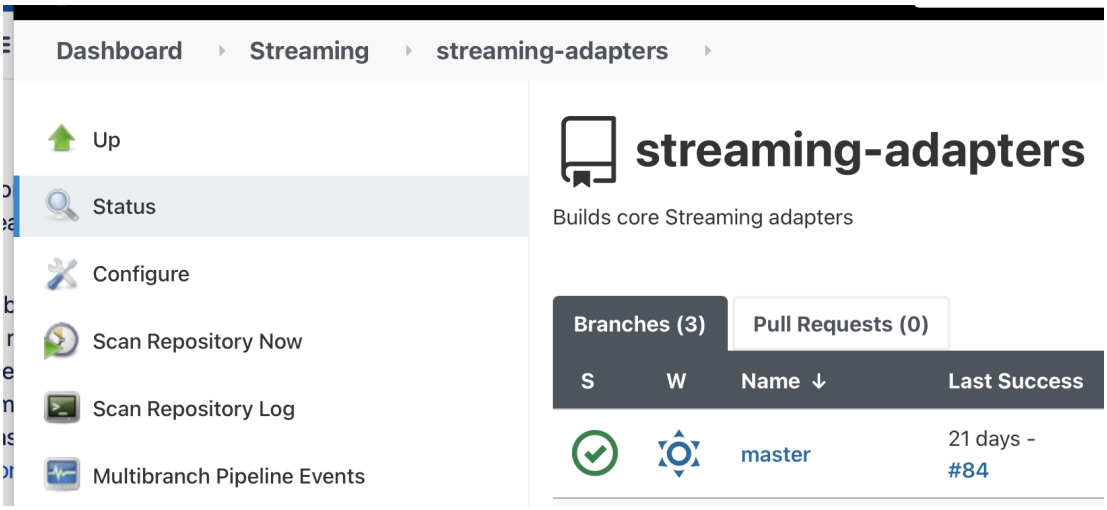


Do not create GitHub pull requests whose name matches an existing branch, even with a different case (like `Main/1.1`). This will confuse Jenkins greatly and hide branches from being discovered, or even remove their corresponding branch jobs! Such PRs should be deleted or renamed, then use "Scan Repository Now" to reintroduce the branch's build (see below).

See the epDeclarative documentation for details.

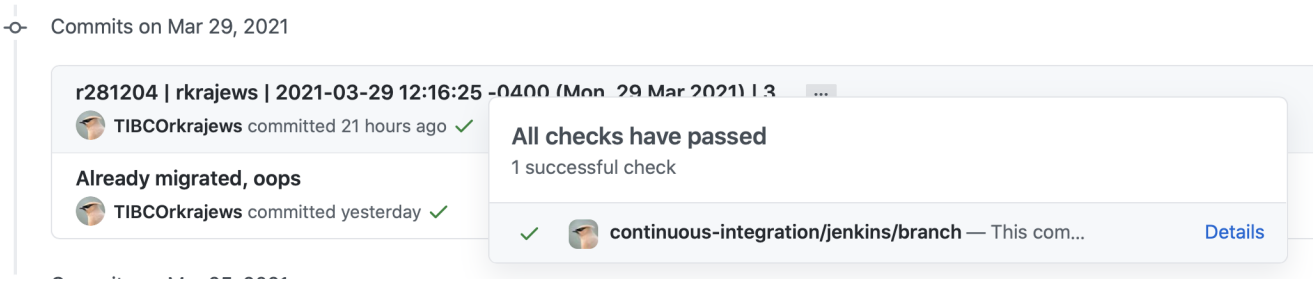
### Branch Discovery

Periodically, the multibranch pipeline discovers new branches that can be built. How often those branches are built is declared in the Jenkinsfile, but side branches might not be built on SCM changes at all if the job is configured that way. To force discovery of new branches, click the "Scan Repository Now" link in the top-level job:




### Build Status Between Jenkins and GitHub

There are links between GitHub and these builds. For example, if you look the history of `master` on ep-streaming-adapters, you will see checkmarks and Xs for builds that worked and didn't. If you click on such an indicator, you'll get a popup like this:




The Details link will bring you to the Jenkins build run, assuming it hasn't aged out there.

### Defining Jobs

The pipeline is defined largely by the source of the Jenkinsfile. You can see that the parameter definitions take up most of the text, with the call to `buildIn it` expressing the most control. There is also some unavoidable copying of typical `post` actions. Although this is called a "declarative" pipeline, the real work still happens in scripted steps; given our requirements, that's hard to avoid.

### Creating Jobs

The basic idea is to take a Jenkins job object that does something similar to what you need, and copy it. The two classes of jobs for prototypes are:

- Streaming-related jobs like Streaming/streaming-adapters.
- Generic jobs that use `main` and `release-work` as the branch names for maintenance and release work, like ModelOps/ep-modelops.linux.

The reason this matters is that certain branch policies need to be configured in the web interface, which can be tricky. If the branch naming patterns are already implemented in a job, you don't have to edit that in the new copy.

As for the Jenkinsfile itself, you should also start from an existing one that is close to what you need. You should pay attention to the mail notification parameters to see if they make sense for the job.

The name of the job is usually the name of the Git repository with `ep-` dropped at the front if present. Pick or create a good folder for the job. Platform-specific job names should end in `.linux`, `.macos`, or `.windows`. The pipeline "stem" name is usually related to the job name, and so the path of the Jenkinsfile will be `.EPDev/build-management/`*`stem`*`/Jenkinsfile`.

Once all that is determined, use the `sbx modify-mb-pipeline` command to copy the job from a good candidate; the `--help` option describes usage and requirements. You need to supply the `--really` option to make it happen. A typical case:

```
sbx modify-mb-pipeline ModelOps/ep-modelops.linux Streaming/ares --codeline ep-ares --pipeline-stem ares --
description "The Ares project."
```

Because of Jenkins limitations, you might need to set the GitHub credentials back to "Account allowed to read/write EP repositories. Unfortunately tied to a user account." in the web interface (using Configure) – if the GitHub source URL is httpsL, it needs to use that account. You can also a use a deploy key and ssh: URL but then you lose build status links.

Note that a repository can default more than one pipeline and a pipeline can be used for more than one job. The latter is convenient when setting up a set of platform-specific jobs for different platforms – the names of the jobs will be used to infer the platform for the job.

## What is a "no deploy" build ?

Since a full milestone build consists of running many test cases, it takes a considerable amount of time to complete.

So in addition to the full builds, we have "no deploy" builds as well - these only have smoke tests enabled so complete much quicker.  No unit tests are run and nothing is deployed to Artifactory. These are scripted pipelines.

## What conventions are followed ?

1. Maven conventions are followed
2. Builds can either be a SNAPSHOT build or a release build
3. SNAPSHOT builds have the following features :
    a. Intended for internal continuous automation builds
    b. Version suffix is SNAPSHOT, for example 1.2.3-SNAPSHOT
    c. Multiple versions for the same build can co-exist on the repository
    d. Repository will automatically clean up old builds
    e. Artifacts are stored in the ep-snapshots repository.
4. Release builds have the following features :
    a. Intended for releases
    b. Version is fixed, for example 1.2.3
    c. A single version of the build can co-exist on the repository
    d. Release builds are never removed
    e. Release candidate artifacts are stored in one of the release candidate repositories - ep-rc, ep-rc-windows, ep-rc-macos, or ep-rc-linux.
    f. Released artifacts are stored in the ep-releases* repository.
5. Builds may have dependencies :
    a. Internally built from source
    b. Internally build from binaries (i.e. wrapped)
    c. External

## Can I simply build everything locally and ignore official builds ?

Yes - developers often build everything locally - this is fast (since tests can be skipped) and flexible (various combinations of branches can be checked out and worked on locally).  There are several reference pages to assist in this :

- Development Environment
- Using vagrant on OS X Mac to create a virtual machine for development
- Building StreamBase On A Workstation
- Build Details

## How do I use my own settings.xml ?

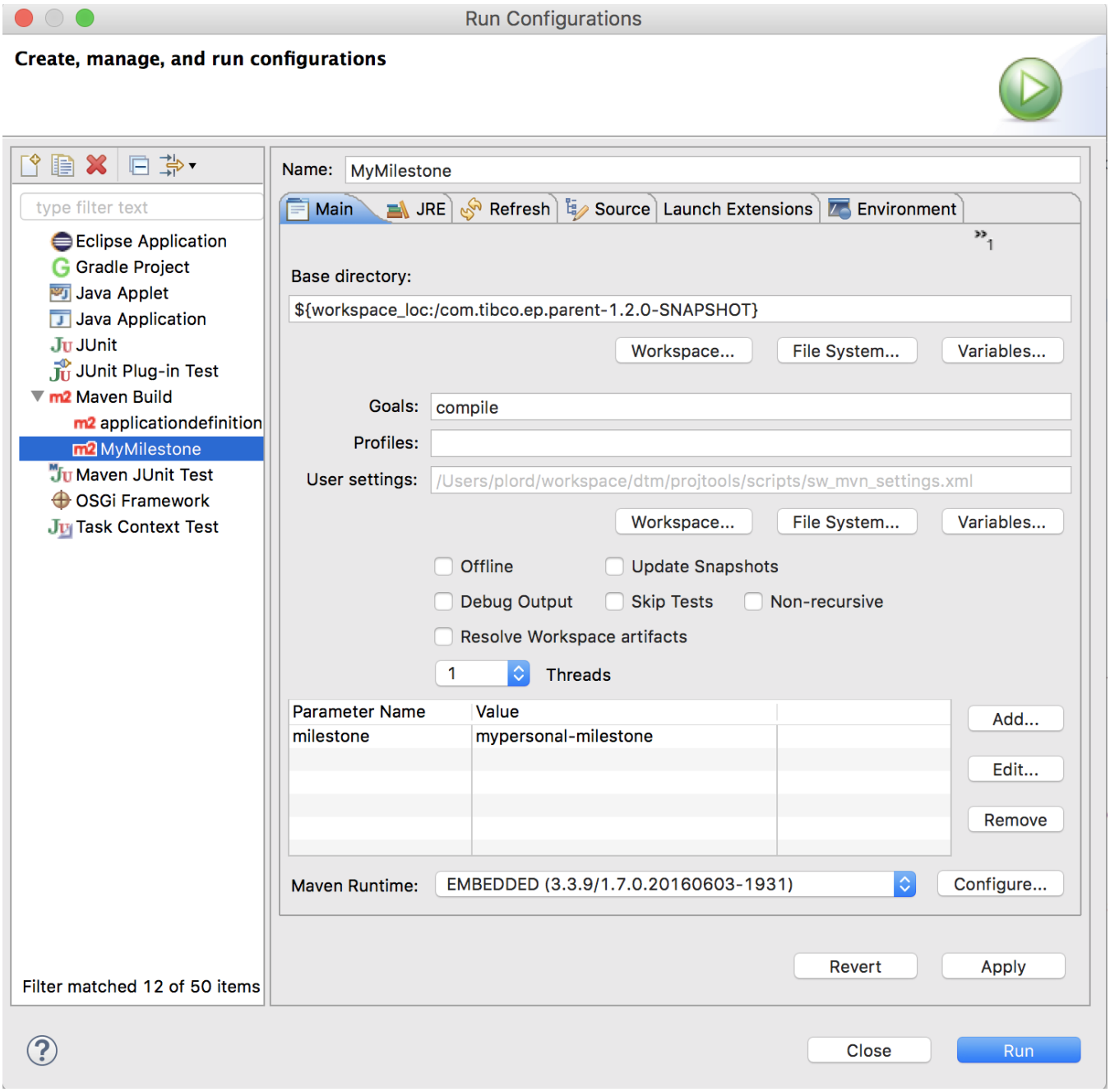
There is a managed Maven `settings.xml` checked into ep-streaming-dtm at projtools/scripts/sw_mvn_settings.xml – you can copy from there, or retrieve it with `sbx epmvn cat-settings`. You can edit your own copy for your own purposes, like a different default Artifactory server.

The `repo.server` property inside the Maven settings file is `ep-artifacts-us-east.tibco.com` – you should change your copy to use `artifacts.tibco.com` (Palo Alto) or `ep-artifacts-pune.tibco.com` (Pune) for your location.

So, for example, to build with a specific milestone use :

```
mvn -Dmilestone=mypersonal-milestone
```

The same settings can be used in eclipse run configurations :




## Can I build/test an application without installing anything ?

Yes ... sometimes you don't want to install much - for example if you want to run an application with an older version of StreamBase without touching your development tree.

- Ensure project pom.xml is set to the product version you want to use :

```
    <!-- common definitions for this version of StreamBase -->
    <parent>
        <groupId>com.tibco.ep.sb.parent</groupId>
        <artifactId>ep-eventflow-fragment</artifactId>
        <version>10.2.1</version>
        <relativePath/>
    </parent>
```

- Set up isolated TIBCO_EP_HOME :

```
$ TIBCO_EP_HOME=$(pwd)/ephome
$ export TIBCO_EP_HOME
$ PATH=${TIBCO_EP_HOME}/distrib/tibco/bin:${PATH}
$ export PATH
$ mkdir -p ${TIBCO_EP_HOME}
```

- Run Maven

```
$ sw_mvn -u -Drelease-candidate -t -b PRODUCTION
...
[INFO] Downloaded: http://ep-artifacts-us-east.tibco.com/ep-rc-macos-virtual/com/tibco/ep/sb/rt
/platform_osxx86_64/10.2.1/platform_osxx86_64-10.2.1.zip (131 MB at 4.7 MB/s)
...
[INFO] ------------------------------------------------------------------------
[INFO] Building My Application - EventFlow Fragment 1.0.0
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO] --- ep-maven-plugin:1.2.1:install-product (default-install-product-1) @ myapp_eventflowfragment ---
[INFO] Installing com.tibco.ep.sb.rt:platform_osxx86_64:zip:10.2.1:test to /Users/plord/workspace/reproductions
/nar/ephome
[INFO]
[INFO] --- ep-maven-plugin:1.2.1:set-resources (default-set-resources) @ myapp_eventflowfragment ---
[INFO]
[INFO] --- maven-resources-plugin:3.0.2:resources (default-resources) @ myapp_eventflowfragment ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO] Copying 2 resources
[INFO] Copying 2 resources
[INFO]
[INFO] --- ep-maven-plugin:1.2.1:unpack-fragment (default-unpack-fragment) @ myapp_eventflowfragment ---
[INFO]
[INFO] --- ep-maven-plugin:1.2.1:unpack-nar (default-unpack-nar) @ myapp_eventflowfragment ---
[INFO]
[INFO] --- ep-maven-plugin:1.2.1:compile-eventflow-fragment (default-compile-eventflow-fragment) @
myapp_eventflowfragment ---
[INFO]
[INFO] --- maven-compiler-plugin:3.6.1:compile (default-compile) @ myapp_eventflowfragment ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:3.0.2:testResources (default-testResources) @ myapp_eventflowfragment ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO] Copying 1 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.6.1:testCompile (default-testCompile) @ myapp_eventflowfragment ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- ep-maven-plugin:1.2.1:start-nodes (default-start-nodes) @ myapp_eventflowfragment ---
[INFO] [A.myapp_eventflowfragment] Running "install node"
[INFO] [A.myapp_eventflowfragment]         Installing node
[INFO] [A.myapp_eventflowfragment]                 PRODUCTION executables
[INFO] [A.myapp_eventflowfragment]                 File shared memory
[INFO] [A.myapp_eventflowfragment]                 7 concurrent allocation segments
[INFO] [A.myapp_eventflowfragment]                 Host name plordimac.plord.co.uk
[INFO] [A.myapp_eventflowfragment]                 Container tibco/sb
[INFO] [A.myapp_eventflowfragment]                 Starting container services
[INFO] [A.myapp_eventflowfragment]                 Loading node configuration
[INFO] [A.myapp_eventflowfragment]                 Auditing node security
[INFO] [A.myapp_eventflowfragment]                 Administration port is 59195
[INFO] [A.myapp_eventflowfragment]                 Discovery Service running on port 52979
[INFO] [A.myapp_eventflowfragment]                 Service name is A.myapp_eventflowfragment
[INFO] [A.myapp_eventflowfragment]         Node installed
[INFO] [A.myapp_eventflowfragment] Finished "install node"
[INFO] [myapp_eventflowfragment] Running "start node"
[INFO] [A.myapp_eventflowfragment]         Starting node
[INFO] [A.myapp_eventflowfragment]                 Loading node configuration
[INFO] [A.myapp_eventflowfragment]                 Auditing node security
[INFO] [A.myapp_eventflowfragment]                 Host name plordimac.plord.co.uk
```

```
[INFO] [A.myapp_eventflowfragment]                Administration port is 59195
[INFO] [A.myapp_eventflowfragment]                Discovery Service running on port 52979
[INFO] [A.myapp_eventflowfragment]                Service name is A.myapp_eventflowfragment
[INFO] [A.myapp_eventflowfragment]         Node started
[INFO] [myapp_eventflowfragment] Finished "start node"
[INFO]
[INFO] --- ep-maven-plugin:1.2.1:test-eventflow-fragment (default-test-eventflow-fragment) @
myapp_eventflowfragment ---
[INFO] [dtm] INFO: Deploy tool version: [TIBCO Distributed Transactional Memory Platform 3.2.1 (build
1801310751)] starting at [Thu Apr 26 11:03:52 BST 2018]
[INFO] [dtm] INFO: Node version: [TIBCO StreamBase Runtime 10.2.1 (build 1801311520)]
[INFO] [dtm] INFO: Starting com.tibco.ep.buildmavenplugin.surefire.Runner on node A.myapp_eventflowfragment ...
[INFO] [dtm] INFO: com.tibco.ep.buildmavenplugin.surefire.Runner started on JVM
com_tibco_ep_buildmavenplugin_surefire_Runner0 on node A.myapp_eventflowfragment.
[INFO] [A.myapp_eventflowfragment] No user-defined Logback configuration, using product default configuration
[INFO] [A.myapp_eventflowfragment] Initializing NodeDispatcher resources for this JVM
[INFO] [A.myapp_eventflowfragment] 2018-04-26 11:03:55.521000+0100 [58178:main] INFO  com.tibco.ep.dtm.stdout:
[INFO] [A.myapp_eventflowfragment] -------------------------------------------------------
[INFO] [A.myapp_eventflowfragment]  T E S T S
[INFO] [A.myapp_eventflowfragment] -------------------------------------------------------
[INFO] [A.myapp_eventflowfragment] Running com.tibco.ep.myapp.TestCase
[INFO] [A.myapp_eventflowfragment] loaded shared lib
[INFO] [A.myapp_eventflowfragment] Transactional services available
[INFO] [A.myapp_eventflowfragment] Test Case 1
[INFO] [A.myapp_eventflowfragment] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.501 sec
[INFO] [A.myapp_eventflowfragment] 2018-04-26 11:03:58.028000+0100 [58178:main] INFO  com.tibco.ep.dtm.stdout:
[INFO] [A.myapp_eventflowfragment] Results :
[INFO] [A.myapp_eventflowfragment] 2018-04-26 11:03:58.028000+0100 [58178:main] INFO  com.tibco.ep.dtm.stdout:
[INFO] [A.myapp_eventflowfragment] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] [A.myapp_eventflowfragment] 2018-04-26 11:03:58.028000+0100 [58178:main] INFO  com.tibco.ep.dtm.stdout:
[INFO] [dtm] INFO: Engine com_tibco_ep_buildmavenplugin_surefire_Runner0 on node [A.myapp_eventflowfragment]
exited with status [0]
[INFO] [myapp_eventflowfragment] Finished "junit"
...
```

So the runtime is downloaded, installed and available for use.

## How can I check for incorrect dependencies ?

The parent pom contains build rules for the maven enforcer - see https://maven.apache.org/enforcer/maven-enforcer-plugin/ - whilst this is always run, the default for enforcer.fail is false, so the build doesn't fail.

```
$ sw_mvn install -Denforcer.fail=true
...
[INFO] --- maven-enforcer-plugin:3.0.0-M2:enforce (Dependencies) @ java-thirdparty ---
[WARNING]
Dependency convergence error for org.apache.mina:mina-core:1.1.5 paths to dependency are:
+-com.tibco.ep:java-thirdparty:10.4.0-SNAPSHOT
  +-org.apache.mina:mina-core:1.1.5
and
+-com.tibco.ep:java-thirdparty:10.4.0-SNAPSHOT
  +-org.apache.mina:mina-filter-ssl:1.1.5
    +-org.apache.mina:mina-core:2.0.4

[WARNING] Rule 0: org.apache.maven.plugins.enforcer.DependencyConvergence failed with message:
Failed while enforcing releasability. See above detailed error message.
```

## How can I check for vulnerabilities?

We are using OWASP via the maven plugin dependency-check-maven.  Vulnerabilities are generated as part of the site reports ( for example see http://ep-artifacts-us-east.tibco.com/site/latest-master-linux/sb/sb-product/dependency-check-report.html ).

To run locally -

```
$ sw_mvn -Dmaven.site.skip=false site
```

and check html reports in target/site/dependency-check-report.html .  The first run will be slow at the databases are downloaded/updated.

The jenkins job mirrorowasp downloads the OWASP data files daily, we can also keep OWASP database persistent across builds to further reduce job run times.  So Jenkins is configured with :

- -DdataDirectory=/opt/tibco/users/jenkins/osaspdb
- -DcveUrl12Modified=http://ep-artifacts-us-east.tibco.com/owasp/nvdcve-modified.xml.gz
- -DcveUrl20Modified=http://ep-artifacts-us-east.tibco.com/owasp/nvdcve-2.0-Modified.xml.gz
- -DcveUrl12Base=http://ep-artifacts-us-east.tibco.com/owasp/nvdcve-%d.xml
- -DcveUrl20Base=http://ep-artifacts-us-east.tibco.com/owasp/nvdcve-2.0-%d.xml

Our GitHub-hosted repositories are also checked by Dependabot.

## How can I check my code coverage ?

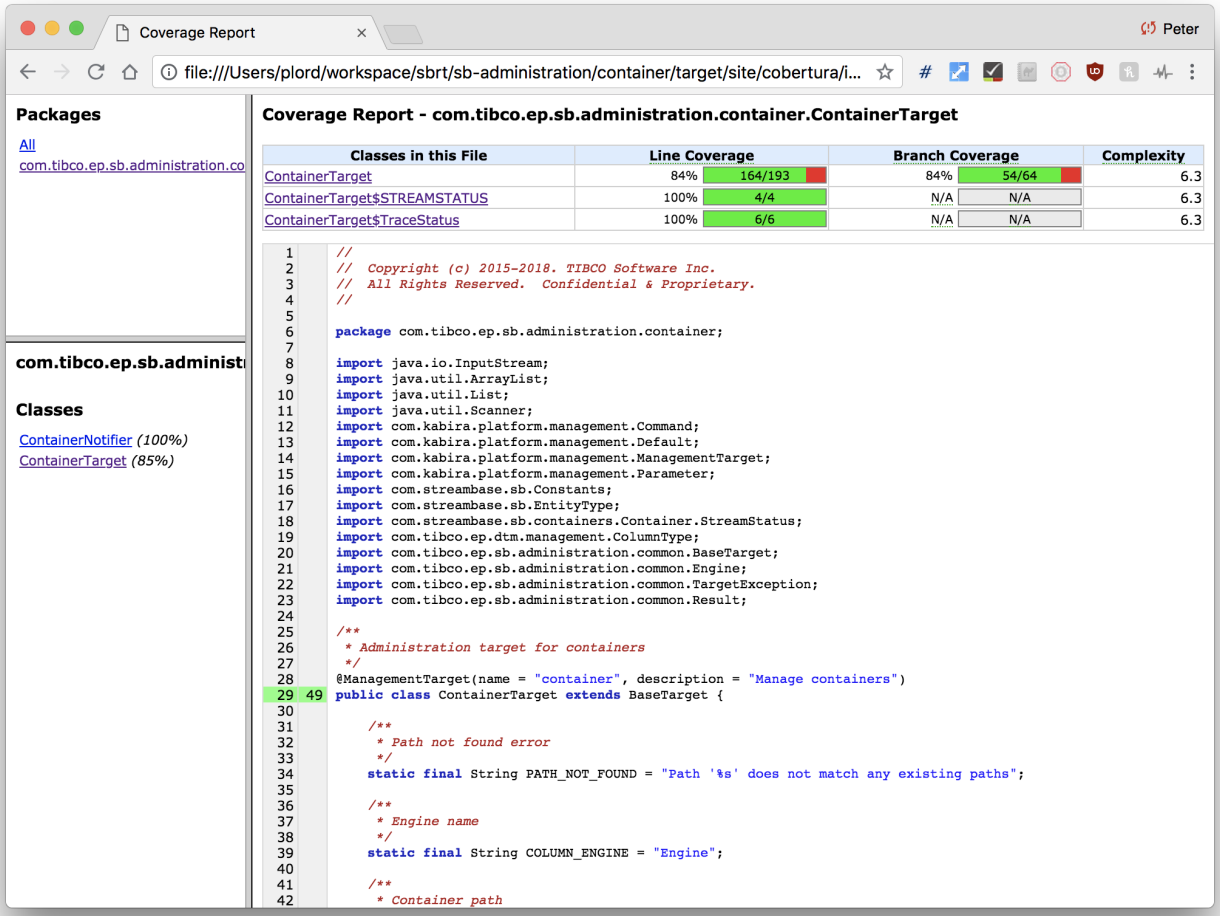
> Cobertura does not work with Java 11: see JIRA Issue SB-48516.

Use Cobertura ( see https://github.com/cobertura/cobertura ) via the Maven plugin ( see http://www.mojohaus.org/cobertura-maven-plugin/ ).  For example :

```
$ sw_mvn clean; sw_mvn -t cobertura:cobertura -u -Dcobertura.skip=false
...

[INFO] Reading node coverage report /Users/plord/workspace/sbrt/sb-administration/container/target/surefire-
reports/cobertura/A.container.cobertura.ser
[INFO] Writing cluster coverage report /Users/plord/workspace/sbrt/sb-administration/container/target/cobertura
/cobertura.ser
[INFO] Cobertura: Loaded information on 4 classes.
[INFO] Cobertura: Loaded information on 4 classes.
[INFO] Cobertura: Saved information on 4 classes.
```

To view the report, direct your browser at target/site/cobertura/index.html :




See also http://devzone.tibco.com/sites/streambase/latest/ep-maven-plugin/java_examples.html#Code_coverage_via_cobertura

Note that cobertura instruments your class files, so its important to clean before and after running cobertura.

Jenkins will also display coverage reports if enabled as will sonarqube.

The flag -Dcobertura.skip=false is used to add cobertura to the dependencies (see parent pom) - this avoid unnecessary dependencies for the real build.

## How can I run Spotbugs ?

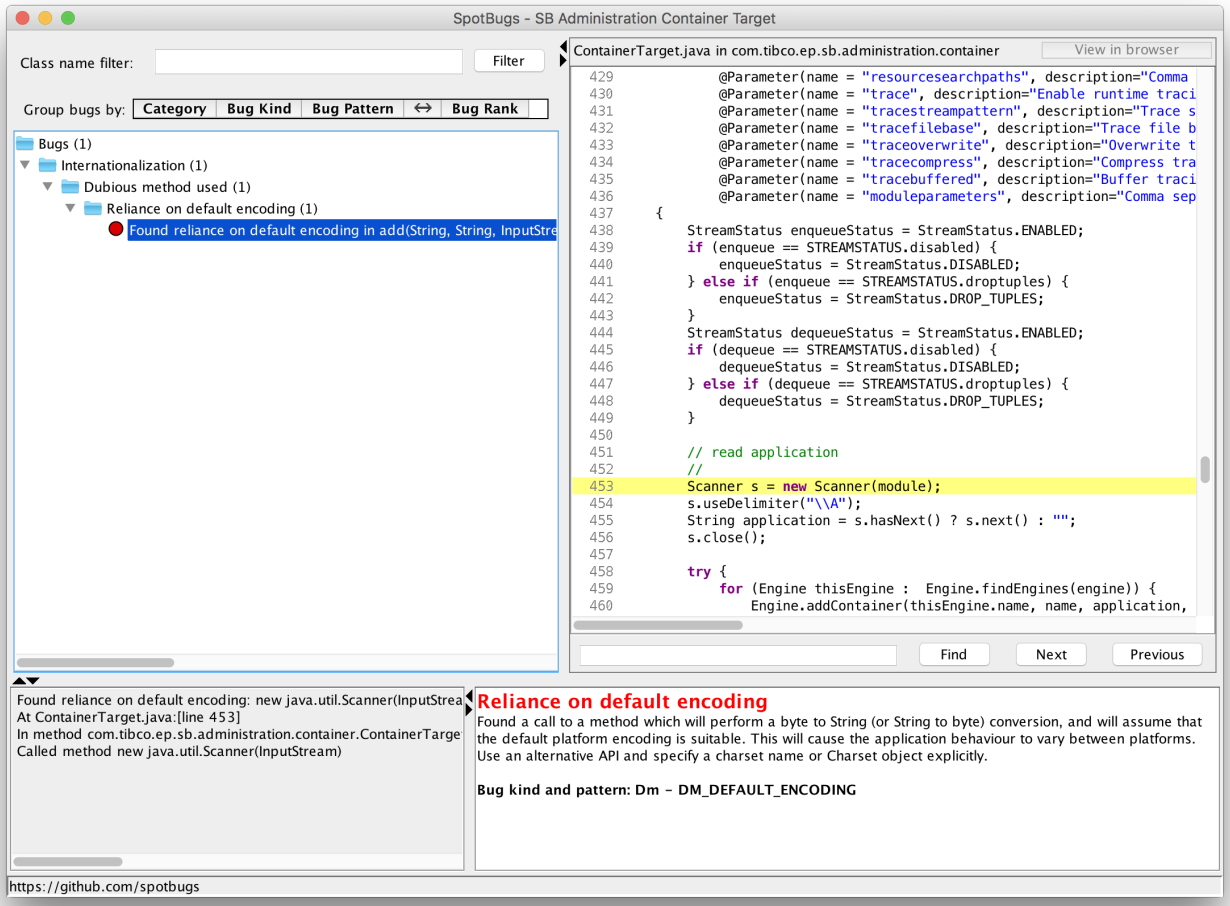
Spotbugs ( https://spotbugs.github.io/ ) can be run within Eclipse, but it can also be run from the command-line using the Spotbugs Maven plugin ( https://spotbugs.github.io/spotbugs-maven-plugin/index.html ).

```
$ sw_mvn -Dspotbugs.skip=false spotbugs:spotbugs
...

[INFO] --- spotbugs-maven-plugin:3.1.6:spotbugs (default-cli) @ container ---
[INFO] Fork Value is true
     [java] Warnings generated: 1
[INFO] Done SpotBugs Analysis....
```

The results can be viewed using the GUI tool, which you probably want to run on just one project, **not** the entire reactor build. Otherwise, you'll launch the GUI tool more than a hundred times, one after the other, in the sbrt projects!

```
$ sw_mvn spotbugs:gui
```




The Spotbugs reports are also sent to http://sadm.tibco.com by Jenkins.

# Finding Things

If you want just want the latest tested or official builds, especially for companion products, see Where To Find Product Builds.

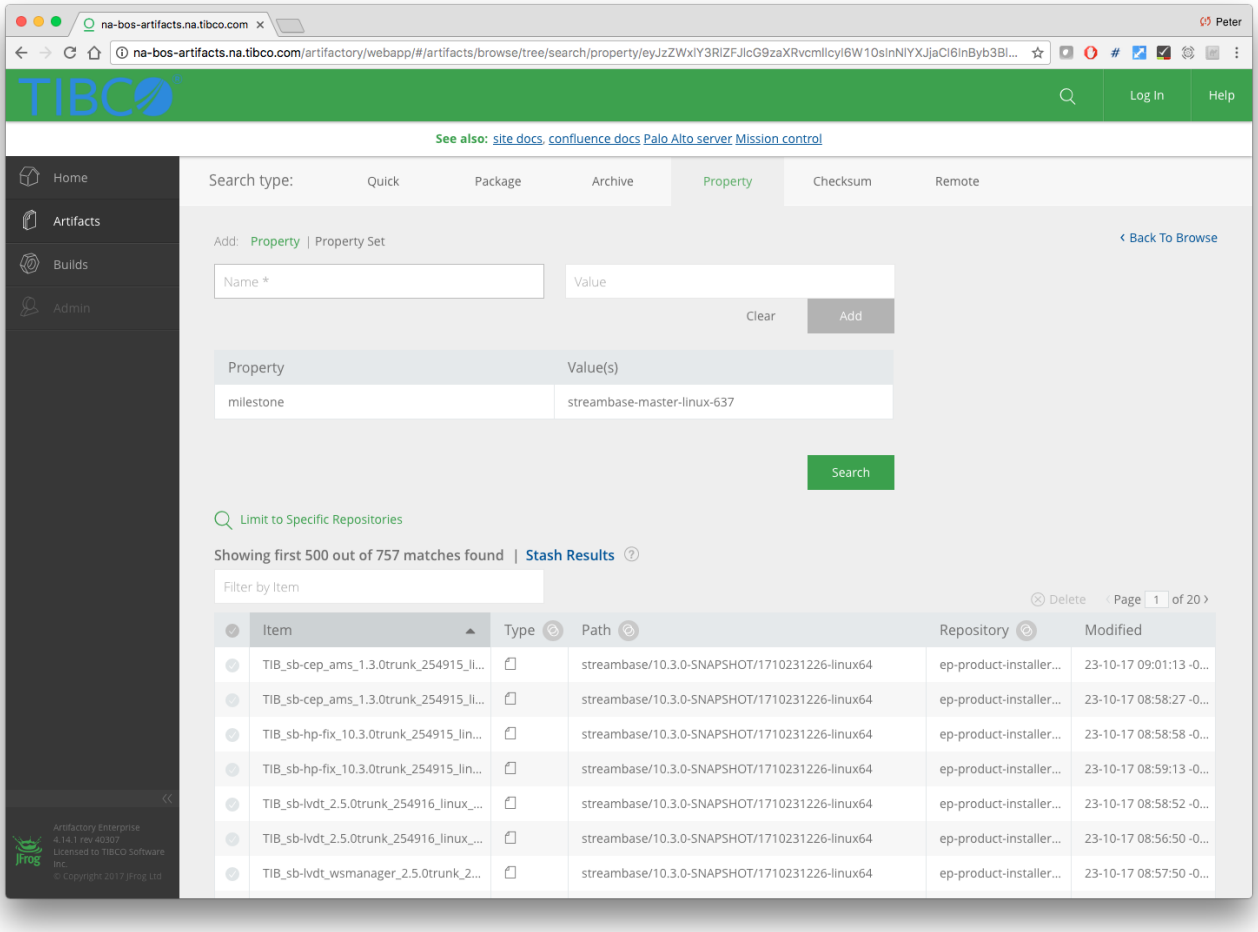
## Where can I find milestone installers ?

The milestone installers are deployed to artifactory in the **ep-product-installers** repository on our Artifactory servers.

- http://ep-artifacts-us-east.tibco.com/artifactory/ep-product-installers/streambase/

> To use Artifactory search, you must be logged into the server with your TIBCO credentials. Artifactory will happily come back with no results if you are **not** logged in.

Artifactory properties can be used for searches ( see What are artifactory properties ), for example, a search by milestone identifier.




Artifactory can also be browsed outside of the GUI using http://ep-artifacts-us-east.tibco.com/artifactory/list/ep-snapshots/.

## Where can I find the latest installer that passed tests ?

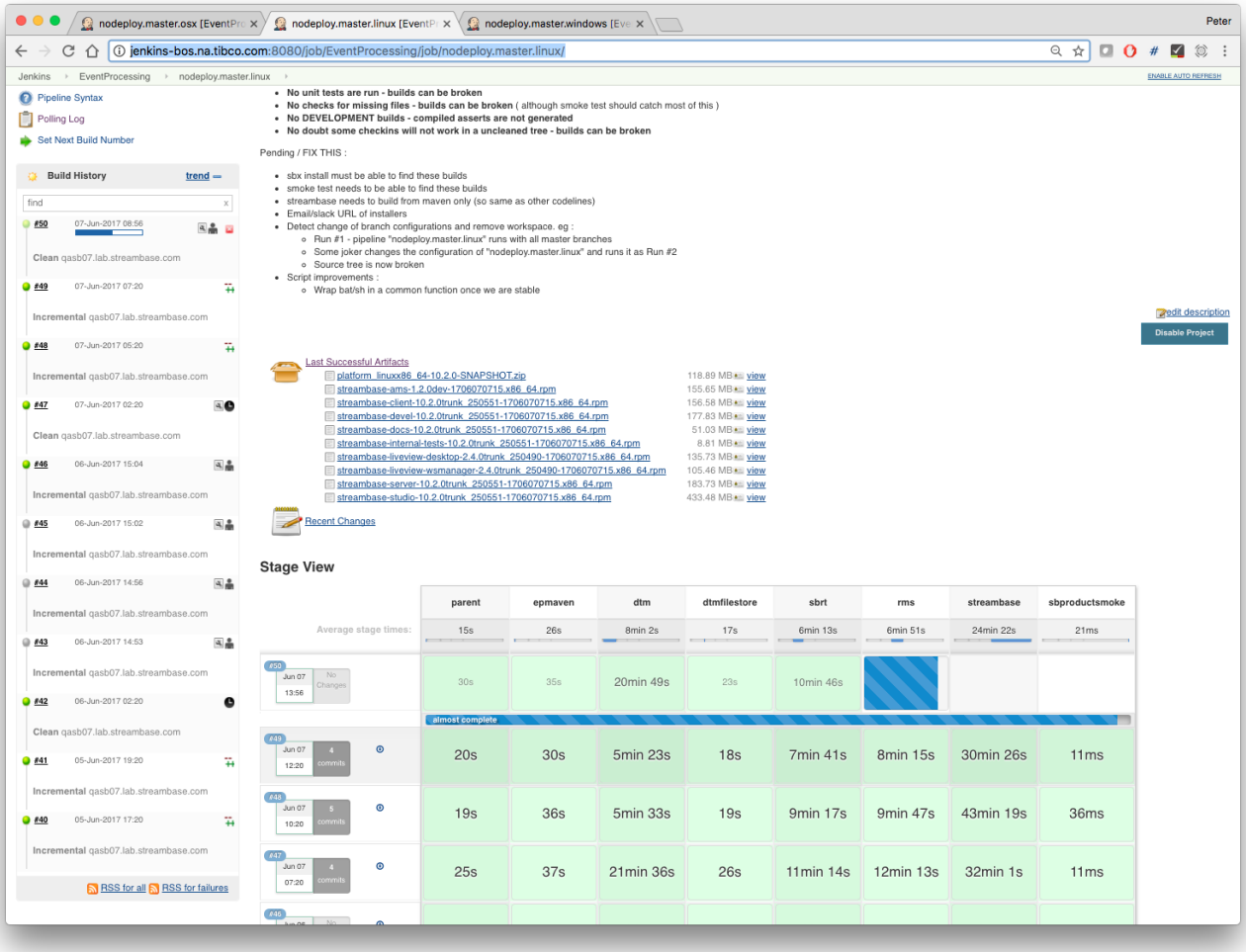
Use the search function to find artifacts with property **milestone.latest=OS** and **milestone.complete=true.** The search can further be contained by choosing the **ep-product-installers-bos** repository.

See Where to find Product Builds for more information.

## Where can I find no-deploy installers ?

The no-deploy installers for `master` can be found in Jenkins :

- http://jenkins-bos.na.tibco.com:8080/job/Streaming/job/nd.master.linux/
- http://jenkins-bos.na.tibco.com:8080/job/Streaming/job/nd.master.macos/
- http://jenkins-bos.na.tibco.com:8080/job/Streaming/job/nd.master.windows/




`sbx install` understands no-deploy job names as a version to install.

## Where can I find internal site documentation builds ?

Maven site documentation is deployed at http://ep-artifacts-us-east.tibco.com/site/. The following special links are maintained :

- Latest master branch for linux - http://ep-artifacts-us-east.tibco.com/site/latest-master-linux/ (or http://ep-artifacts-us-east.tibco.com/site/latest-linux/)
- Latest master branch for macos - http://ep-artifacts-us-east.tibco.com/site/latest-master-macos/ (or http://ep-artifacts-us-east.tibco.com/site/latest-macos/)
- Latest master branch for windows - http://ep-artifacts-us-east.tibco.com/site/latest-master-windows/ (or http://ep-artifacts-us-east.tibco.com/site/latest-windows/)

Maven sites are also linked at the build distribution page.

## Where can I find released site documentation ?

Maven sites for released products are found at http://ep-dist.tibco.com/doc/maven-site/ – older versions used to be posted to DevZone.

## Where can I find Maven artifacts ?

Maven artifacts are deployed to Artifactory. Maven should be configured with a settings.xml file to reference the nearest server, which you can override with `repo.server` property. That property is sometimes referenced in Maven projects as well.

## Can I use sbx to download any milestone installer ?

Yes. Use **sbx install** using the **--milestone** parameter. For example:

```
sbx install --milestone streambase-101-linux-61
```

Using sbx install with the `--milestone` option does not download installers from Artifactory and is strongly discouraged. The installers being accessed may have not been successfully tested.

### Can I use sbx to download the latest installer ?

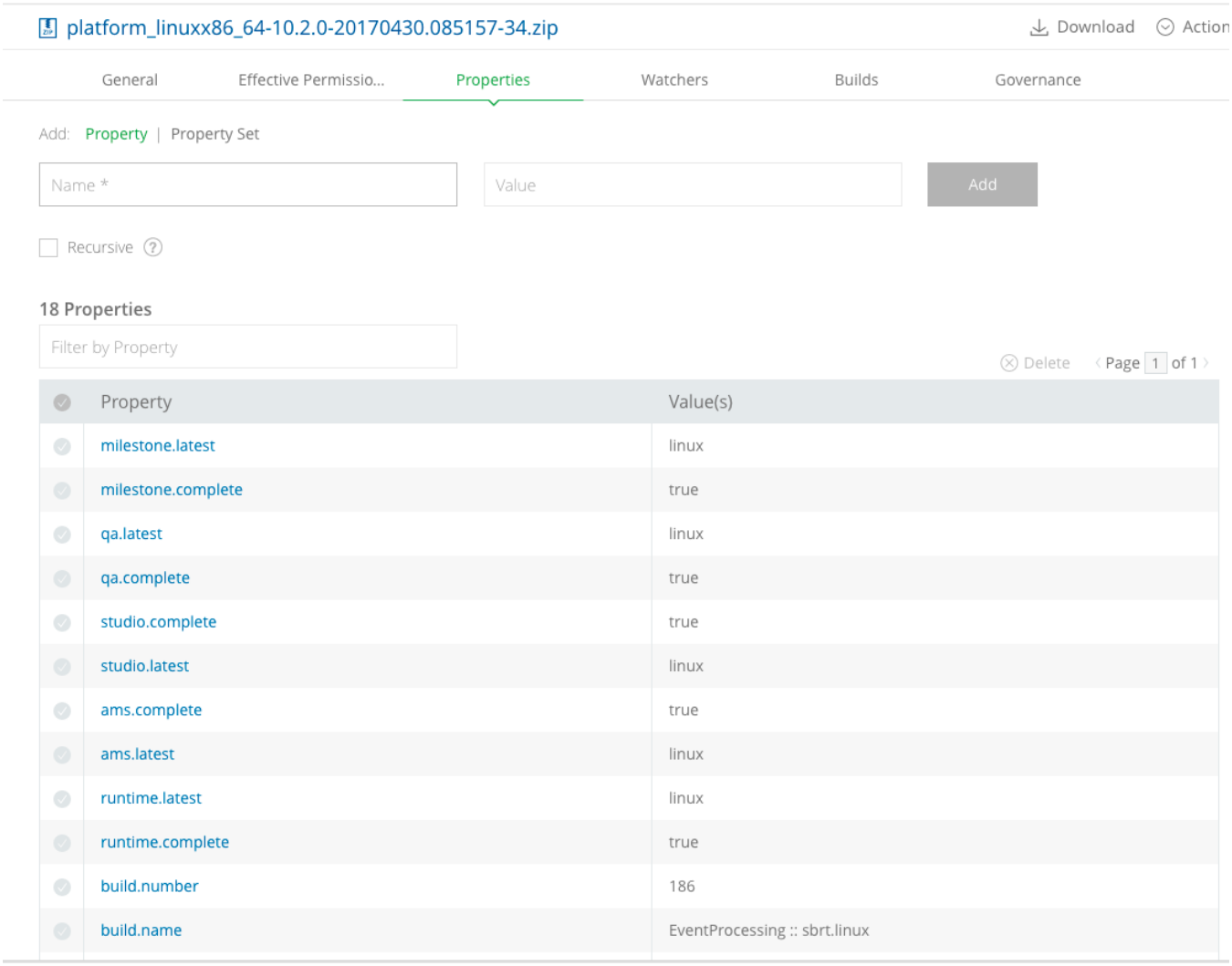
Yes, `sbx install` understands Maven versions like `11.0.0-SNAPSHOT`; see Where To Find Product Builds for more details.

# Using Artifactory

### What are artifactory properties ?

Artifacts in artifactory contain properties.  Beyond the standard properties we've defined :

- Runtime pipeline specific :
  - **runtime.complete** - set to true if this artifact is part of a completely successful runtime build
  - **runtime.latest** - set to either linux, macos or windows.  There is one set of artifacts with **runtime.latest** per branch
- Ams pipeline specific :
  - **ams.complete** - set to true if this artifact is part of a completely successful ams build
  - **ams.latest** - set to either linux, macos or windows.  There is one set of artifacts with **ams.latest** per branch
- Studio pipeline specific :
  - **studio.complete** - set to true if this artifact is part of a completely successful studio build
  - **studio.latest** - set to either linux, macos or windows.  There is one set of artifacts with **studio.latest** per branch
- QA pipeline specific :
  - **qa.complete** - set to true if this artifact is part of a completely successful qa build
  - **qa.latest** - set to either linux, macos or windows.  There is one set of artifacts with **qa.latest** per branch
- General :
  - **milestone** - milestone identifier
  - **milestone.complete** - set to true if this artifact is part of a completely successful build
  - **milestone.keep** - set to true to keep this build (ie don't automatically remove)
- **milestone.latest** - set to either linux, macos or windows.  There is one set of artifacts with **milestone.latest** per branch
  - This is set in the final stages of the qa pipelines - so if qa fails or doesn't complete, **milestone.latest** is set to **none**




## How are Artifactory properties set ?

The **milestone** property can be set via a matrix parameter on deploy - for example by using a deployment URL such as http://artifacts.tibco.com/pete-virtual-test;milestone=streambase-master-linux-83;

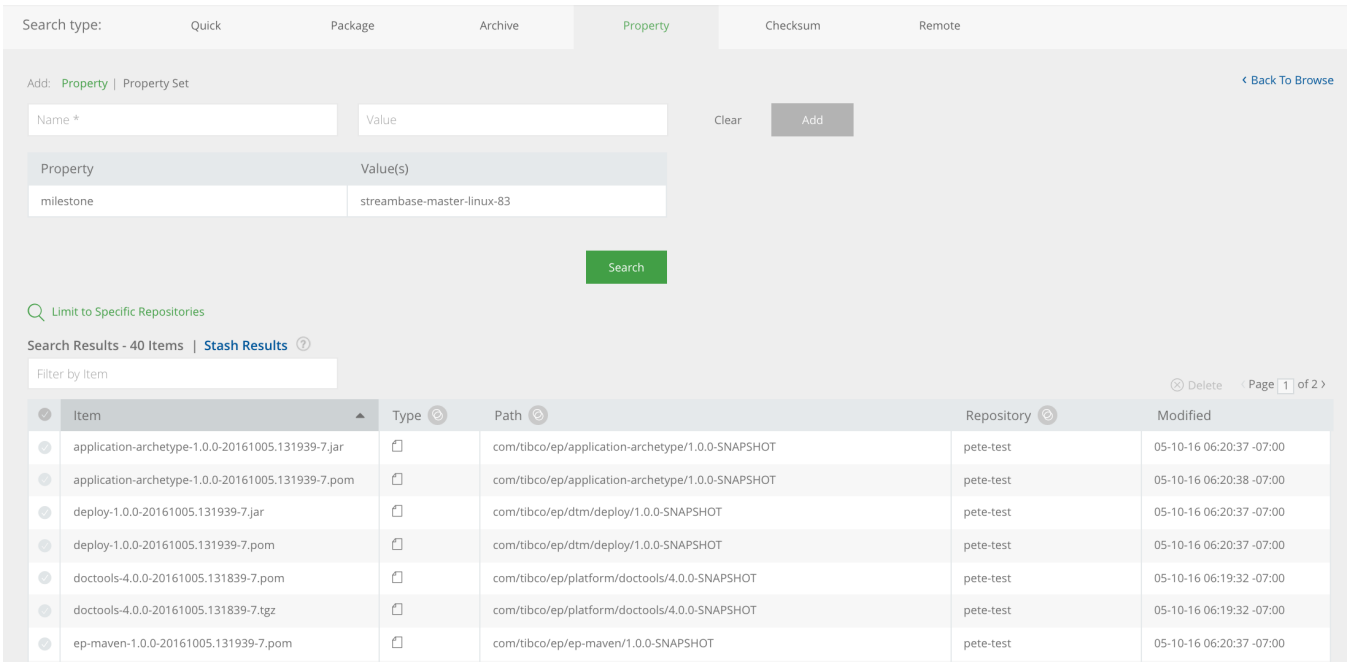
Likewise, when retrieving artifacts from artifactory, using the same URL will fetch artifacts containing that property. If the requested artifact exists in the repository but no artifact matches the value of the milestone property, then the artifact is not found. Hence simply including **;milestone=identifer;** in the repository URL is sufficient to use a consistent set of artifacts.

Note, however, the matching rules are such that if an artifact doesn't have the property at all (which will be the case for 3rdparty dependencies) then the latest will match. So we must avoid deploying our artifacts with milestone unset.

The **milestone.complete** property is set by the setproperty build as the last step in the pipeline build.

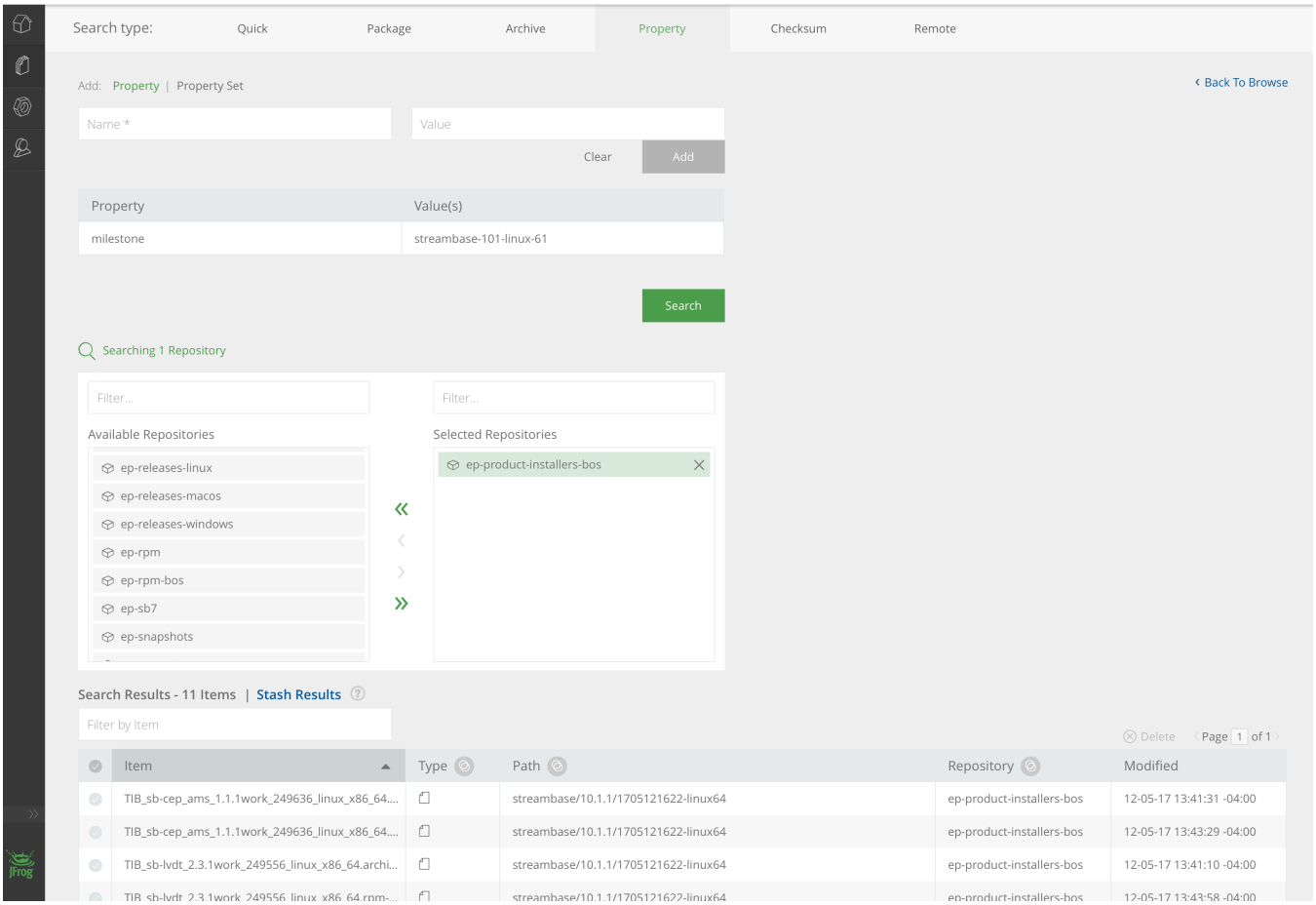
## How do I find artifacts belonging to a specific milestone ?

Use artifactory to search by property :




## How do I find installers belonging to a specific milestone ?

Installers are stored in the ep-product-installers-bos repository. They can be searched using a milestone identifier as described here, with the search limited to the installer repository.




The installer can be downloaded directly from Artifactory or using **sbx install** using the **--milestone** parameter. For example:

```
sbx install --milestone streambase-101-linux-61
```



Using sbx install without the --milestone option does not download installers from Artifactory and is strongly discouraged. The installer's being accessed have not been successfully tested.

## Can I deploy to Artifactory from my desktop ?

Pretty much always we want Jenkins to do the deployment.  However there are some edge cases where we do want to deploy from a desktop - one example is running a test deploy of site documentation to validate links.

Note that in the manual case, the UEP credentials are the same as you TIBCO credentials. You might need to ask an Artifactory administrator to give you deployment rights.

To do this the username and password for the server has to be added to you Maven settings.xml :

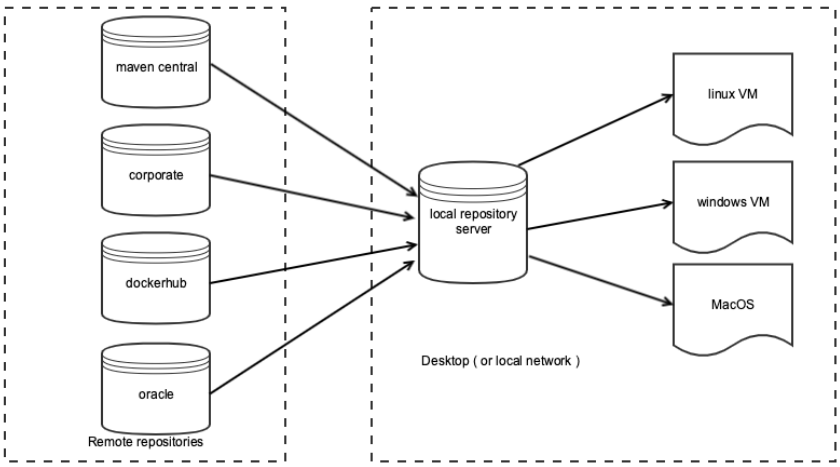
```
<servers>
        <server>
                <id>UEP</id>
        <server>
            <id>UEP</id>
            <!-- These is an actual Unix account in the dev main and thus the Artifactory server. -->
            <username>-TIBCO-account-</username>
            <!-- Supply this property on the command line. -->
            <password>${UEP.deployment.password}</password>
        </server>
        </server>
</servers>
```

## How can I set up a local repository server ( and why should I ) ?

Generally speaking, a local repository server is not needed since Maven will download artifacts automatically from the network into the local maven repository (~/.m2/repository).  However it can be useful in certain cases such as :

- Multiple isolated builds - eg using vagrant for cross-platform development ( shown below ) and multiple branch builds
- Removing the local repository - eg testing releases
- Automatic going offline - eg moving from good connectivity to poor
- Multiple desktops - eg cross platform testing with multiple local servers, classroom use, demo use
- Testing devops ( local repository server can be deployed to )
- Testing shared repository ( or subsequent changes )
- Mixture of the above plus slow internet connection
- Unreliable internet connection
- Overnight syncing from corporate servers

So any remote artifact is downloaded once to the local repository server and can be used locally many times.  For example :



Running nexus in a docker container is a good choice since the nexus filesystem format matches the way we ship maven artifacts (so its easy to add GA artifacts). The process is :

```
datadir=$HOME/nexus-data
artifacts="$HOME/Desktop/TIBCO Streaming 10.4.0/sdk/maven/product-artifact/str-10.4.0-artifacts.zip"

mkdir -p ${datadir}/storage/releases
unzip "${artifacts}" -d ${datadir}/storage/releases

docker run -d -p 8081:8081 --name nexus -v ${datadir}:/sonatype-work sonatype/nexus:oss
```

Maven will use this with the following `~/.m2/settings.xml` file :

```
<settings>
        <mirrors>
                <mirror>
                        <id>nexus</id>
                        <mirrorOf>*</mirrorOf>
                        <url>http://localhost:8081/nexus/content/groups/public</url>
                </mirror>
        </mirrors>
</settings>
```

Note for sharing the repository outside of the local machine, the hostname should be used instead of localhost.

Whilst this should be enough for end-user needs, the web UI at http://localhost:8081/nexus/ ( with userid admin / password admin123 ) can be used to add additional repositories as needed to support internal development. I found I needed to add :

- proxy of http://ep-artifacts-us-east.tibco.com/artifactory/ep-3rdparty/
- proxy of http://ep-artifacts-us-east.tibco.com/artifactory/ep-releases/
- proxy of http://ep-artifacts-us-east.tibco.com/artifactory/ep-snapshots/
- proxy of http://ep-artifacts-us-east.tibco.com/artifactory/oracle-maven/
- proxy of http://jcenter.bintray.com/

See http://www.vogella.com/tutorials/Nexus/article.html#p2-nexus-plugins on how to add p2 repository. I added additional p2 repositories :

- proxy of https://download.eclipse.org/releases/2018-12/201812191000
- proxy of https://repo1.maven.org/maven2/.m2e/connectors/m2eclipse-egit/0.15.1/N/0.15.1.201806191431/
- proxy of https://hermes.streambase.com/mirrors/eclipse/repos/subversive/4.0.5.I20170425-1700/
- proxy of https://hermes.streambase.com/mirrors/p2/repository.takari.io/content/sites/m2e.extras/m2eclipse-buildhelper/0.15.0/N/0.15.0.201207090124

| Repository | Type | IQ Polic... | Health Check | Format | Policy | Repository S... | Repository Path |
|---|---|---|---|---|---|---|---|
| **Public Repositories** | group | | ANALYZE | maven2 | | | http://localhost:8081/nexus/content/groups/public |
| 3rd party | hosted | | ANALYZE | maven2 | Release | In Service | http://localhost:8081/nexus/content/repositories/thirdparty |
| Apache Snapshots | proxy | | ANALYZE | maven2 | Snapshot | In Service | http://localhost:8081/nexus/content/repositories/apache-snapshots |
| boston-3rdparty | proxy | | 0 0 | maven2 | Release | In Service | http://localhost:8081/nexus/content/repositories/boston-3rdparty |
| boston-releases | proxy | | 0 0 | maven2 | Release | In Service | http://localhost:8081/nexus/content/repositories/boston-releases |
| boston-snapshots | proxy | | ANALYZE | maven2 | Snapshot | In Service | http://localhost:8081/nexus/content/repositories/boston-snapshots |
| Central | proxy | | 335 748 | maven2 | Release | In Service | http://localhost:8081/nexus/content/repositories/central |
| Central M1 shadow | virtual | | ANALYZE | maven1 | Release | In Service | http://localhost:8081/nexus/content/shadows/central-m1 |
| eclipse-2018-12-201812191000 | proxy | | 0 0 | p2 | | In Service | http://localhost:8081/nexus/content/repositories/eclipse-2018-12-201812191000 |
| jcenter | proxy | | 11 4 | maven2 | Release | In Service | http://localhost:8081/nexus/content/repositories/jcenter. |
| m2eclipse-egit-0.15.1 | proxy | | 0 0 | p2 | | In Service | http://localhost:8081/nexus/content/repositories/m2eclipse-egit-0.15.1 |
| Releases | hosted | | ANALYZE | maven2 | Release | In Service | http://localhost:8081/nexus/content/repositories/releases |
| Snapshots | hosted | | ANALYZE | maven2 | Snapshot | In Service | http://localhost:8081/nexus/content/repositories/snapshots |
| subversive-4.0.5.I20170425-1700 | proxy | | 0 0 | p2 | | In Service | http://localhost:8081/nexus/content/repositories/subversive-4.0.5.I20170425-1700 |
| takari-m2eclipse-buildhelper-0.15.0 | proxy | | 0 0 | p2 | | In Service | http://localhost:8081/nexus/content/repositories/takari-m2eclipse-buildhelper-0.15.0 |

Giving a settings.xml with p2 :

```
<settings>
        <localRepository>${env.TIBCO_EP_HOME}/BUILD/repository</localRepository>
        <mirrors>
                <mirror>
                        <id>nexus</id>
                        <mirrorOf>*</mirrorOf>
                        <url>http://localhost:8081/nexus/content/groups/public</url>
                </mirror>
        </mirrors>

        <profiles>
                <profile>
                        <id>eclipse</id>
                        <repositories>
                                <repository>
                                        <id>sbrt-eclipse</id>
                                        <layout>p2</layout>
                                        <url>http://localhost:8081/nexus/content/repositories/eclipse-2018-12-
201812191000/</url>
                                </repository>
                                <repository>
                                        <id>sb-eclipse-base</id>
                                        <url>http://localhost:8081/nexus/content/repositories/eclipse-2018-12-
201812191000</url>
                                        <layout>p2</layout>
                                </repository>
                                <repository>
                                        <id>sb-subversive</id>
                                        <url>http://localhost:8081/nexus/content/repositories/subversive-4.0.5.
I20170425-1700</url>
                                        <layout>p2</layout>
                                </repository>
                                <repository>
                                        <id>m2eclipse-buildhelper</id>
                                        <url>http://localhost:8081/nexus/content/repositories/takari-m2eclipse-
buildhelper-0.15.0</url>
                                        <layout>p2</layout>
                                </repository>
                                <repository>
                                        <id>m2eclipse-egit</id>
                                        <url>http://localhost:8081/nexus/content/repositories/m2eclipse-egit-
0.15.1</url>
                                        <layout>p2</layout>
                                </repository>
                        </repositories>
                </profile>
        </profiles>

        <activeProfiles>
                <activeProfile>eclipse</activeProfile>
        </activeProfiles>

</settings>
```

Here is an early script to start and configure nexus3 :

```
#!/bin/sh

datadir="/Volumes/LaCieShare/nexus3-data"
port=8081
adminpassword=admin123
tibcoserver=http://ep-artifacts-us-east.tibco.com/artifactory
tibcoserver=http://ep-artifacts-us-east.tibco.com/artifactory

mkdir -p ${datadir}
```

```
docker run -d -p ${port}:8081 -p 8181:8181 --name nexus3 --restart always -v "${datadir}:/nexus-data" sonatype
/nexus3

# wait for nexus to be up
while ! curl --silent http://localhost:${port} -o /dev/null
do
        sleep 1
done

# change password
oldadminpassword=$(docker exec nexus3 cat /nexus-data/admin.password)
if [ "${oldadminpassword}" != "" ]
then
        curl --silent --request PUT --user admin:${oldadminpassword} "http://localhost:${port}/service/rest/v1
/security/users/admin/change-password" -H "accept: application/json" -H "Content-Type: text/plain" -d
"${adminpassword}"
fi

# helm
curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/helm/hosted" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\": \"
helm\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"strictContentTypeValidation\": true,
\"writePolicy\": \"allow\" }}"

# container
curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/docker/proxy" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\": \"
dockerhub-proxy\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"
strictContentTypeValidation\": true, \"writePolicy\": \"ALLOW\" }, \"cleanup\": null, \"docker\": { \"
v1Enabled\": false, \"forceBasicAuth\": true }, \"dockerProxy\": { \"indexType\": \"HUB\" }, \"proxy\": { \"
remoteUrl\": \"https://registry-1.docker.io\", \"contentMaxAge\": 1440, \"metadataMaxAge\": 1440 }, \"
negativeCache\": { \"enabled\": true, \"timeToLive\": 1440 }, \"httpClient\": { \"blocked\": false, \"
autoBlock\": true }}"
curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/docker/hosted" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\":
\"container\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"
strictContentTypeValidation\": true, \"writePolicy\": \"allow\" }, \"docker\": { \"v1Enabled\": false, \"
forceBasicAuth\": false, \"httpPort\": null, \"httpsPort\": null }}"
curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/docker/group" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\": \"
container-group\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"
strictContentTypeValidation\": true, \"writePolicy\": \"allow\" }, \"docker\": { \"v1Enabled\": false, \"
forceBasicAuth\": false, \"httpPort\": 8181, \"httpsPort\": null }, \"group\": { \"memberNames\": [ \"
container\", \"dockerhub-proxy\" ] }}"
curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1/security
/roles" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"id\": \"push\", \"name\": \"
push\", \"description\": \"push\", \"privileges\": [ \"nx-repository-view-docker-registry-*\" ]}"
curl --silent --request PUT --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1/security
/users/anonymous" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"userId\": \"
anonymous\", \"firstName\": \"Anonymous\", \"lastName\": \"User\", \"emailAddress\": \"anonymous@example.org\",
\"source\": \"default\", \"status\": \"active\", \"roles\": [ \"nx-anonymous\", \"push\" ]}"
curl --silent --request PUT --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1/security
/realms/active" -H "accept: application/json" -H "Content-Type: application/json" -d "[ \"
NexusAuthenticatingRealm\", \"NexusAuthorizingRealm\", \"DockerToken\"]"

# python
curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/pypi/proxy" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\": \"
pypi-proxy\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"strictContentTypeValidation\":
true }, \"proxy\": { \"remoteUrl\": \"https://pypi.org/\", \"contentMaxAge\": 1440, \"metadataMaxAge\": 1440 },
\"negativeCache\": { \"enabled\": true, \"timeToLive\": 1440 }, \"httpClient\": { \"blocked\": false, \"
autoBlock\": true }}"

# jcenter
curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/maven/proxy" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\": \"
maven-jcenter\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"
strictContentTypeValidation\": true }, \"proxy\": { \"remoteUrl\": \"https://jcenter.bintray.com/\", \"
contentMaxAge\": 1440, \"metadataMaxAge\": 1440 }, \"negativeCache\": { \"enabled\": true, \"timeToLive\": 1440
}, \"httpClient\": { \"blocked\": false, \"autoBlock\": true, \"connection\": { \"retries\": 0, \"
```

```
userAgentSuffix\": \"string\", \"timeout\": 60, \"enableCircularRedirects\": false, \"enableCookies\": false }
}, \"maven\": { \"versionPolicy\": \"MIXED\", \"layoutPolicy\": \"STRICT\" }}"

# tibco proxies
for repo in ep-3rdparty ep-releases ep-snapshots ep-rc-linux ep-rc-macos rp-rc-windows ep-releases-linux ep-
releases-macos ep-releases-windows oracle-maven-cache
do
        # curl --silent --request DELETE --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/tibco-${repo}" -H "accept: application/json"
        curl --silent --request POST --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/maven/proxy" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\": \"
tibco-${repo}\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"
strictContentTypeValidation\": true }, \"proxy\": { \"remoteUrl\": \"${tibcoserver}/${repo}\", \"
contentMaxAge\": 1440, \"metadataMaxAge\": 1440 }, \"negativeCache\": { \"enabled\": true, \"timeToLive\": 1440
}, \"httpClient\": { \"blocked\": false, \"autoBlock\": true, \"connection\": { \"retries\": 0, \"
userAgentSuffix\": \"string\", \"timeout\": 60, \"enableCircularRedirects\": false, \"enableCookies\": false }
}, \"maven\": { \"versionPolicy\": \"MIXED\", \"layoutPolicy\": \"STRICT\" }}"
done

# group
curl --silent --request DELETE --user admin:${adminpassword} "http://localhost:${port}/service/rest/v1
/repositories/public" -H "accept: application/json"
curl --silent --request POST --user admin:${adminpassword}  "http://localhost:${port}/service/rest/v1
/repositories/maven/group" -H "accept: application/json" -H "Content-Type: application/json" -d "{ \"name\": \"
public\", \"online\": true, \"storage\": { \"blobStoreName\": \"default\", \"strictContentTypeValidation\":
true }, \"group\": { \"memberNames\": [ \"tibco-ep-snapshots\", \"maven-central\", \"tibco-ep-3rdparty\", \"
maven-jcenter\", \"tibco-ep-releases-macos\", \"tibco-ep-releases-linux\", \"tibco-ep-rc-macos\", \"tibco-ep-rc-
linux\", \"tibco-ep-releases\", \"tibco-oracle-maven-cache\" ] }}"
```

## How do we publish Maven plugins to Maven Central ?

We have created a OSSRH (a Maven Central staging service - see https://central.sonatype.org/pages/ossrh-guide.html) namespace *com.tibco.ep* for publishing artifacts. At least one account needs to be registered with publishing rights at all times. Such accounts can:

- Deploy snapshot artifacts into repository https://oss.sonatype.org/content/repositories/snapshots
- Deploy release artifacts into the staging repository https://oss.sonatype.org/service/local/staging/deploy/maven2
- Promote staged artifacts into repository 'Releases'
- Download snapshot and release artifacts from group https://oss.sonatype.org/content/groups/public
- Download snapshot, release and staged artifacts from staging group https://oss.sonatype.org/content/groups/staging

Once you have an account, you can log in at https://oss.sonatype.org to see the status of things. You should create yourself an access token, which is username/password credential that doesn't use your user ID or password, so it's safer to put in configuration files. It can be used for both read and write access to our staging repositories.

The managed Maven settings stored in Jenkins under the name `CommonSettings` contains deployment credentials from such an access token.

Artifacts for Maven Central to be signed by `gpg` - see https://central.sonatype.org/pages/working-with-pgp-signatures.html.  For a local build, `gpg` needs to be set up with a key associated with your user id with `tibco.com` mail address.

We deploy just one project to Maven Central, the EP maven plugin. This project and any others we might deploy to Maven Central need to follow the deployment rules as documented at https://central.sonatype.org/pages/apache-maven.html – most notably, the specification of project metadata such as the where the sources are located, organization contacts, and so on.

Release versions of such projects must be performed on a branch *off* master. A branch following our usual conventions like `release-work/2.1.5` should be used.

Deployment to OSSRH command is driven by `mvn -Doss-stage.skip=false deploy`. Maven builds like tibco-streaming-maven-plugin should be designed that only the artifacts that need to be deployed are deployed – before careful about parent pom references. Two helpful techniques are to use the Maven flatten plugin, and to start off with the `maven.deploy.skip` as true at the top of the build and then set it to false for projects that need to go into Maven Central.

The Jenkins job for ep-maven-plugin cannot currently deploy to OSSRH – see JIRA Issue SB-51371. You'll have to do it locally.

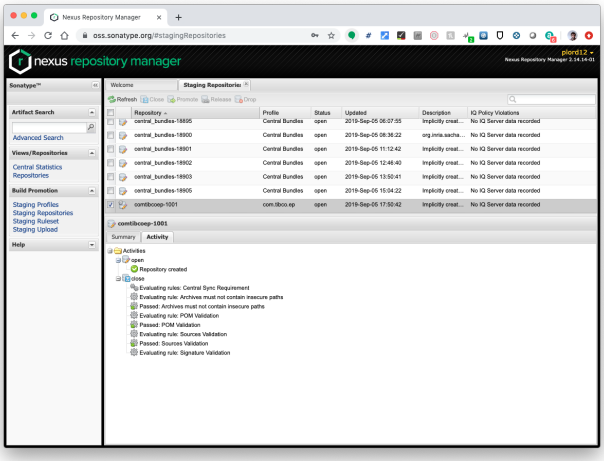
Releases are initially placed in temporary repository with a generated name, for example https://oss.sonatype.org/service/local/repositories/comtibcoep-1001/content/; access requires the OSSRH account credentials.

The internal Artifactory repository http://ep-artifacts-us-east.tibco.com/artifactory/list/sonatype-epmaven-staging/ can be used to access this OSSRH repository: it should normally work to change the URL when a new staging effort is underway. If the remote repository needs be created, you need to specify:

1. The generated URL from the deployment.
2. The OSSRH credentials for username and password.
3. Uncheck "handle snapshots" since we're just interested in releases.
4. Include the artifact patterns `com/tibco/ep-maven-plugin/**/*` and `com/tibco/ep/ep-maven-interface/**/*`

5. Add the `sonatype-epmaven-staging` repository to the `ep-maven-virtual`, `ep-rc-linux-virtual`, `ep-rc-macos-virtual` and `ep-rc-windows-virtual` repositories to give users and release candidate builds transparent access. Unfortunately, if `sonatype-epmaven-staging` this needs to be recreated, you need to visit the page for each of these virtual repositories in the JFrog Artifactory administrative interface and add it.

The temporary OSSRH repository can be closed and released via the GUI ( https://oss.sonatype.org/#stagingRepositories ) or on the Maven command line - at which point they are published to Maven Central.  The close process performs a number of checks on the artifacts.




Note we have seen that the staging repository can timeout (or auto drop) - there doesn't seem to be documentation on this behaviour, but I guess it makes sense.  If this happens, the plugin needs to be re-deployed to the staging area.

The signed jar can be verified locally with: *(not sure if this is true – SKS might be going away):*

```
$ gpg --keyserver hkp://pool.sks-keyservers.net --recv-keys 28D6DAD3
( or gpg --keyserver hkp://51.38.91.189 --recv-keys 28D6DAD3 if on a Mac )
$ gpg --verify ep-maven-plugin-1.5.0.jar.asc
```

Snapshots are available at https://oss.sonatype.org/content/repositories/snapshots/com/tibco/ep/, but ordinarily we just deploy to `ep-snapshots` in Artifactory as part of the polled github-epmaven builds.

# Managing Sources For A Build

## How Should I Define Pipelines?

Documentation for this is spotty, as coming up with best practices is still an adventure, but the best thing to do is look at existing pipelines.

Pipelines should be kept under source control; there is a "from SCM" option for pipelines. By conventions, the repository that is most essential to a pipeline (if there's more than one codeline, you must choose) is kept in `.EPDev/build-management/pipelines/`*`name`*`/Jenkinsfile`. This scheme allows for a single repository to participate in multiple pipelines. Newer pipelines leave more configurable fields undefined and then compute defaults based on pipeline naming conventions and other support functions in the `ep-jenkins-workflow` support library. Distinct projects should use separate Jenkins folders from the beginning.

Pipelines that are based on Streaming builds tend to look for an upstream pipeline; there is helper code for inferring what this should be from the `pom.xml` of the first codeline. Note that all pipelines need to check out all codelines of a pipeline so that SCM polling will check all those repositories – the checkout from a worker job started by the Jenkins `build` step of a pipeline *won't* make this happen. This is not a huge overhead if the reference repository feature of Git checkout is used.

The default quiet period for builds that would start from an SCM poll is ten minutes (600 seconds). Jobs and pipelines can configure their own quiet periods.

If you write your pipeline correctly, it should not mention any host names or embedded any version numbers about the project itself.

## How Do I Specify The Credentials And URLs For Jenkins Pipelines?

Make an instance of the com.tibco.workflow.SCM class – storing it in a member or variable called `epscm` in the pipeline is a good choice to avoid clashing with Jenkins-defined classes and variables (like steps) called `scm`, and use the `getCodelineCredentialsId` and `getCodelineCredentialsUrl` functions.

The credentials for multi-branch projects will be replaced by per-organization GitHub App keys that can also specify a GitHub web hook. Otherwise, each repository on GitHub needs a new deploy key (actually an SSH key pair), which can be created and deployed to GitHub and EP Jenkins with `sbx new-github-deploy-keys`. Actual access is then over SSH instead of the https protocol that many developers tend to use. See JIRA Issue SB-50817 and related issues.

The codeline concept is just a name for the source repository, and support functions are coded to assume that, sources are in `tibco` at GitHub. The functions figure out where the source repo is located and give back the correct details accordingly. Generally the functions to figure out where a codeline really is only need to be updated for new exceptions – any codeline with a name starting with `tibco-streaming-` is assumed to be owned by the `TIBCO software` organization, for example.

## What Permissions Must I Grant On A Git Repository?

See Git Repository Conventions.

# Using Jenkins

## What is a Jenkins Executor ?

"Executor" is the Jenkins term for build servers – the term "slave" is still mentioned in documentation and plugins but is now discouraged. We have a number of executors configured; see http://jenkins-bos.na.tibco.com:8080/computer/ for the complete list for executors that are pre-configured.

Executors created from Ansible Tower should use the two-letter **`je`** host name in the self-service workflow request. The IT naming convention in US East lab is already compressed enough that we're not going to try to shoehorn any information besides the fact that the machine is a Jenkins executor. The Ansible Tower workflow will allocate a new number after "je" for the host.

Executors can also be dynamically created from cloud facilities, and we'll probably be using that in 2022 in the US East lab.

The monitoring plugin is enabled for the slaves via the **Monitoring** sidebar icon. For example http://jenkins-bos.na.tibco.com:8080/computer/na1devstrje19.dev.tibco.com/monitoring/ is the main monitoring page for the executor `na1devstrje19`.

## What are Jenkins labels ?

Most jobs and pipelines determine the eligibility of build server for tasks by using labels. Labels are simple identifiers that denote a configuration, capability or administrative property of the machine. Each label has a description on the Jenkins server. Current labels in use include:

- a.x86_64 (we might actually get to support non-Intel enterprise servers, so the architecture needs to be expressed)
- c.ansible-2.8-compatible
- c.cygwin-hardlinks
- c.dtm-config
- c.ms-toolset-v90
- c.ms-toolset-v100
- c.ms-toolset-v110
- c.ms-toolset-v120
- c.RAM-32GB-plus
- c.sbdeps
- c.subversion
- c.str-runtime
- c.tibsign-2-auth
- c.unix-tools
- c.windows-10-compatible
- c.windows-perl
- c.windows-python2.7
- http://jenkins-bos.na.tibco.com:8080/label/devtest/
- http://jenkins-bos.na.tibco.com:8080/label/linux_docker (legacy: http://jenkins-bos.na.tibco.com:8080/label/docker_linux)
- http://jenkins-bos.na.tibco.com:8080/label/linux-nodeploy/
- http://jenkins-bos.na.tibco.com:8080/label/linux/
- http://jenkins-bos.na.tibco.com:8080/label/o.macos/
- http://jenkins-bos.na.tibco.com:8080/label/o.rh6/
- http://jenkins-bos.na.tibco.com:8080/label/o.rh7/
- http://jenkins-bos.na.tibco.com:8080/label/o.rh8/
- http://jenkins-bos.na.tibco.com:8080/label/o.windows/
- http://jenkins-bos.na.tibco.com:8080/label/osx-nodeploy/ (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/osx/ (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/osx_docker (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/qa-linux/ (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/qa-osx/ (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/qa-windows/ (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/qa/ (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/q.str/
- http://jenkins-bos.na.tibco.com:8080/label/squish/ (obsolete, use c.squish)
- http://jenkins-bos.na.tibco.com:8080/label/windows_docker (legacy: http://jenkins-bos.na.tibco.com:8080/label/windocker)
- http://jenkins-bos.na.tibco.com:8080/label/windows-nodeploy/ (obsolete)

- http://jenkins-bos.na.tibco.com:8080/label/windows/ (obsolete)
- http://jenkins-bos.na.tibco.com:8080/label/c.docker-compatible-linux-3.10/

The c. labels describe capabilities. The existing Docker and QA tool labels are fine the way they were, so they were supplanted for 2019.

Note that even the plain `o.` labels imply a certain level of functionality besides the operating system. You can assume that any machine with such a label can run a reasonably sized Java job, along with any other Jenkins-installed tools (such as Maven), since every Jenkins environment is managed by Java of some version (8 and possibly newer versions later).

Some labels are easier to earn than others. The standard macOS and Linux configurations that are implied by the legacy `osx` and `linux` labels are very similar to osx-nodeploy and `linux-nodeploy` respectively, while `windows-nodeploy` requires a lot more work than those two. In late 2019 and after, we will be moving to using more capability-based labels allow for more flexible use of executors. For example, in the Streaming 10.5 branch, `windows-nodeploy` is really equivalent to the Jenkins label expression

```
c.unix-tools && c.sbdeps && squish && c.cygwin-hardlinks && c.subversion && c.windows-perl && c.windows-python2.7 && c.ms-toolset-v90 && c.ms-toolset-v100 && c.ms-toolset-v110 && c.ms-toolset-v120 && o.windows
```

whereas `linux-nodeploy` is like:

```
c.unix-tools && c.sbdeps && c.subversion && o.rh6
```

– which is a lot simpler to build.

For Microsoft Visual Studio, we use the "toolset" identifier instead of the year associated with the release, which is slightly less obvious, but has the advantage of being less ambiguous if the compiler toolset gets a new version inside a Visual Studio service pack. Newer versions that we are not yet using include:

| Toolset identifier | Visual Studio Release |
|---|---|
| v140 | Visual Studio 2015 |
| v141 | Visual Studio 2017 |
| v142 | Visual Studio 2019 |

It is also expected that these labels can be fulfilled by command-line-only installers of the toolsets, if Microsoft offers editions that meet our needs – that is, including all the tools for C++ and the .NET code that we build.

A `q.str` label should be applied to any machine that is ready (**q**ualified) for Streaming project use. All pipelines for Streaming 10.5 and after will require this label for executors, so that other labels can be applied to an executor that *describe* the executor before it's actual *ready* to be used. This approach allows executors to be online for Jenkins *and* labeled before they are actually ready, without potentially breaking other builds.

Not all production Jenkins executors have `q.str` labels. We can also have other `q.` labels so that other teams can qualify executors and then reserve them for their purposes. There are also a few machines in the Jenkins pool for legacy Kabira use that don't have a `q.str` label.

Obsolete labels should not be applied to new executors and should be removed from current executors.

## What are the main Jenkins pipeline jobs ?

We have divided build responsibilities to runtime, ams, studio, qa, cloud and performance.  Each of these has a jenkins pipeline job configured for both the master and release branches.

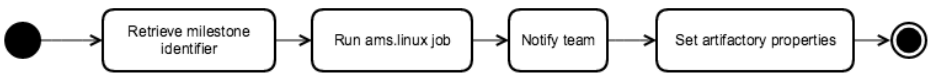
### Runtime pipelines

The runtime pipelines build the runtime components, one for each platform :



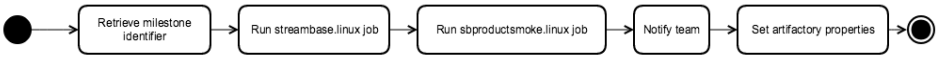

### AMS pipelines (replaced by ModelOps after Streaming 10.6)

The ams pipelines build the ams components, one for each platform :




### Studio pipelines

The Studio pipelines build the various Streaming components, including installers, Studio, tools, most adapters, and operators, one for each platform :



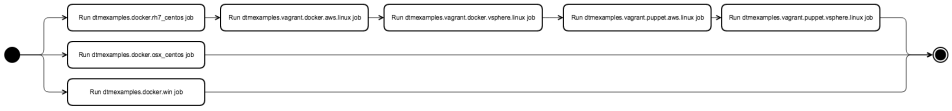

### QA pipelines

The qa pipelines build the qa components, one for each platform :



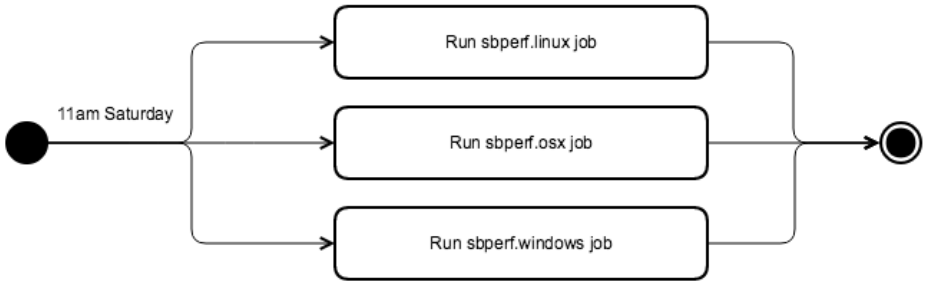

### Cloud pipeline (Currently disabled)

The cloud pipelines build the cloud components :




### Performance pipeline (Currently disabled)

The performance pipeline runs the performance test cases. It used to be run weekly on Saturdays at 11 AM ET (Jenkins "Builds periodically" cron syntax `H 11 * * 6`), but has been disabled for the time being. The pipeline runs all the supported platforms in parallel.

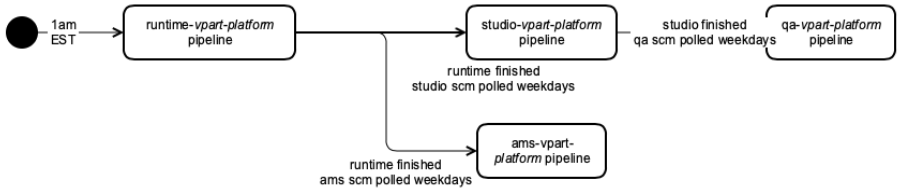



## How are the main pipeline jobs scheduled ?

- The **master** runtime pipeline is triggered by a wall-clock time. That is configured as a Build Trigger **Build periodically** schedule of `H(0-15) 1 * * 1-4`. Older runtime pipelines of maintenance branches such as `sb/main/10.6` may start later.
- The ams pipeline is triggered when the upstream runtime completes or on source code change:  Build Trigger **Poll SCM** schedule of `H 9-17/4 * * *`.
- The studio pipeline is triggered when upstream runtime completes or on source code change: Build Trigger **Poll SCM** schedule of `H 9-17/4 * * *`.
- The qa pipeline is triggered when upstream studio completes.

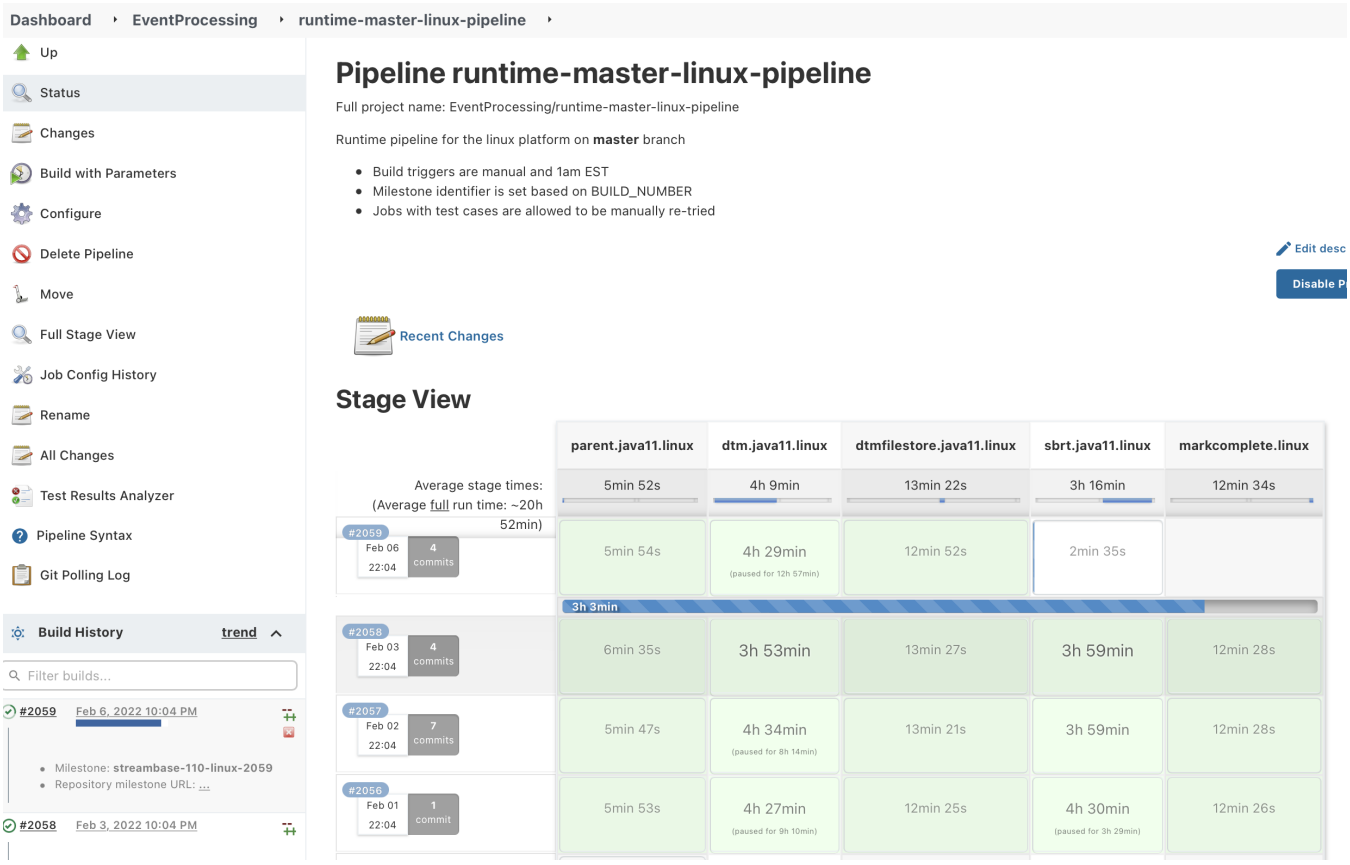
In the following diagrams, *vpart* is either the **master** branch, or, for maintenance branches, *major.minor* version with the period removed, like **10.6** becoming **106**.

## Streaming 10.6 to Current Branches




## How can I view the status of a pipeline ?

The progress of a running pipeline is available graphically on its Jenkins page:

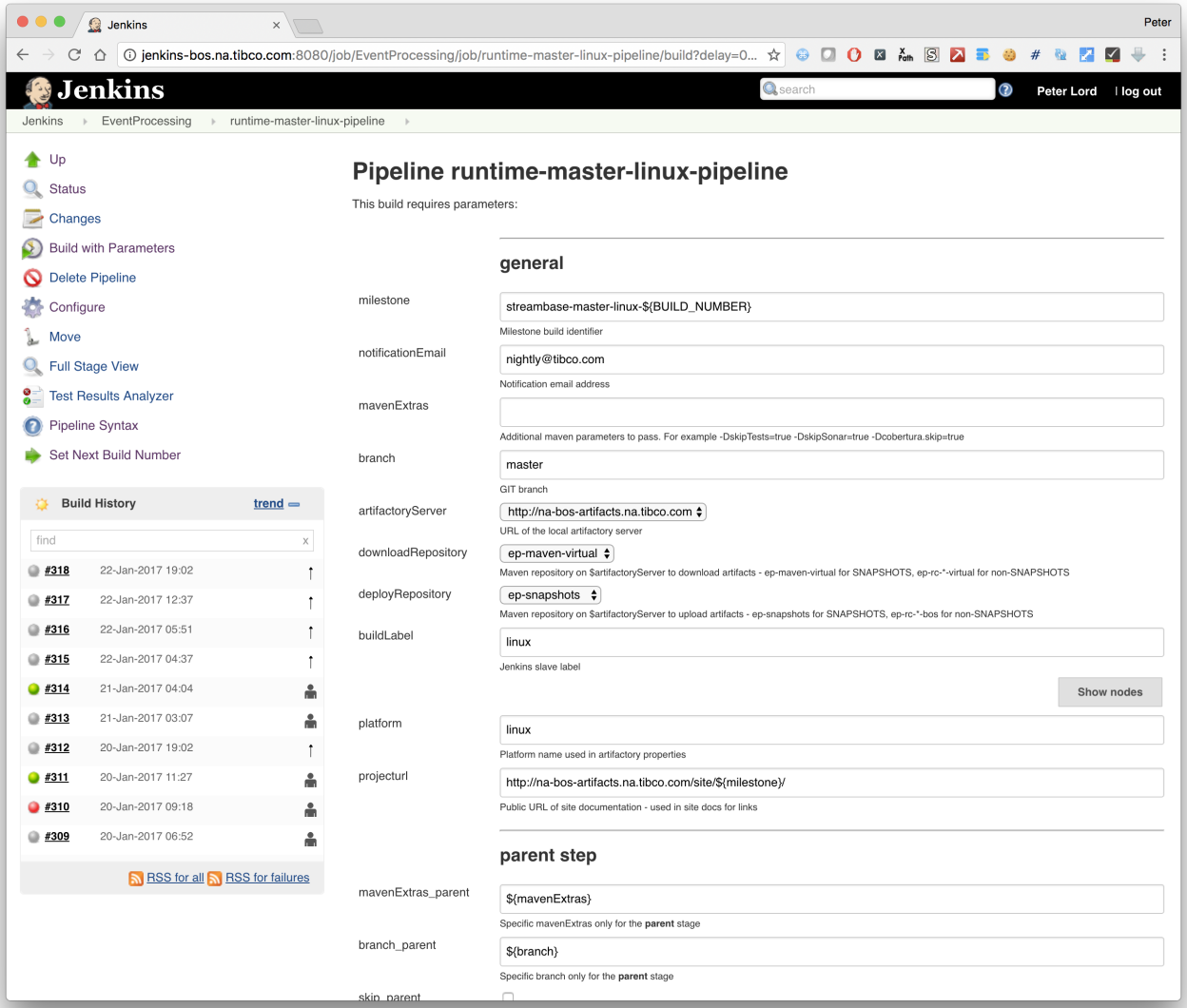



## How are old pipelines aborted ?

The jenkins plugin Pipeline Milestone Step Plugin is used to track jobs as the pipeline progresses. If a newer pipeline gets further than older pipelines i.e., the older one is stuck) then the older will be automatically aborted.

## Can I manually start a pipeline ?

Each platform pipeline can be manually started - a number of parameters can be set (the defaults are used when running automatically):

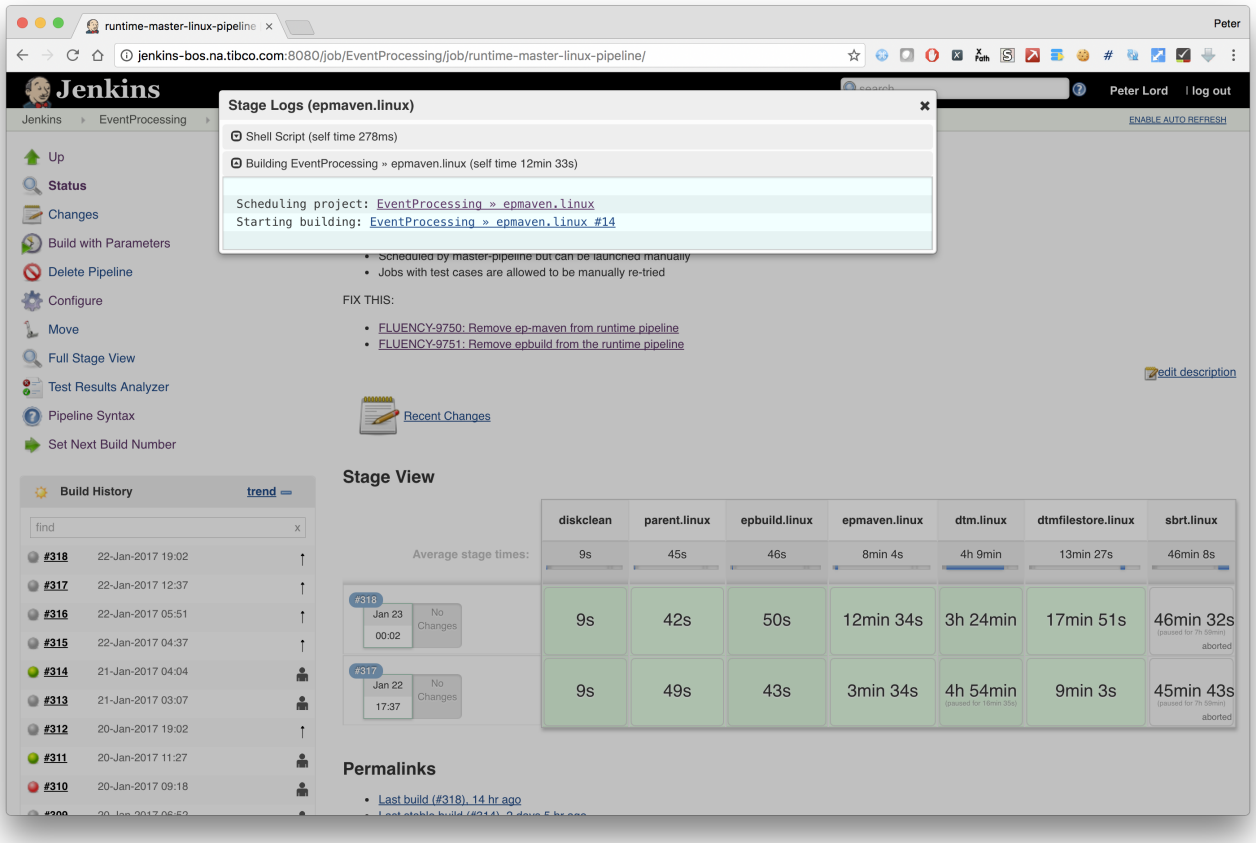



The following parameters are used :

- **builsingMilestone** - Milestone build identifier.  Always generated in the runtime pipelines but left empty in others, so that it is derived from previous pipeline
- **notificationEmail** - Notification email address.  Usually nightly@tibco.com
- **mavenExtras** - Additional maven parameters to pass.  For example -DskipTests=true -DskipSonar=true -Dcobertura.skip=true -Dsw_mkall.stripOption=-S
- **branch** - GIT or SVN branch
- **artifactoryServer** - URL of the local artifactory server
- **downloadRepository** - Maven repository on $artifactoryServer to download artifacts
- **deployRepository** - Maven repository on $artifactoryServer to upload artifacts
- **platform** - Platform name used in Artifactory properties; normally defaulted from the name of the pipeline
- **projecturl** - Public URL of site documentation - used in site docs for links; normally defaulted in the pipeline script
- **mavenExtras_*step*** - Specific mavenExtras only for the *step* stage
- **branch_*step*** - Specific branch only for the *step* stage
- **skip_*step*** - Skip building the *step* stage - requires milestone to be set.

## How can I navigate from a pipeline job to a build job ?

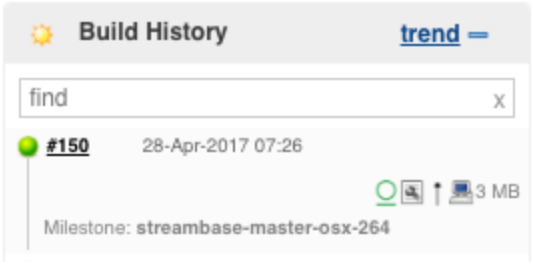
To locate a build job logs from the pipeline, first right click on the stage and click on the Logs icon :




You can also view the console log of a pipeline job - this contains useful links.

## How can I find the milestone identifier in a pipeline build ?

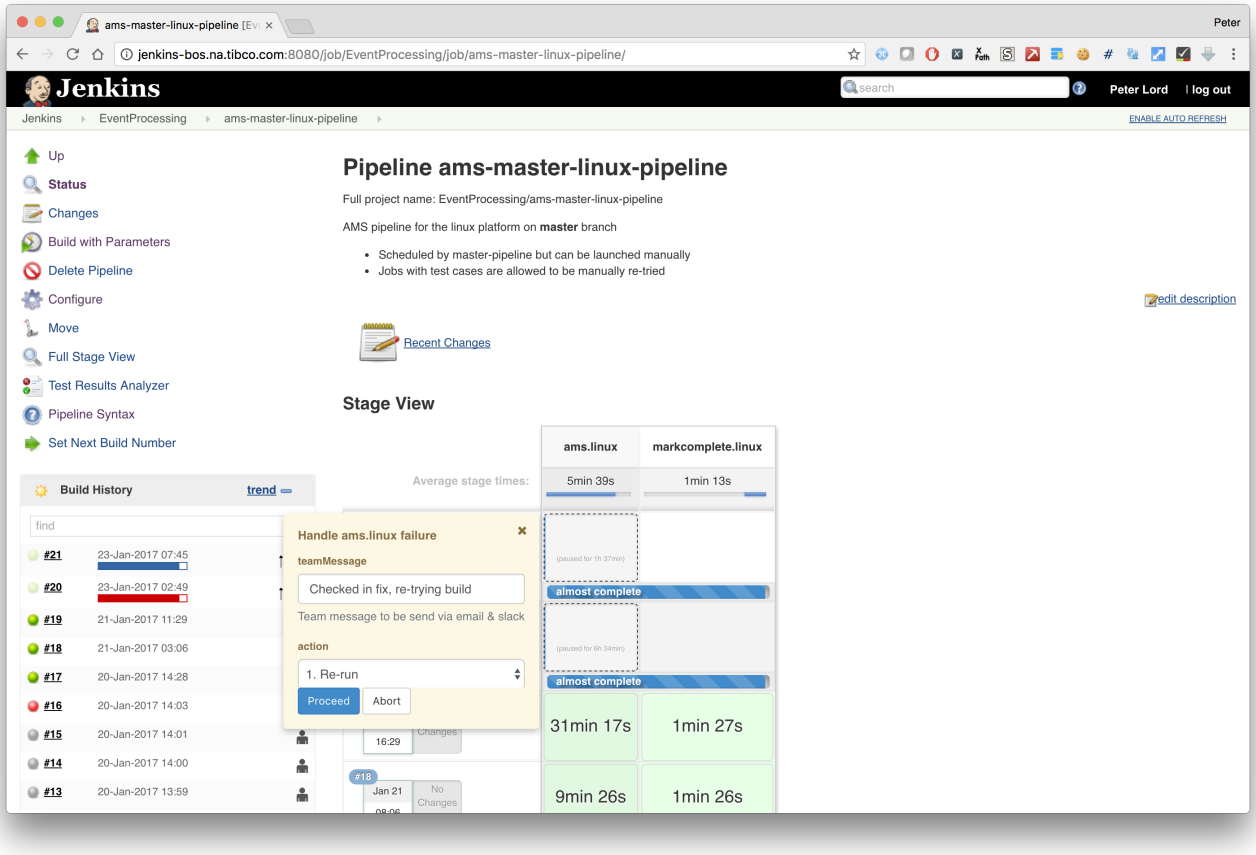
Job and pipeline descriptions are updated so that the milestone and other information is available :






## How do I resume a pipeline ?

Sometimes a build step fails - maybe there was a duplicate port or perhaps a fix needs to be checked in.  In this case the failed step can be re-tried :



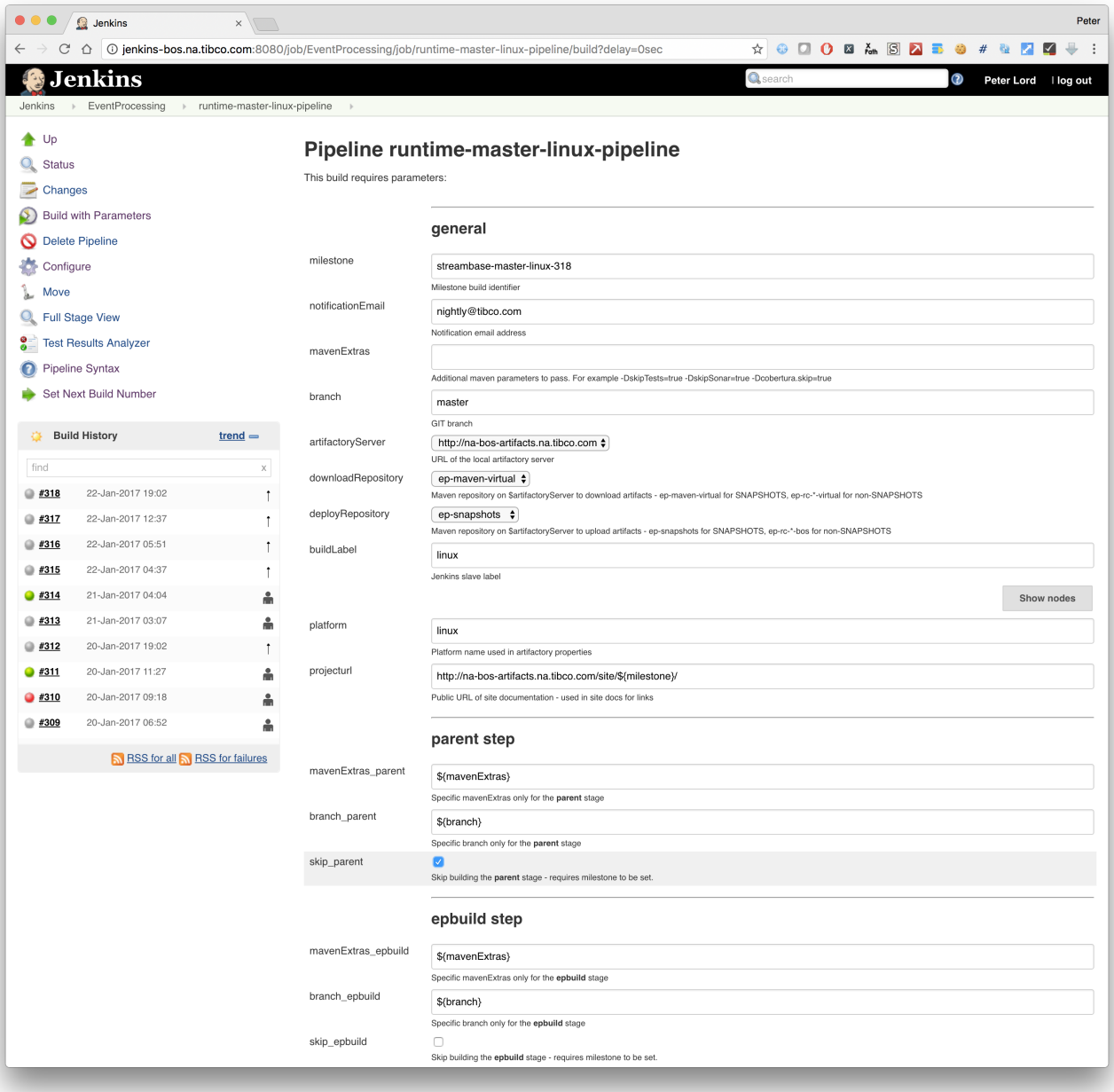

The action options are :

1. **Re-run** - job is re-run with the same parameters, including removing old workspace
2. **Re-run with tests disabled** - as above, but include -DskipTests in the maven options
3. **Deploy and continue** - attempt to deploy the previous build, so workspace is not removed and tests are skipped.  Note that this won't allow failed builds to be deployed, only failed tests

The team message is sent to colleagues via email and slack.

## Can I re-start a pipline and skip already completed steps ?

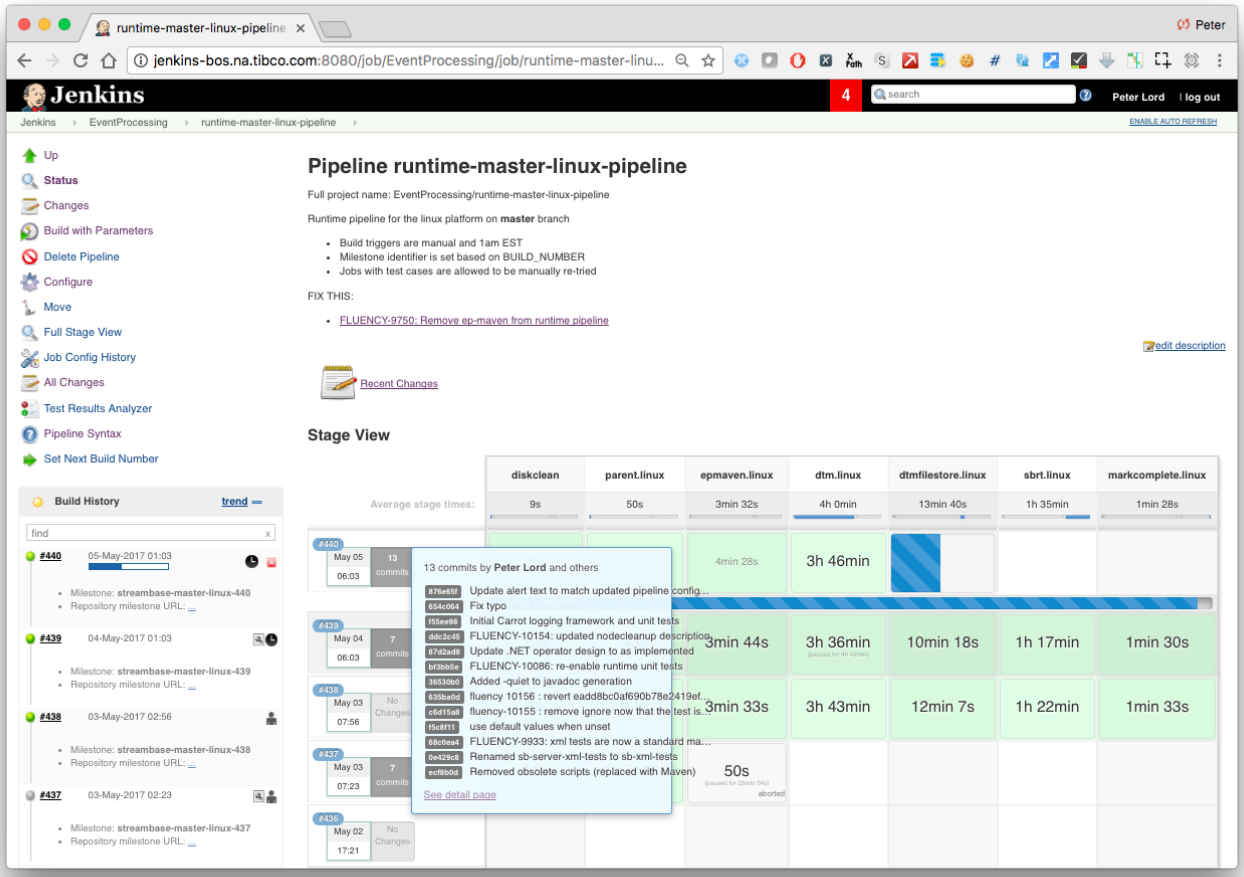
Yes.  A new pipeline can be run with the same milestone number but with the steps already completed disabled :



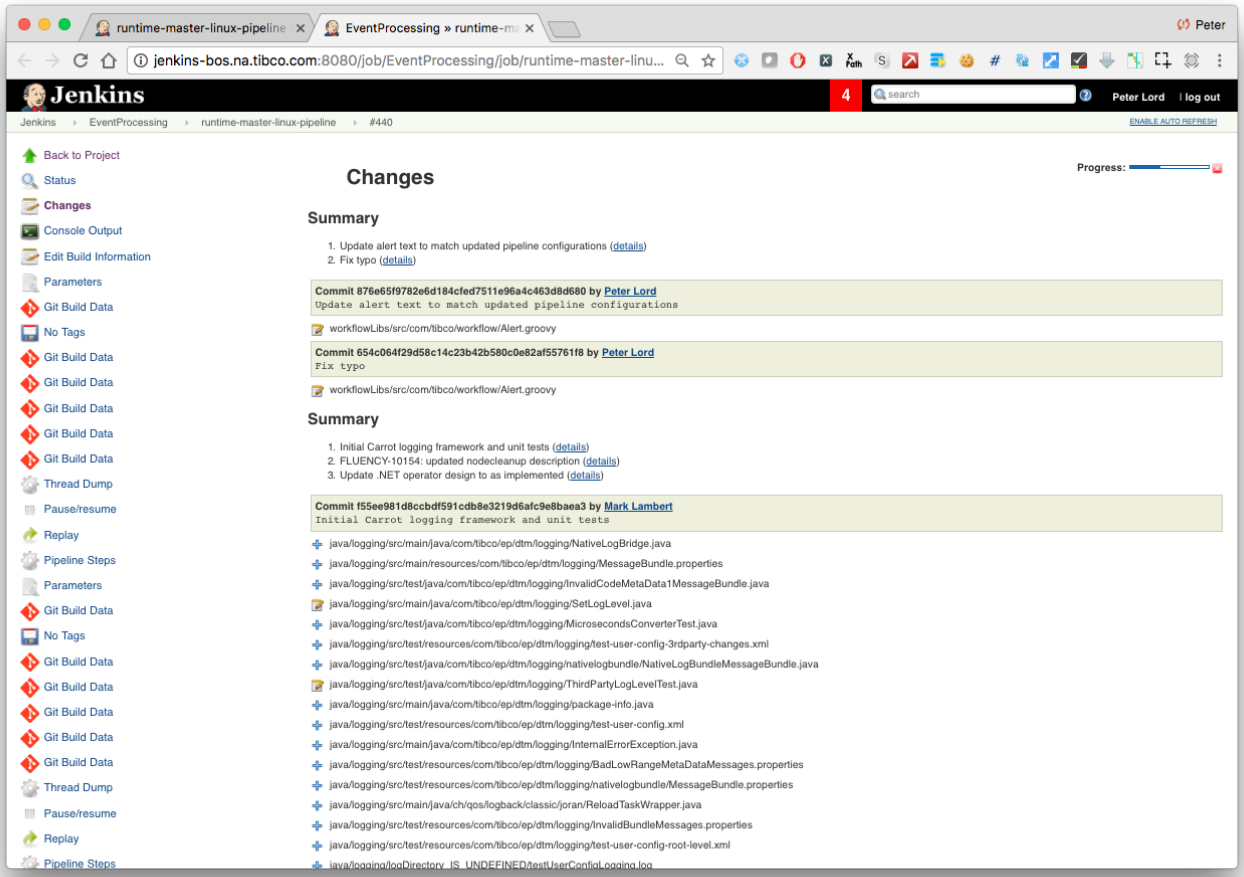

## How do I find what changed in each build ?

Each build pipeline polls the source code to determine if anything has changed.  This also generates changes per milestone which are available to view on the pipeline page.

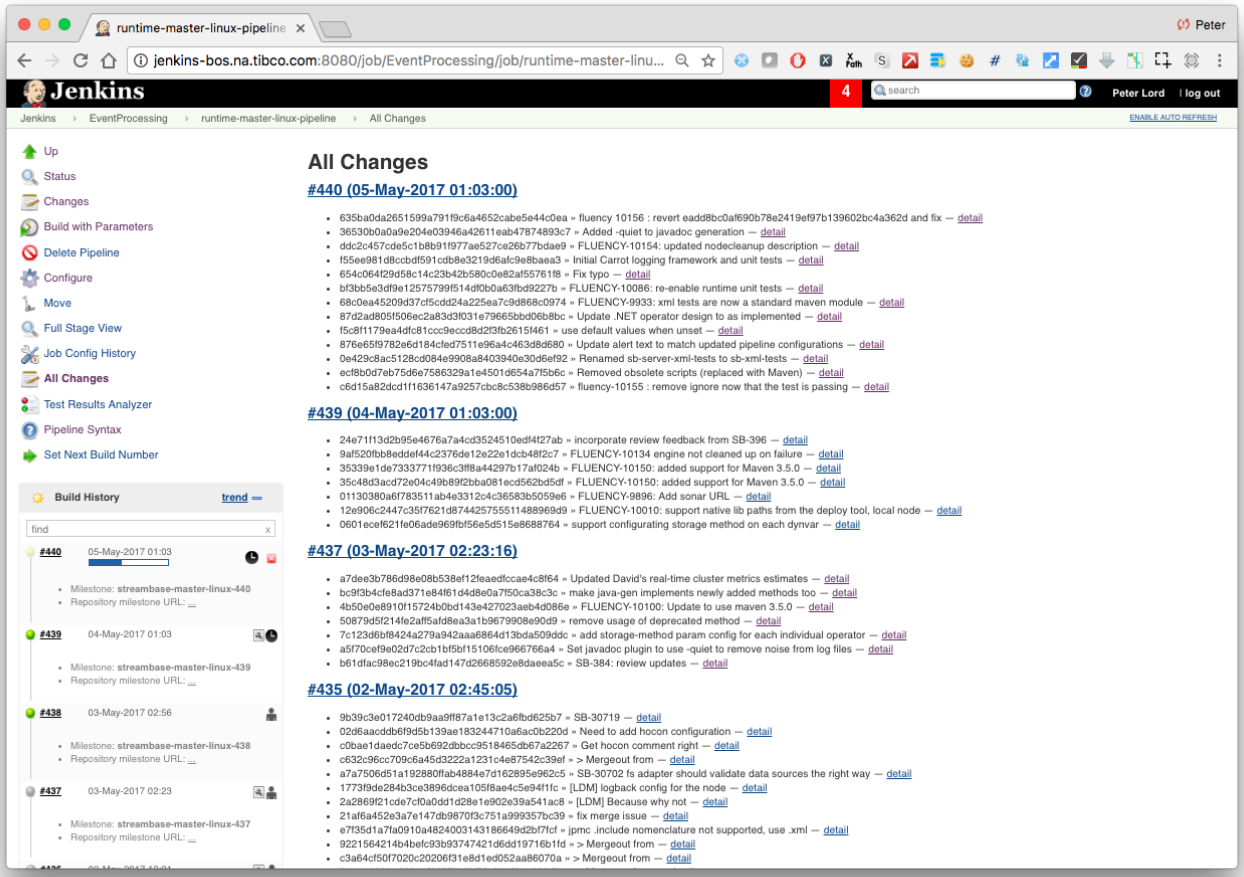
In the screenshot below, we can see runtime build #440 is for milestone streambase-master-linux-440 and the list of changes across the codelines in the pipeline is listed in the pop-up :




The detail page gives more information, including changed files :

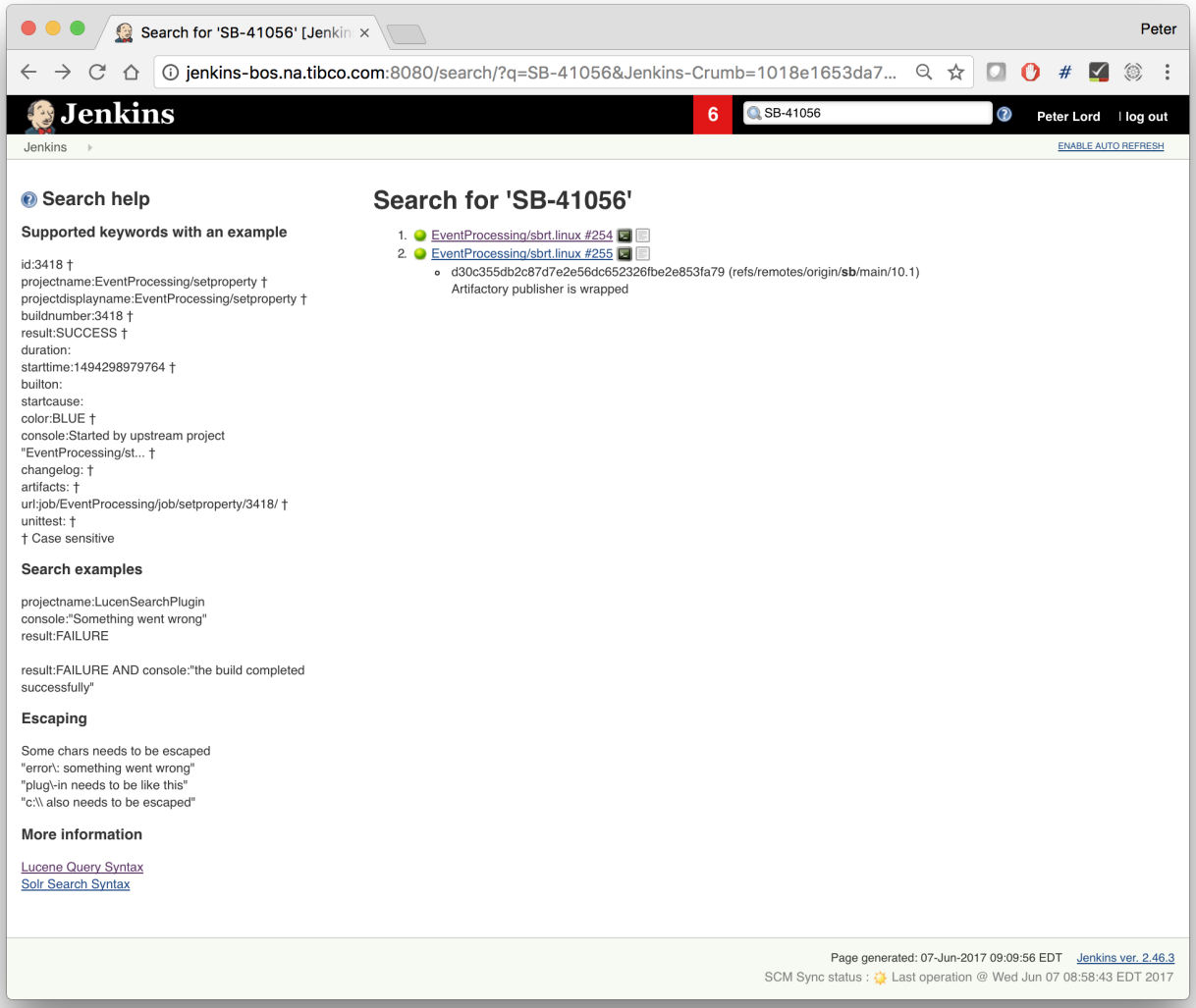



The **All Changes** gives the same information :




## How do I find what JIRA issues are resolved in a build?

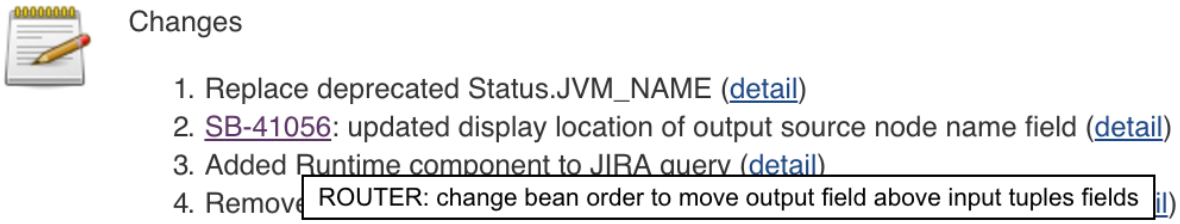
The Jenkins search should be able to (but currently cannot: SB-49340) locate builds from JIRA issues by using the search box :




Jenkins can also parse correctly formatted JIRA issues in checkin notices.  This gives us :

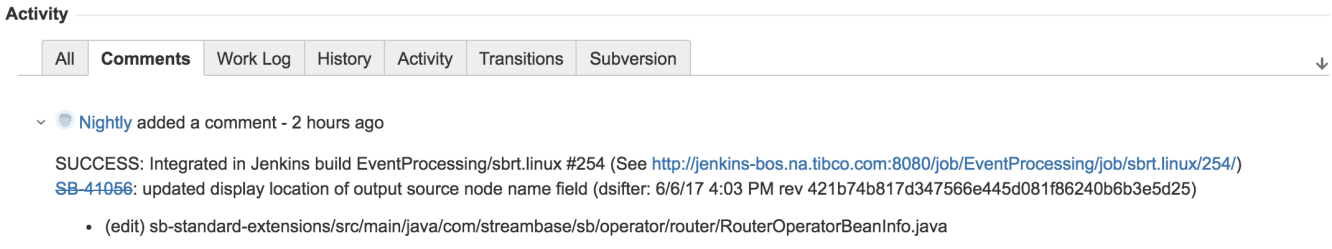
## Links to issues

In lists of changes, JIRA issues are converted to links and the issue description shown :




## Original JIRA issue updated

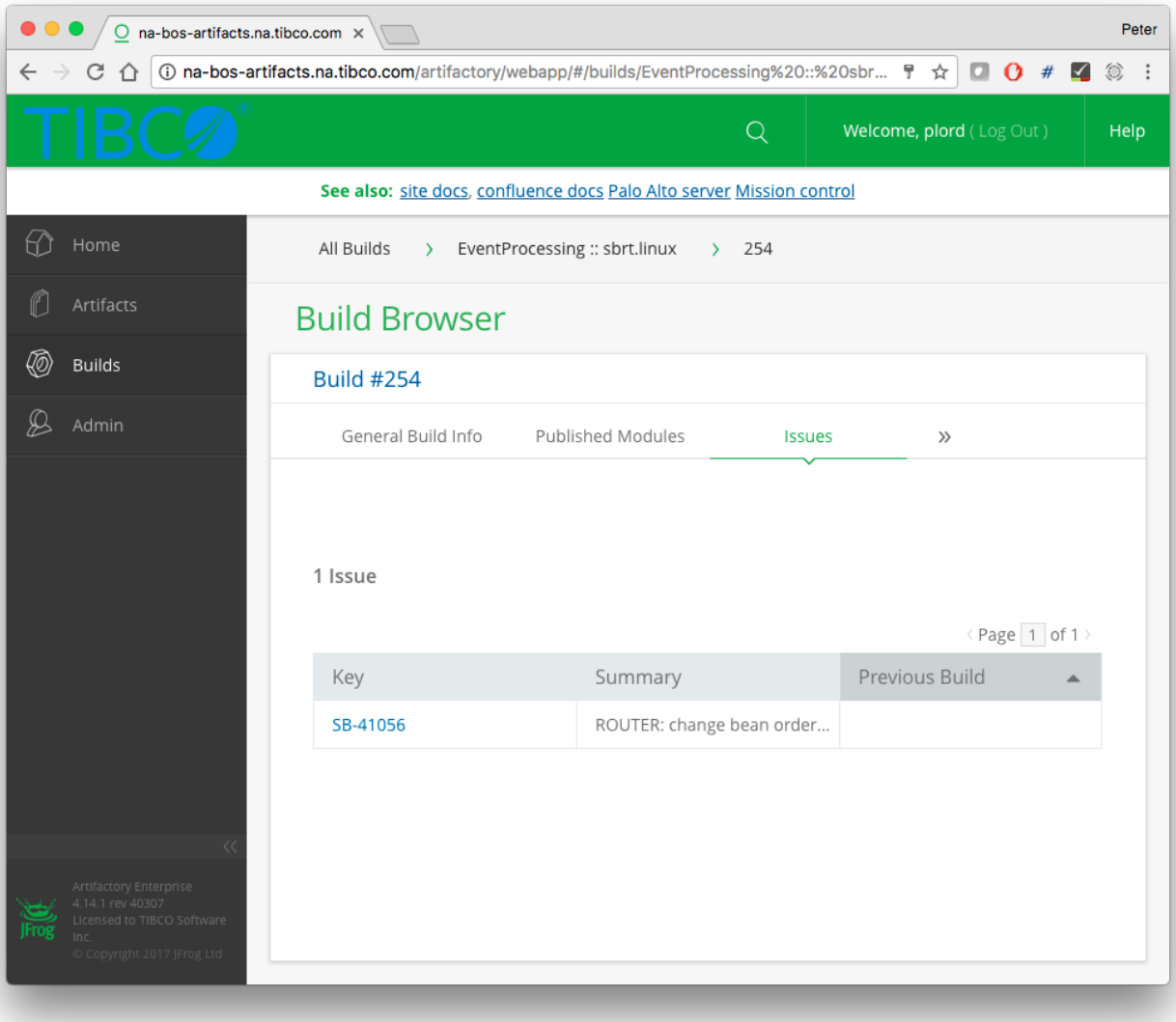
If the build is successful, jenkins will add a comment to the issue with links back to the jenkins build :




After navigating from the JIRA issue to the jenkins build job, you can select the build parameters to determine the milestone.

## Artifactory build updated

The artifactory build is also updated with the list of issues :

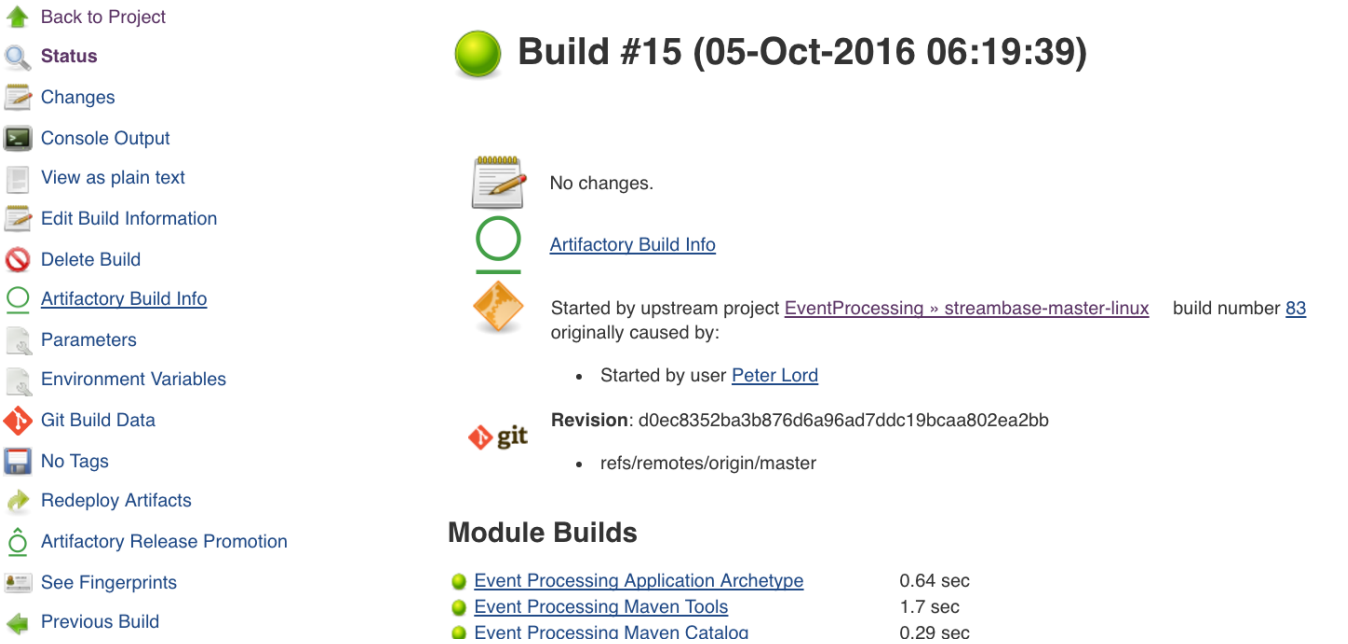



## Maven site reports

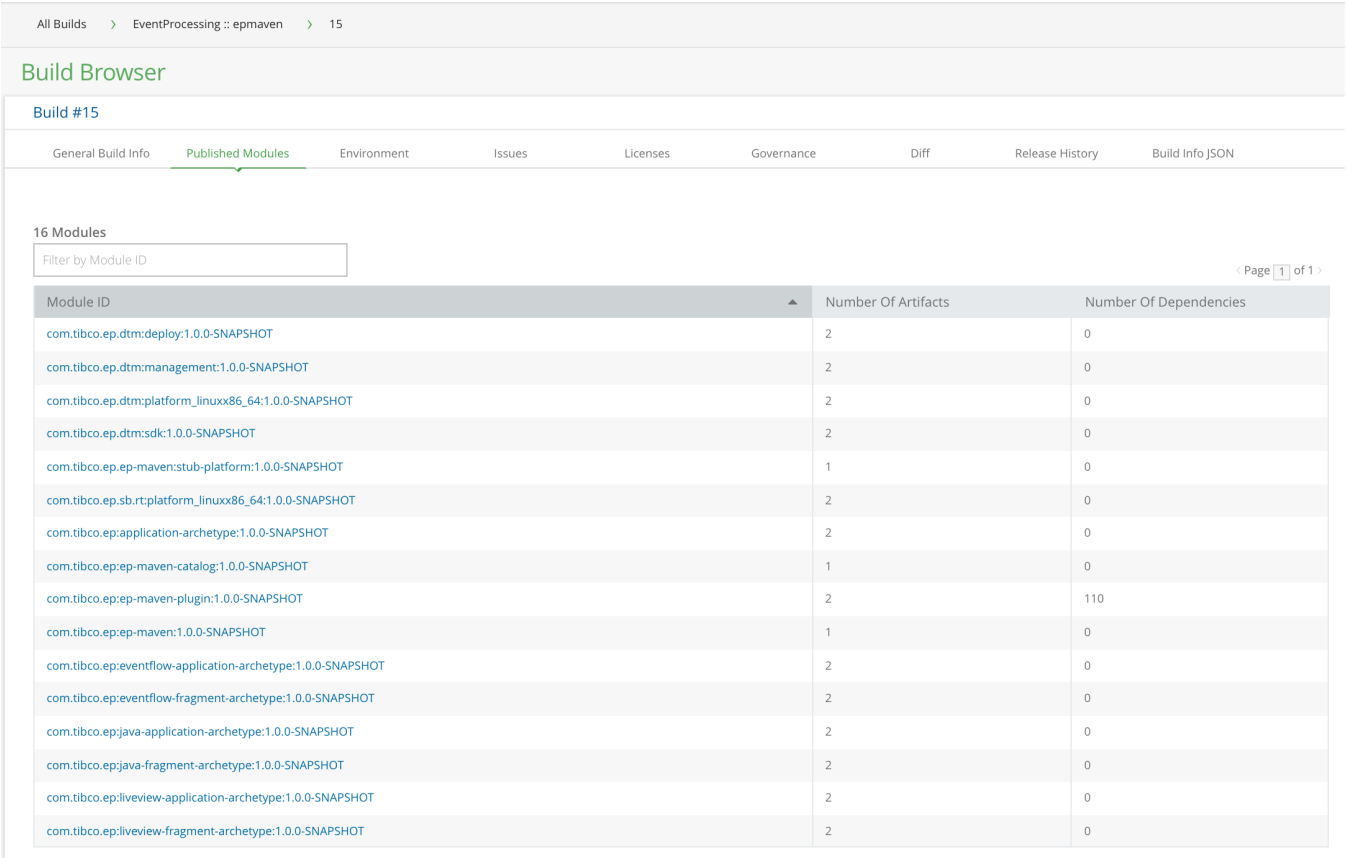
Builds themselves can use the Maven plugins to query JIRA and generate reports in the standard site docs - for example http://ep-artifacts-us-east.tibco.com/site/latest-master-linux/sb/sb-product/jira-report.html.

# How to I navigate from jenkins to artifactory ?

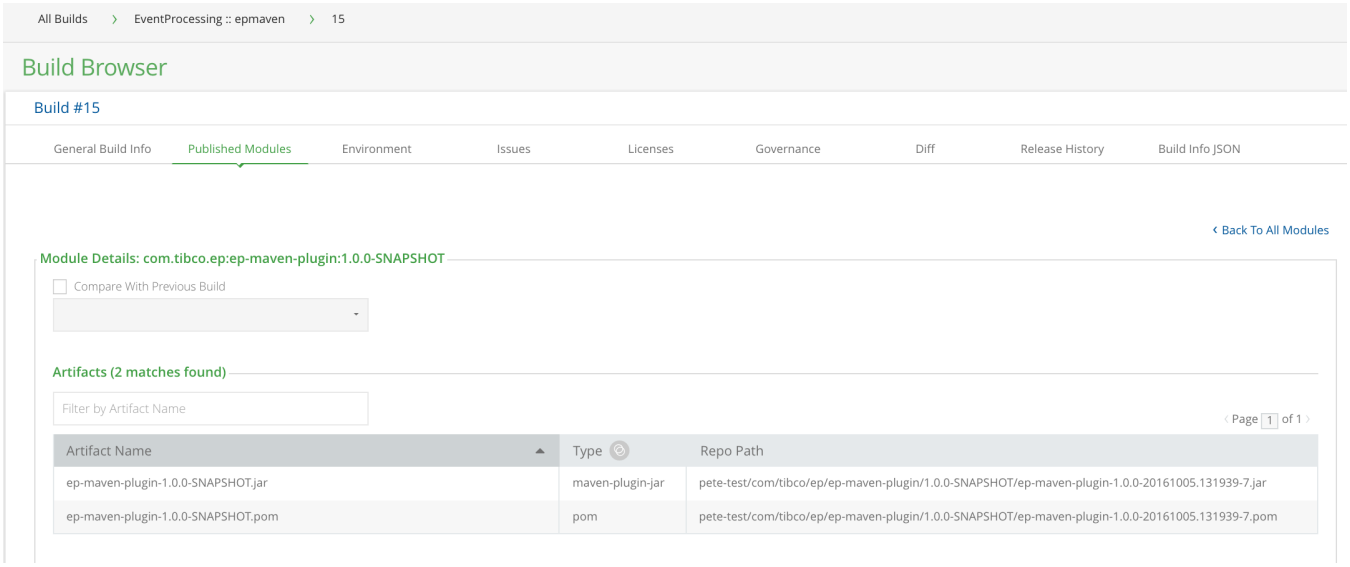
Use the Artifactory Build Info link in the jenkins job :



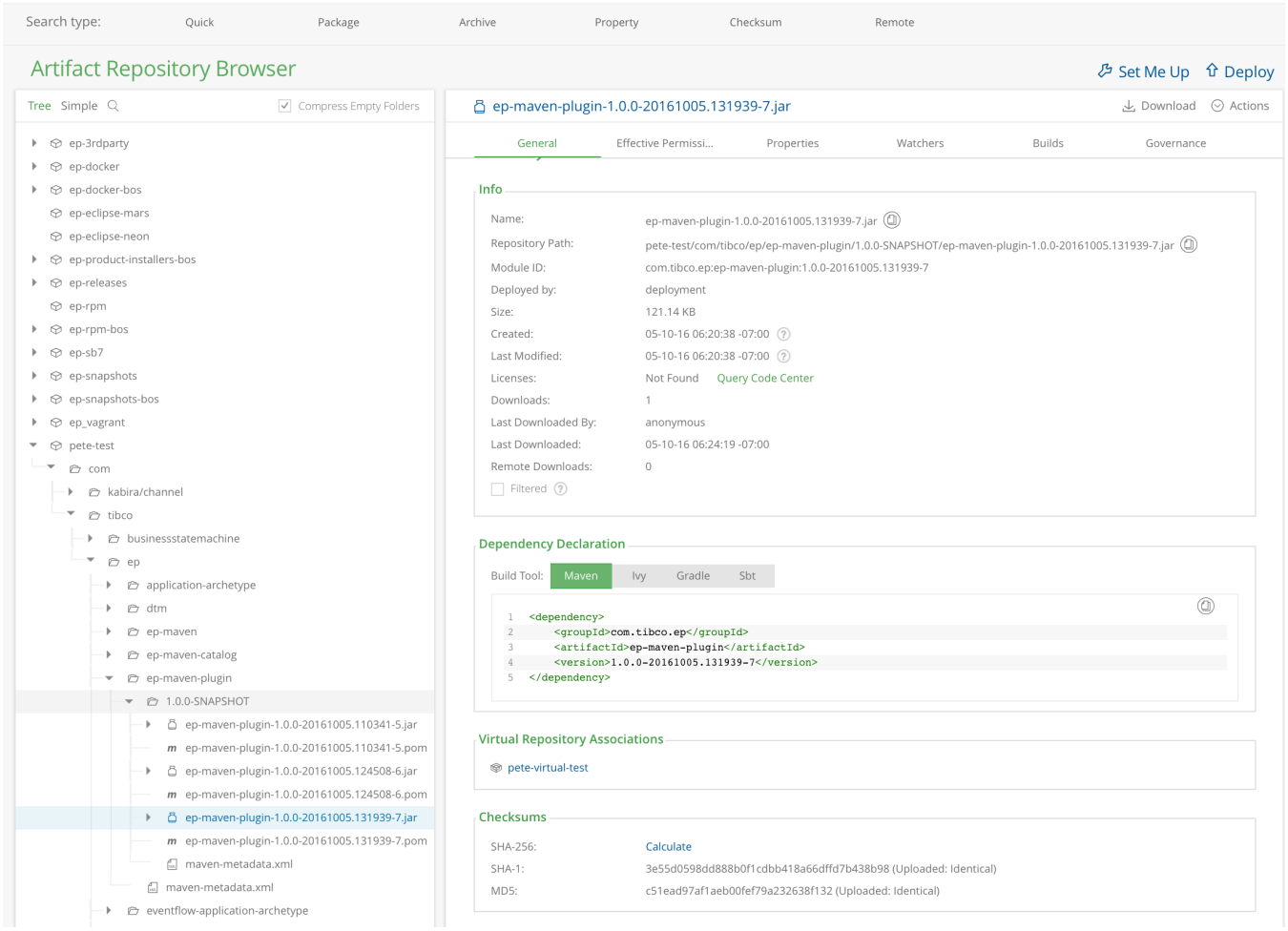

In the artifactory build, select Published Modules :




then the module you are interested in :




The Show in tree after the Repo path :




At this point you can look at the artifact properties which will contain the milestone identifier (and others).

## How does Jenkins handle Maven ?

We deliberately configure Jenkins jobs to be isolated so that one job doesn't conflict with another.  This includes having a separate build workspace and Maven repository per workspace.

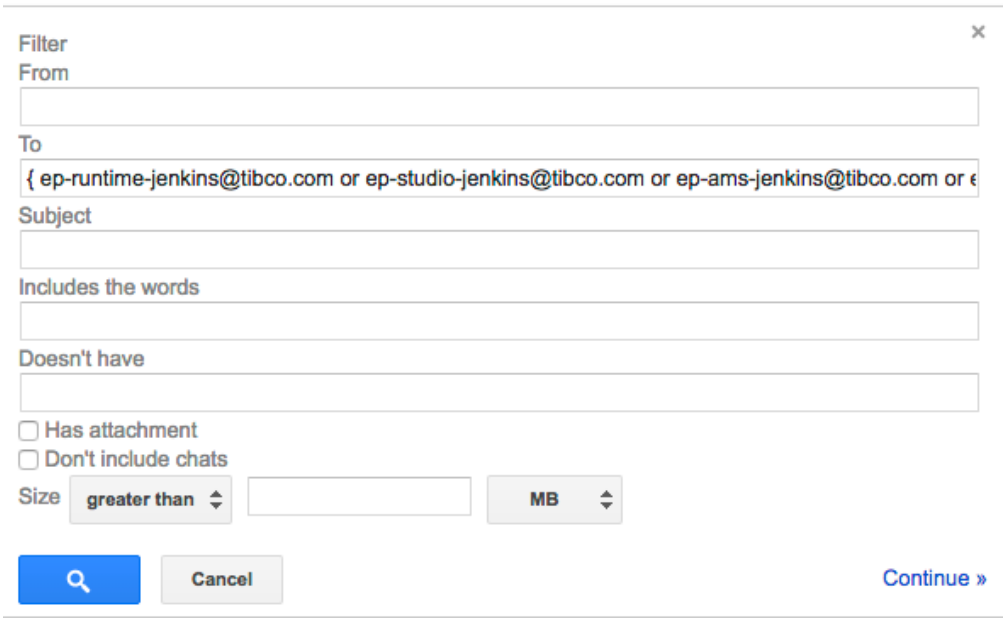
Having said that, we've found that maven **can** make use of ~/.m2/ directory in some cases.  Hence its important to make sure user `epnightly` doesn't have ~/.m2.

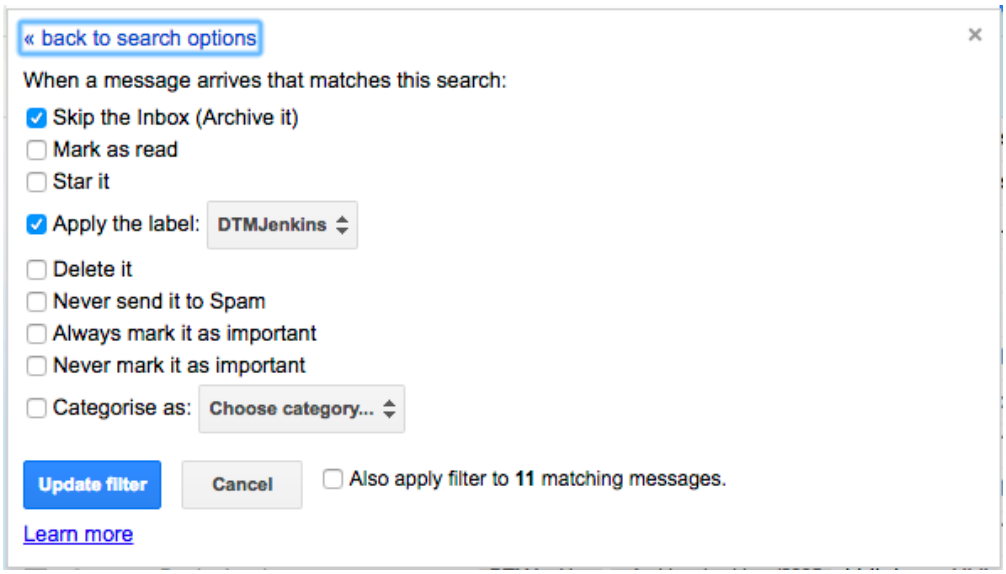
## How can I subscribe to Jenkins email notifications ?

We've setup some email aliases for Jenkins alerts and follow-on discussions that can be joined :

- Runtime :
  - Email address: ep-runtime-jenkins@tibco.com
  - Add yourself: https://directory.tibco.com/webdir/properties.aspx?DN=CN=Ep-runtime-jenkins,OU=Distribution%20Groups,OU=Exchange%20Resources,DC=na,DC=tibco,DC=co
- Ams :
  - Email address: ep-ams-jenkins@tibco.com
  - Add yourself: https://directory.tibco.com/webdir/properties.aspx?DN=CN=Ep-ams-jenkins,OU=Distribution%20Groups,OU=Exchange%20Resources,DC=na,DC=tibco,DC=com
- Studio :
  - Email address: ep-studio-jenkins@tibco.com
  - Add yourself: https://directory.tibco.com/webdir/properties.aspx?DN=CN=Ep-studio-jenkins,OU=Distribution%20Groups,OU=Exchange%20Resources,DC=na,DC=tibco,DC=com
- QA :
  - Email address: ep-qa-jenkins@tibco.com
  - Add yourself: https://directory.tibco.com/webdir/properties.aspx?DN=CN=Ep-qa-jenkins,OU=Distribution%20Groups,OU=Exchange%20Resources,DC=na,DC=tibco,DC=com
- Jenkins Build Machine Outages:
  - Email address: nightly@tibco.com
  - Add yourself: https://directory.tibco.com/webdir/Properties.aspx?DN=CN=Nightly%20Regressions,OU=Kabira,OU=Distribution%20Groups,OU=Exchange%20Resources,DC=na,DC=tibco,DC=com

Its recommended to setup gmail filtering rules so that gmail doesn't hide messages from yourself.  For example any message To these addresses, filter to a specific label :






## Can I use the no-deploy builds for my own branch ?

Yes. Once you've created a job for yourself and it completes successfully, you can install it using `sbx install` with the usual options.

### Creating an nd job from the Jenkins GUI.

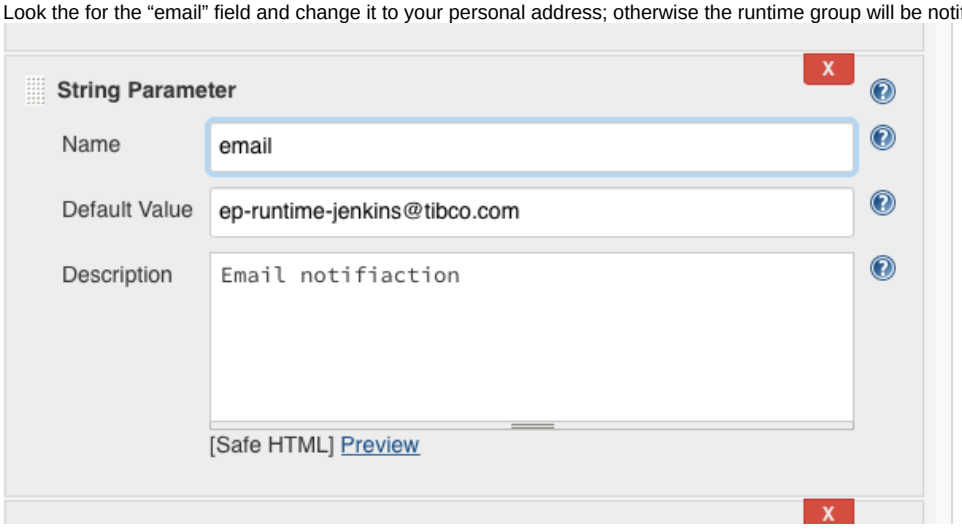
You must be authenticated to http://jenkins-bos.na.tibco.com:8080/job/Streaming/ – be patient and check your signed-in status in the upper right.

Click the “New Item” link at the left.

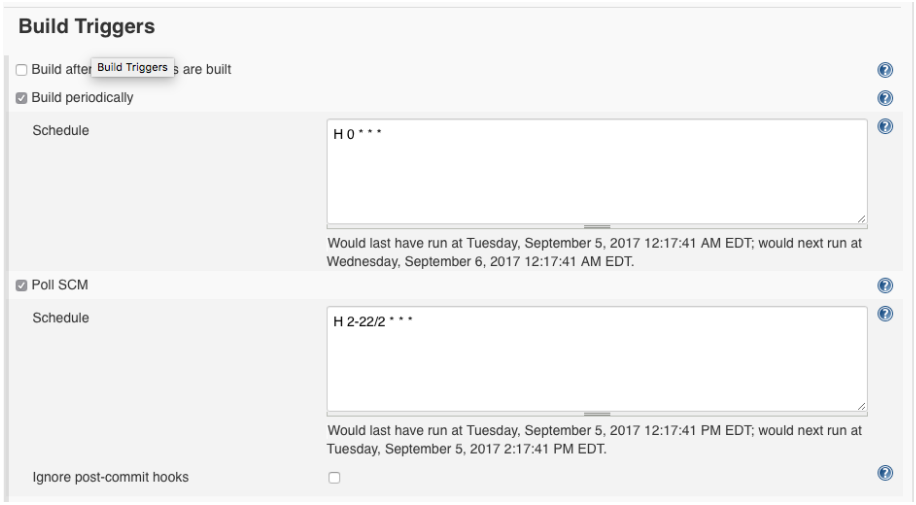
Enter a name, copy from one of `nd.master.<os>` – there is completion. Or, if your branch is off `sb/main/10.x`, use `nd.10x.<os>` as a starting point.

This should give you a new job with its configuration page. Change these configuration elements to customize the job's default parameters:

Look the for the “email” field and change it to your personal address; otherwise the runtime group will be notified.




Look for the build triggers (fast navigation with a tab called Build Triggers at the top)




Uncheck both boxes so that you're not triggering builds all the time.

Hit the bluish Save button.

You are now on the job page and can use the **Build With Parameters link** (which you'll want to use to actually kick off a build).

You have two options when kicking off a build, depending on the branches you've created:

1. If you've created branches for *all* the repos (`parent`, `sbrt`, `dtm`, `dtmfilestore`, `ep-str-smoketests`, `rms` (until sb/main/10.6), StreamBase Subversion) with the same name, you should change the branch name in the `branch` field.
2. **Or,** leave the `branch` field alone and instead fill it in for every repo listed on the form, replacing `${branch}` for the Git repositories and `trunk` for `streambase`. (You can see how the branch specification and defaulting works.) Ugly, but it should work.

Hit the bluish Build button. Hope for the best. You should be able to install the build (make sure you're on the right platform). Suppose you want the base kit kit on Linux:

```
$ sbx install –base deend.linux # assuming you created that job for that platform
$ sbx broken-build deend.linux # if something goes wrong, this might locate the problem
```

Note that the branch and even the build label for your job (which affects the platform for which you're building) can be changed to anything that works when you kick off a build from Jenkins. If you don't specify a build label, you'll get a default based on part of the job name after the last period, which should name a platform like `linux`, `linuxarm`, `macos`, or `windows`.

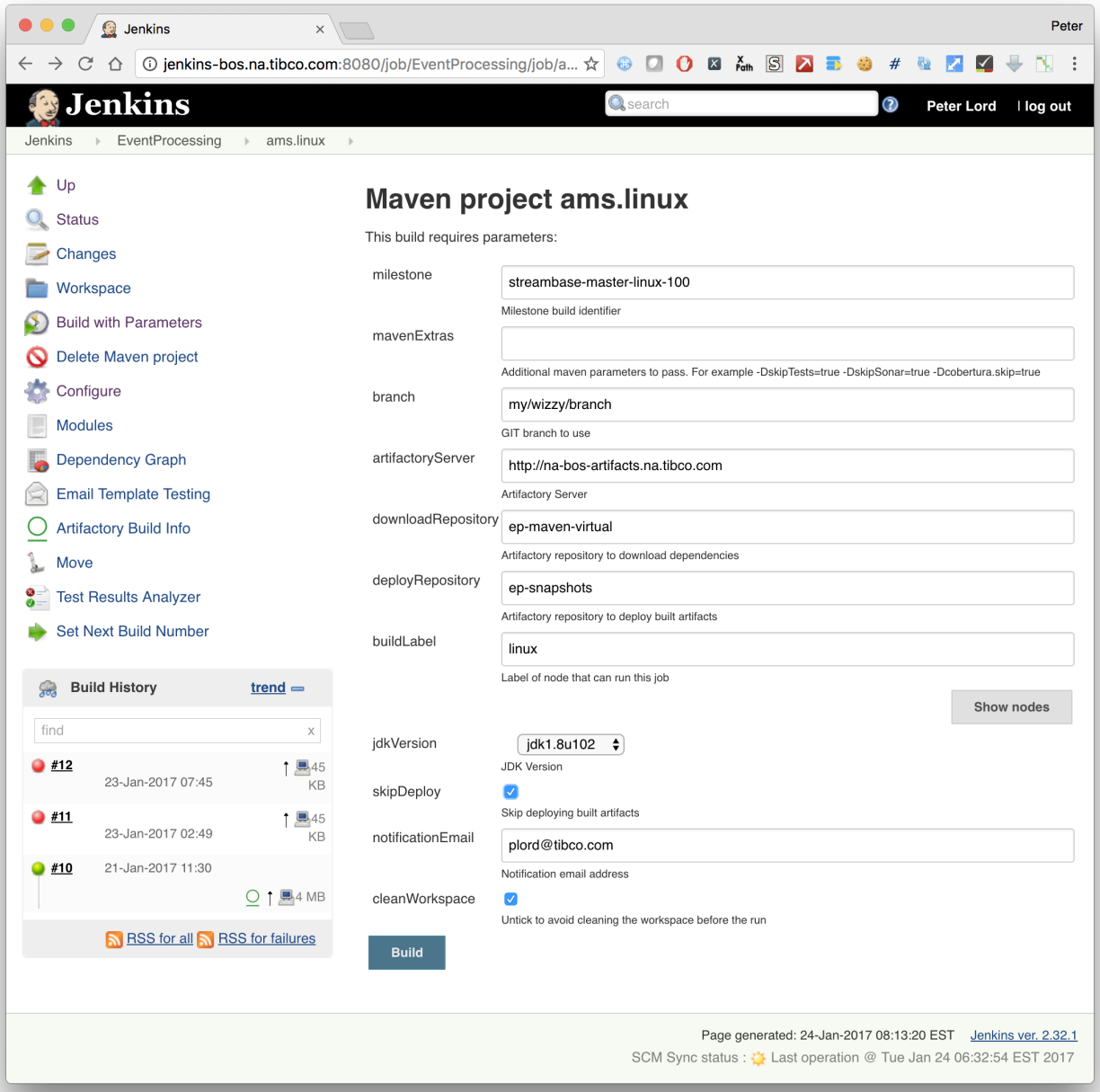
Just make sure your re-use of the build job agrees with branch and platform you want to try, because each Jenkins no-deploy job holds the full set of installers from the last successful build *only*. So if you use the same job for branch *A* and then *B*, *A*'s installers will disappear in favor of *B*'s.

### Creating an nd job using sbx copy-nd-job

You can also copy from an existing nd job to a new one using the `sbx copy-nd-job` command. This command requires at least three parameters – the source job, your new job, and the main branch to build. Other options are useful (see the `--help`) and to make it really happen, you'll need to supply `--really` on the command line. You'll also need a `~/.netrc` entry for `jenkins-bos.na.tibco.com` - for a password you should make an API token for yourself – this is recommended over using your actual TIBCO password lest it fall into the wrong hands.

## How can I run a single job test cases without deploying ?

- Use a standard build job
- Set email address to self
- Change branch if needed
- Tick "skipDeploy"

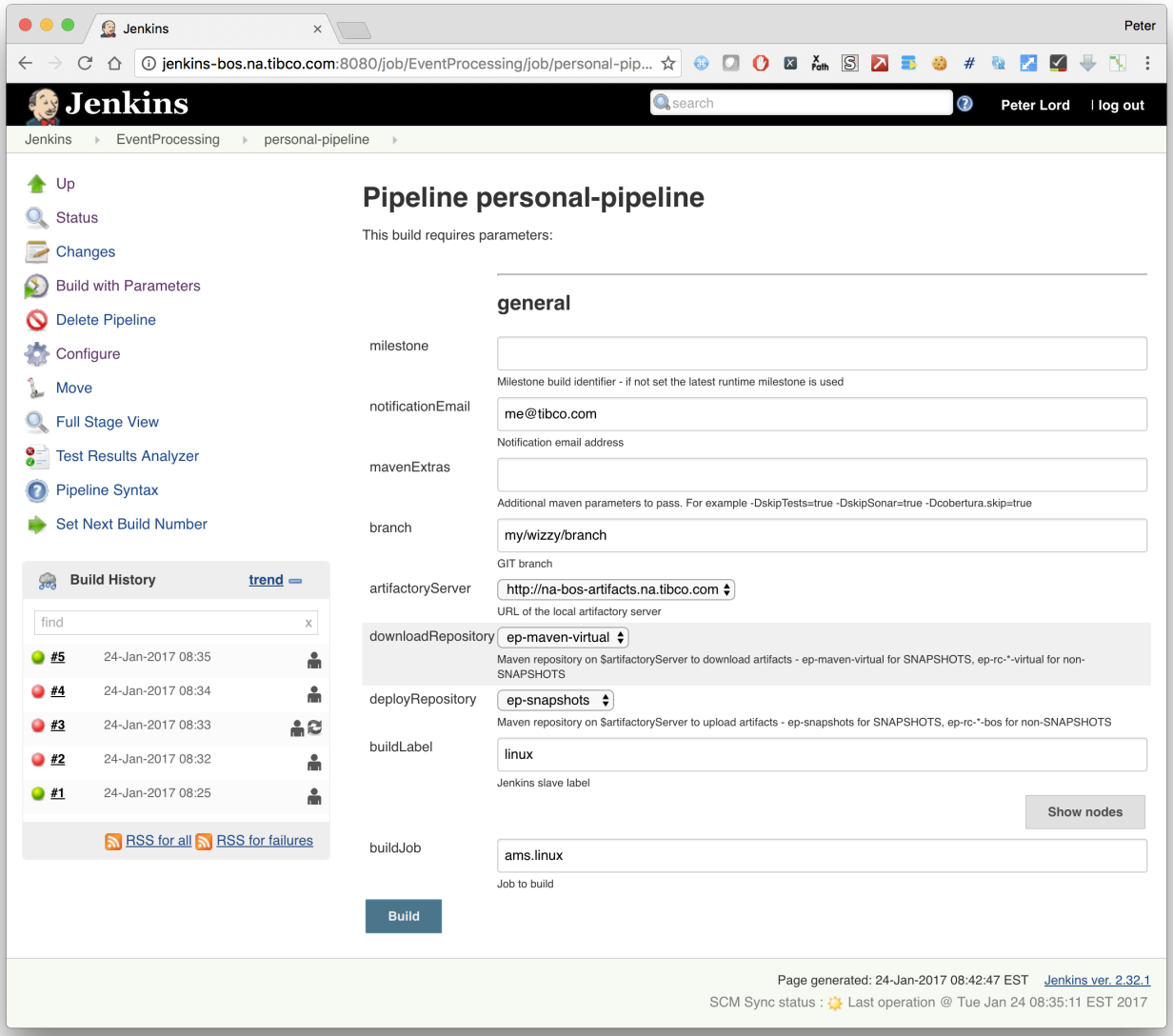



## How can I setup my own pipeline ?

For regular testing of a personal branch, creating a personal pipeline is likely to be best.  There is the pipeline personal-pipeline available for this purpose, this pipeline find the right dependnecy milestone, runs the configured job with deploy disabled and emails the results.

To run :

- Add a specific milestone identifier to be used to resolve dependencies - leaving blank is the latest
- Set notification email address
- Set personal branch name
- Set build job name



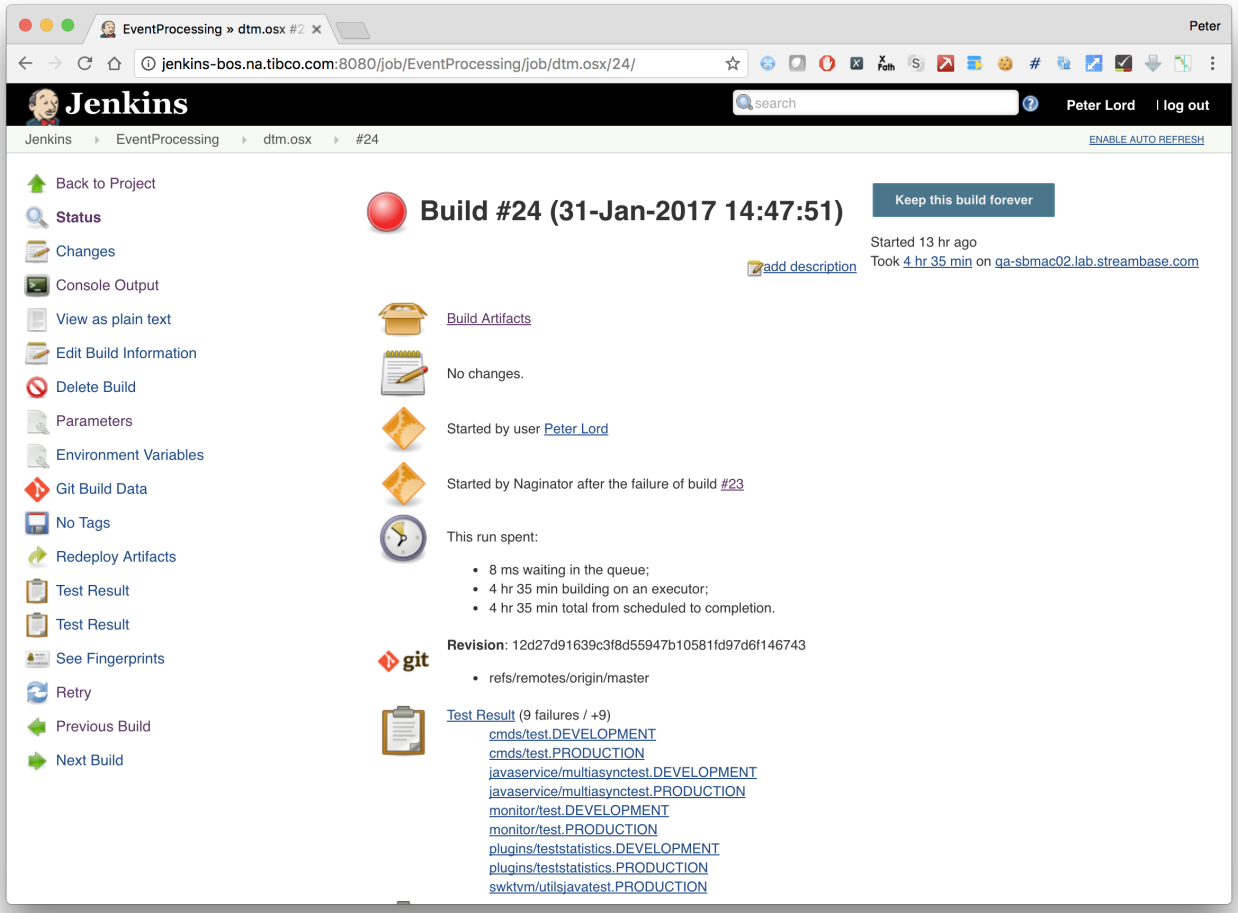

This pipeline job can be copied to avoid re-entering the above parameters each time.

## As a night watch person, how do I understand Jenkins ?

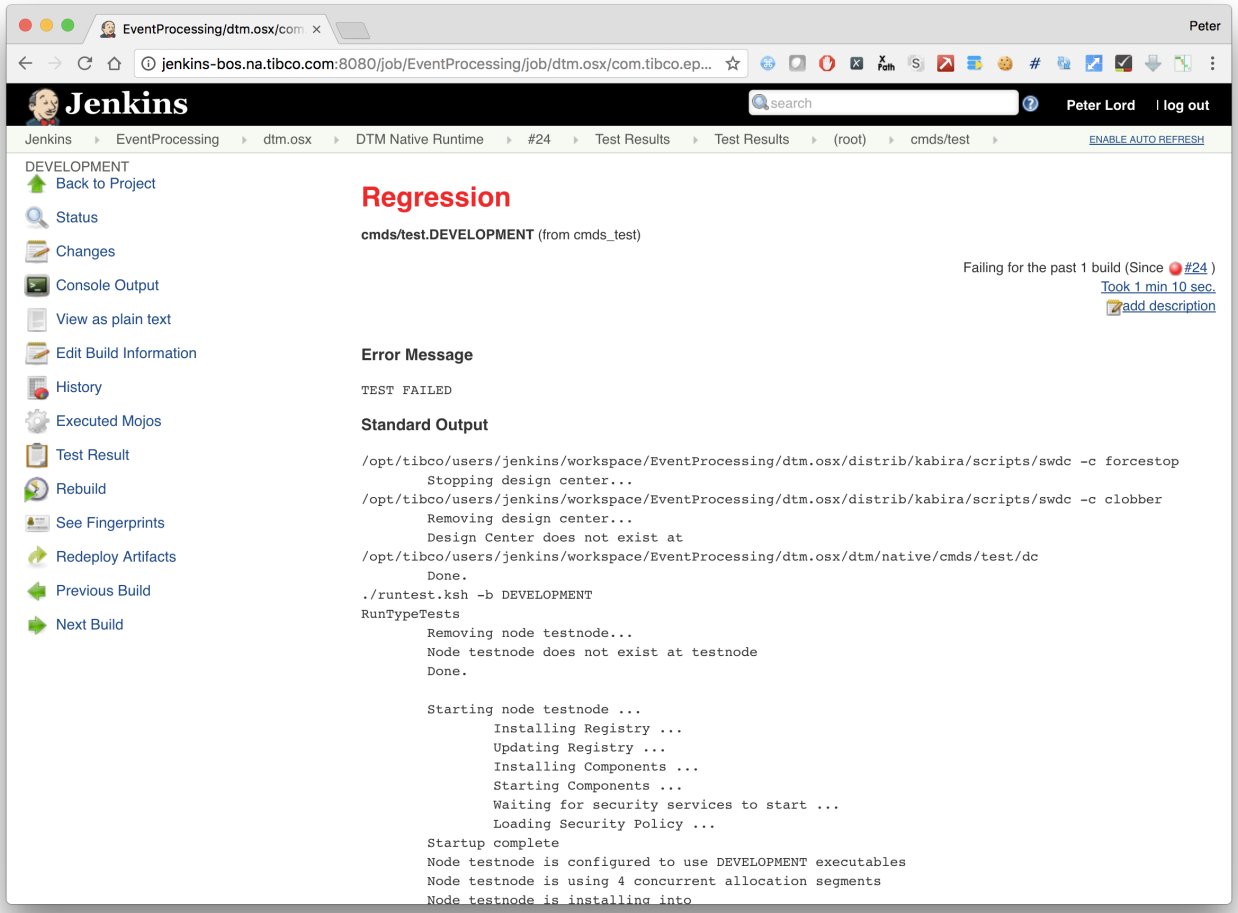
Sometimes figuring out the status of a build can be non-obvious, here are some hints and tips.

### Junit test results

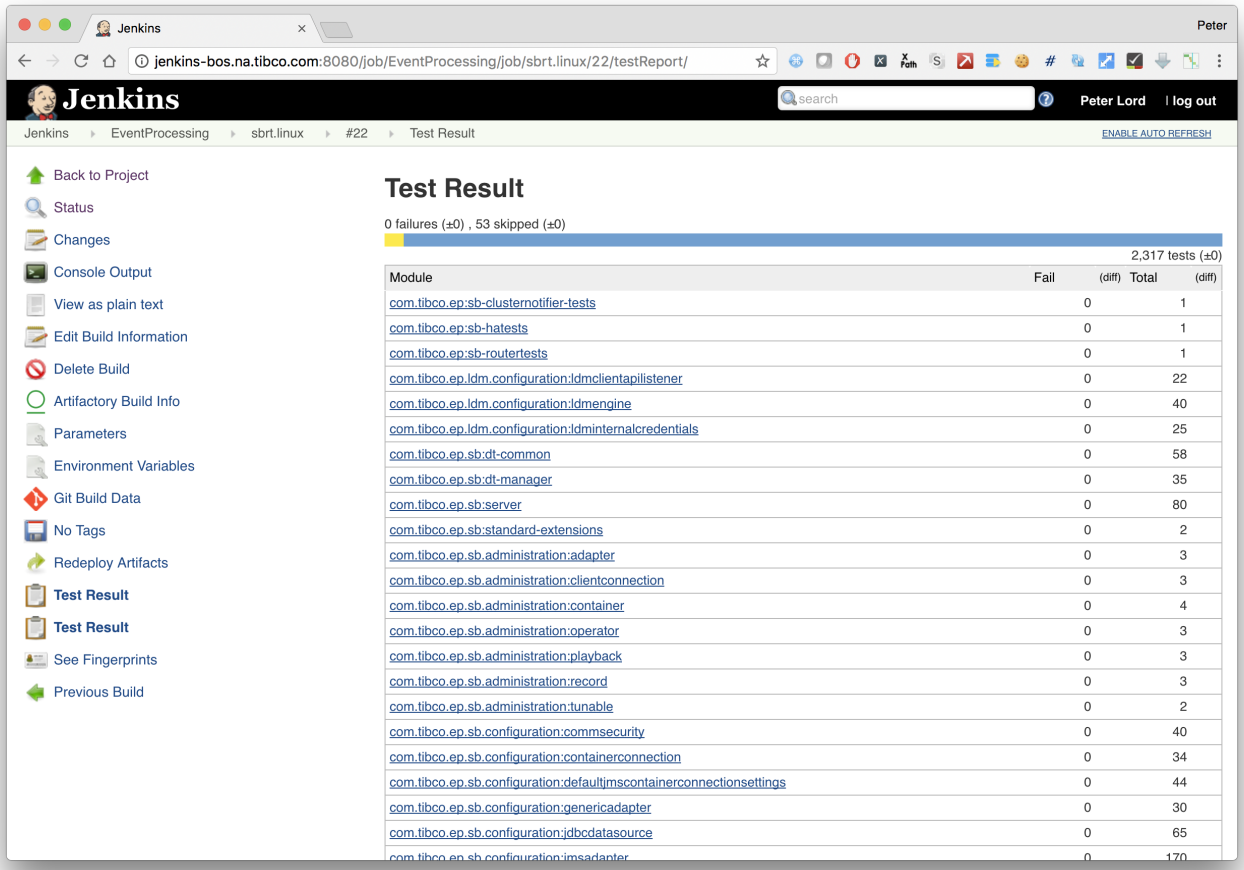
Jenkins supports finding and reporting on junit results.  Junit test failures should appear on the job home page :




Navigating through the link should get you more details :




All junit test results can be found, regardless of results, by following the **Test Results** link :




## Console logs

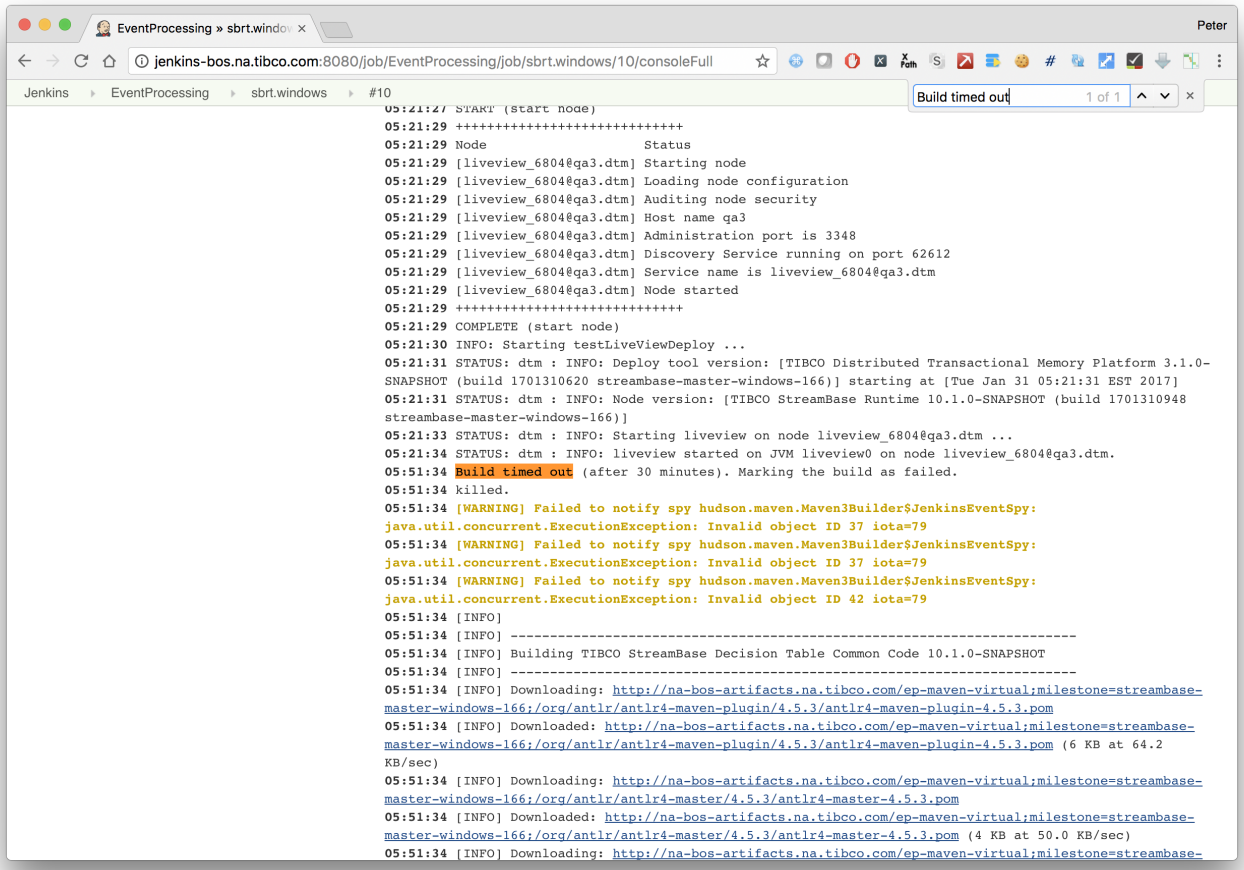
Unfortunately not all failures are from within junit test cases, other common failures include failure to start (or stop) nodes and timeouts. Selecting **Console Output** and then possibly **Full Log** will display the full output.

Since this can be large, one trick is to copy the URL of **View as plain text** and paste this on the command-line fir curl/wget - then a local editor can be used.

The email notifications contain a filtered version of the console logs (grep for specific errors).

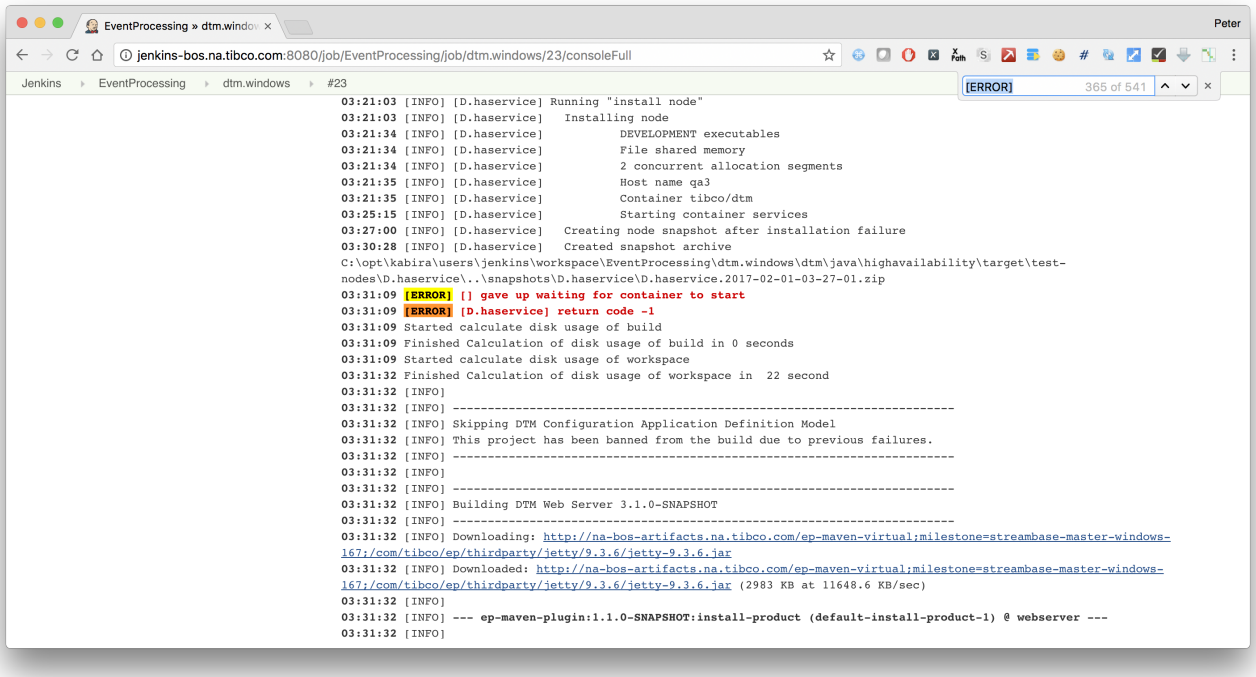
### Build timed out

Search for Build timed out to locate what step stalled - note that for slow builds with no output, the jenkins job configuration can be tweaked.




## Maven errors

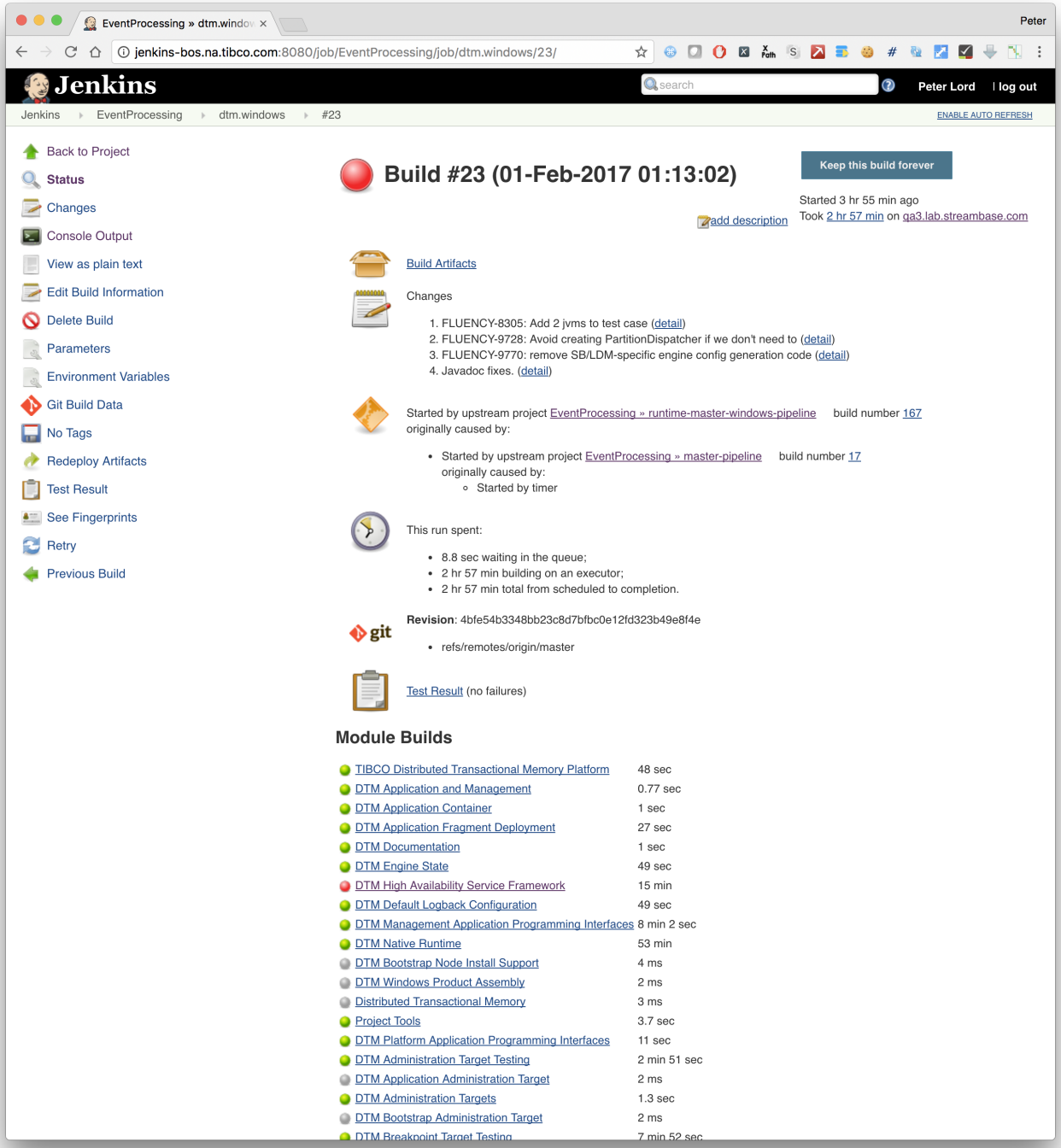
Understanding of the Maven output is very useful, for example a search of **[ERROR]** can be useful :




Although test developers will often use **[ERROR]** for negative test cases, which can cause overload of the night watch person.

## Module failures

Sometimes we will see no unit test failures but a maven module failure :




This link can be followed and further explored - including looking at the console logs for just that module.

## Product (Un)Installation Errors

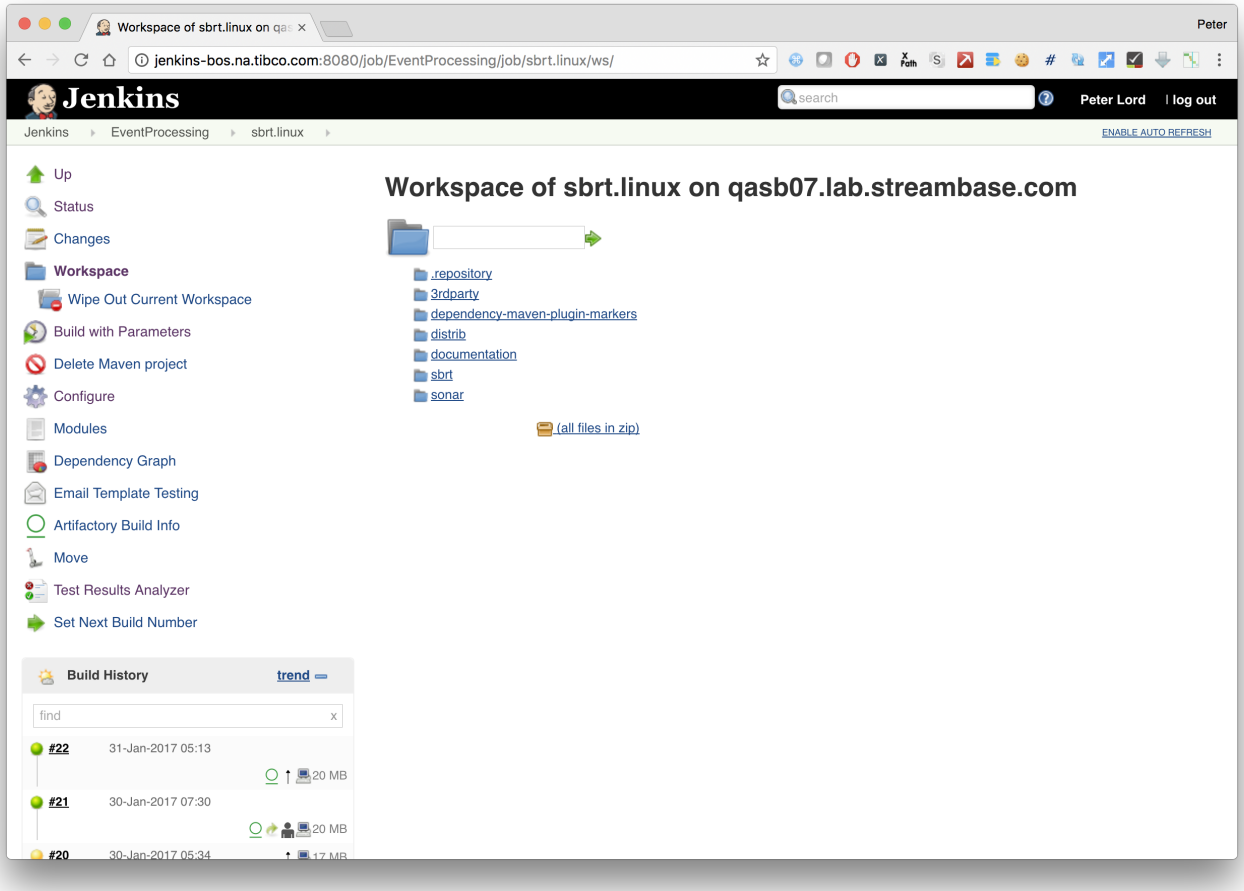
QA and smoke tests for Streaming install the entire product. If it goes wrong, the most common cause, especially on Windows, is that a Streaming process is still running despite all efforts to clean up such processes. In the case, the machine should be taken offline, rebooted (perform updates if you can), and the product manually uninstalled before bring the executor back online.

In some rare cases, Streaming cannot be uninstalled using `sbx` or the Windows Apps & Features Control Panel (see JIRA Issue SB-50243). Use the Microsoft Program Install and Uninstalled Troubleshooter tool to use a bigger hammer to uninstall Streaming.

## Exploring the workspace

The build tree (physically located on the build server) can be access from jenkins via the **Workspace** link.  Note, though, this only applies to the last build.

The tree can be explored, files viewed and zip archives downloaded locally :




## Logging into the executor

Sometimes it is required to log into the executor to explore the workspace. You'll get the user, machine, and workspace directory of the job that failed from the failure mail. Although the epnightly user is the usual account that runs builds on an executor, the exact name of the account might be different, either because it's not `epnightly` or because Windows qualifies an account name with a domain name like `TSI-PA+`.

For hosts in the `dev.tibco.com domain`, jobs are usually run as user `epnightly` under `/mnt/jenkins/workspace`. The build failure emails will tell you exactly which workspace directory is being used. The `nightly` and `epnightly` users have the same password.  You can use `ssh-copy-id epnightly@`*`executor`* (or `ssh-copy-id TSI-PA+epnightly@`*`executor`* for Windows) so that the next time you need to `ssh` in, you don't need the password.

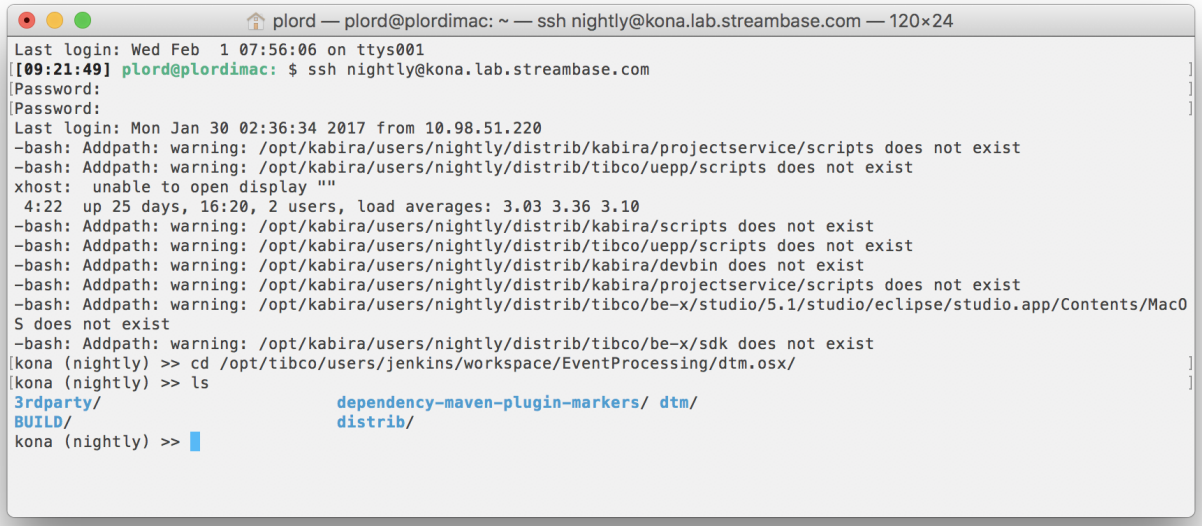
GUI access to a Windows executor is normally through Microsoft Remote Desktop Protocol, but GUI access to machines that run Squish is offered with VNC, by using the typical `sb-test` password as the VNC password. The GUI test suites in the `sbproductsmoke` and `qa` test jobs use Squish, which is not compatible with Windows Remote Desktop, so use a VNC client instead.

If you want to log into a Windows executor as yourself, you should log in first via the GUI so that Windows creates your home directory – you *might* need `TSI-PA\user` as the user id – some machines seem to pick up the domain automatically if not explicitly supplied. For SSH, you need the `TSI+` prefix because we run Cygwin SSHD on our Windows executors. On a related note, see Cygwin User IDs From Domain Logins.

**When you leave**

**Be sure to close or minimize all windows before disconnect. DO NOT LOG OUT NIGHTLY USER FROM THE CONSOLE.**

**Especially for Windows, but polite conduct generally: Make sure that no programs running with files open in the workspace. In particular COMMAND LINE WINDOWS SHOULD *NOT* HAVE A WORKSPACE DIRECTORY AS THE CURRENT DIRECTORY. Issue a cd command when done, if you want to keep a command window open.**



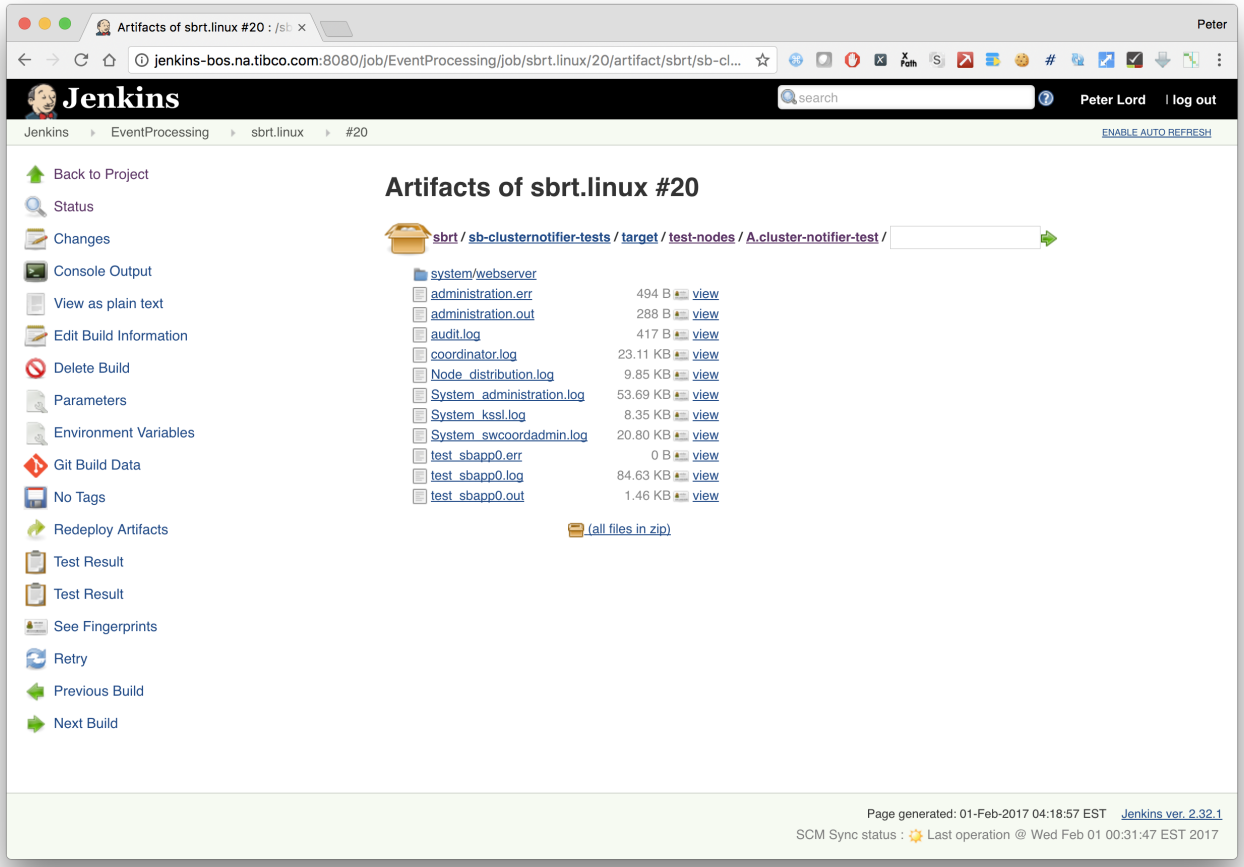

We try to keep processes running after a failed job to tools like gdb can be used.

Its possible to re-run test cases from the command line if needed, providing the appropriate environment is set.  The process is :

- Login as test user, nightly or epnightly depending on the machine.
- Download a copy of the maven settings.xml jenkins uses (likely to be common settings found at http://jenkins-bos.na.tibco.com:8080/configfiles/editConfig?id=b41d5c2c-4b1f-4c10-bc6e-ee6277ec505d)
- Set JAVA_HOME
    - `export JAVA_HOME=/mnt/jenkins/tools/hudson.model.JDK/jdk11.0.10`
- Set PATH to include Maven and Git (Git for RHEL 8 doesn't need to be specified; it's new enough) :
    - `PATH=/mnt/jenkins/tools/hudson.tasks.Maven_MavenInstallation/3.6.3/bin:/sb/tools/sbdeps/git/git-2.28.0-x86_64-linux-rhel7/bin:$PATH`
    - `export PATH`
- Set variables referenced in settings.xml - these are set in the jenkins job "
    - `artifactoryServer=http://ep-artifacts-us-east.tibco.com`
    - `milestone=streambase-100-linux-20`
    - `downloadRepository=ep-maven-virtual`
    - `export artifactoryServer milestone downloadRepository`
- Set TIBCO_EP_HOME - will include your build name
    - `export TIBCO_EP_HOME=/mnt/jenkins/workspace/sbrt.linux/master`
- Run maven, specifying the local repository and downloaded settings.xml :
    - `cd ${TIBCO_EP_HOME}/some/test/directory`
    - `mvn -Dmaven.repo.local=${TIBCO_EP_HOME}/.repository -s ~/jenkins_settings.xml install`

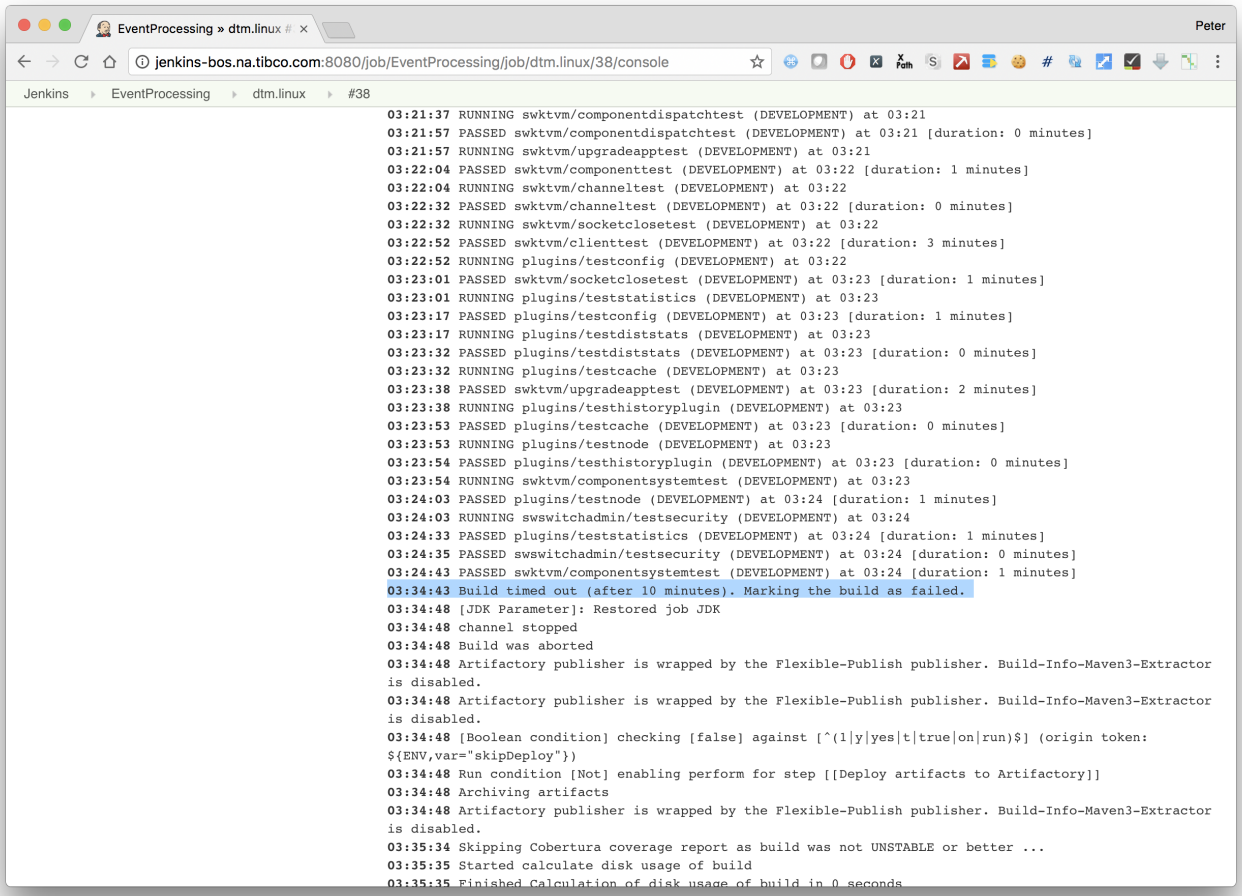
## Historical log files

Jenkins is configured to archive certain file types (eg *.log, *.out etc) along with the build, so if the workspace has been removed, some analysis of of the build can still be done.  Select the **Build Artifacts** link on the build page and explore the tree :




## dtm native timeouts

Sometime you will see :




But which test hung ?

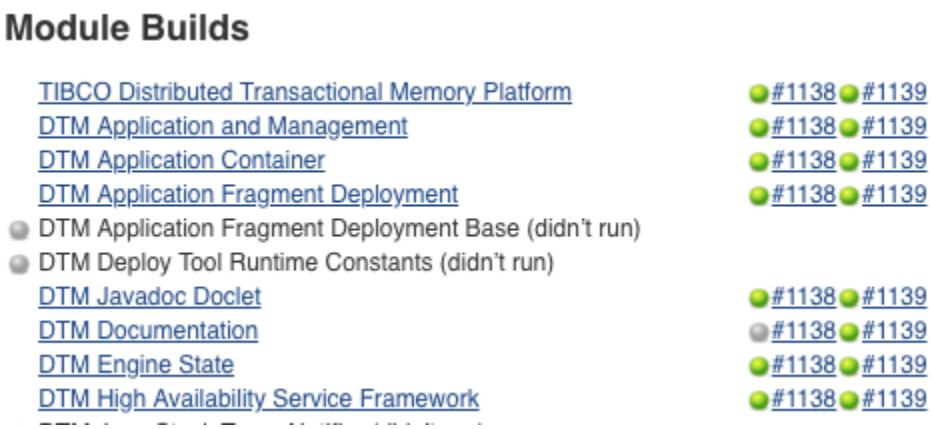
The intention was that this would get reported to Jenkins as a test failure ... however this isn't working right now.

To find the test, login to the server and grep the logs :

```
$ cd /mnt/jenkins/workspace/dtm.linux/master/dtm/native/test-reports
$ fgrep "TEST RUNNING" *junit*
distsrv-quorumtest-DEVELOPMENT-junit.xml:        <failure message="TEST RUNNING"></failure>
```

## What can we do about duplicate jobs ?

Sometimes jenkins reports duplicate jobs shown by duplicate build numbers :




Cause is currently unknown. As far as we know the piplelines just need to be re-started.

## How Do I Reboot A Test Machine ?

1. First, take the machine offline from Jenkins so that you don't get a race condition where Jenkins assigns a job to the machine before you have change to reboot the machine. We all dislike race conditions, don't we? Make sure you're authenticated. You can start from a link where the machine is mentioned on a Jenkins page, or from the machines page.

2. Then reboot. Most if not all test machines have `sbx` installed. If you can ssh (Windows included), issue the command `sbx reboot`. On Unix, `sudo /usr/local/bin/sbx reboot`. Or use GUI access (VNC or Remote Desktop) to access a Windows machine and issue `sbx reboot` in a Cygwin window.

There is nothing special about the `sbx reboot` command, but it's easier to remember than the various reboot commands on different platforms.
The `epnightly` user should be an admin on all Jenkins executors; so whatever reboot method you know can be used as the `nightly` user.
Also on Windows: there is a reboot option when using the Log Off item from the Start Menu or the IEC "Power" symbol near the Start Menu on newer versions of Windows.

(On the Ducky Unix machines, the `sb-test` user gets to run `sudo sbx` commands without a password. We could do something similar for the `epnightly` user. Or even just allow all domain users to run `sudo sbx` commands without a password.)

3. Bring machine back online from Jenkins once you're done.

## What is special about the GitHub samples ?

The github samples ( https://github.com/TIBCOSoftware/tibco-streaming-samples ) don't contain SNAPSHOTS for the branches - after all, this is customer facing. So we have a branch 10.4.0 that contains 10.4.0 ... no 10.4.0-SNAPSHOT. For development testing, the qa pipelines specify the version in their `project_version` parameter, passing it down to samples Maven run as the `project.version` property. See JIRA Issue SB-47051 for improvements in this area when it comes to changing versions of the product.
So :

- The github samples build job needs to be tied to a github branch ( 10.4.0, 10.4.1 etc )
- The github samples build job updates pom.xml to add a specific version as per project.version parameter
- Once a new release is created, the github samples needs to be branched :
  - Create a new branch with a name equal to the new product version
  - Update versions in pom.xml
  - Run the github-samples jenkins job and select the new branch - this includes re-generating the docs
  - Update the branch_githubsamples parameter of the QA pipelines
- Builds may fail during the release process
- Note that for 10.5.x we hope to use version ranges to improve on this

# Administrating Artifactory

## What respositories do we use ?

To avoid clashes in the repositories, we have created :

- General use
  - **central** - cache of Maven central
  - oracle-maven – cache of Oracle's Maven site, which would normally require authentication. Also available via **ep-maven-virtual**.
  - **eclipse-mars** *et al.* - cache of Eclipse Mars update site. Because Eclipse version management uses update sites to separate releases, we create a cache of the update site of each Eclipse release that we adopt: for details on how they are created and managed, see Adopting a New Eclipse Release.
  - **nodejs** - cache of Node distributions

    - **npmjs** - cache of NPM distributions
    - **registry-npmjs-org** - cache of the Nodejs registry (analogous to Maven Central, but for Node Package Manager)
    - **jcenter** - cache of Jfrog tools
    - **centos** - cache of centos updates
- Event processing specific - this includes support for TIBCO products ActiveSpaces Transactions, StreamBase, Live Datamart, and Business Events Extreme.
    - **ep-snapshots** - Event processing Maven snapshots
    - **ep-releases** - Event processing platform-independent releases, like LiveView Web and ep-build. This is the preferred repo for releases going forward, as only our own release artifacts that are promoted to this repository are available from ep-maven-virtual. Access to platform-specific Maven artifacts need alternate settings or properties that are looked at by our standard settings files to get access to platform-specific artifacts.
    - **ep-releases-linux / ep-releases-macos / ep-releases-windows** - Event processing maven platform releases, never accessible from ep-maven-virtual.
    - **ep-3rdparty** - Event processing Maven third party.  Note that this repository has permissions to re-deploy releases.
    - **ep-generic** – A site for generic artifacts; Eclipse p2 in particular.
    - **ep-docker** - Event Processing Docker images – mostly for experimentation, available as container registry with `ep-artifacts-us-east.tibco.com:2001`.
    - **ep-rpm** - Event processing rpm files
    - **ep-product-installers** - Event processing installers
    - **ep-eclipse-mars** - Event processing Eclipse Mars plugins
    - **ep-vagrant** - Event process vagrant boxes.  See Using vagrant to create a virtual machine for DTM development
    - **ep-rc-windows** / **ep-rc-macos** / **ep-rc-linux** / **ep-rc** - Event Processing Maven release candidates from platform specific and agnostic builds.  Used by release process only.
- Event process virtual repositories
    - **ep-maven-virtual** -  Contains the local event processing maven repositories above plus public Maven repositories .  Suitable for use by Maven clients to download dependencies.
    - **ep-docker-virtual**, accessible as a container registry via `ep-artifacts-us-east.tibco.com:2000`; pushes go to **ep-docker-bos**, and it caches Docker hub via the associated **docker-hub.** Credentials for pushes are by TIBCO user id and API key; passwords work but are discouraged.
    - **ep-eclipse-mars-virtual** -  Contains the local event processing eclipse mars repository plus public eclipse mars update date.  Suitable for use by Eclipse Mars.
    - **ep-eclipse-neon-virtual** -  Contains the local event processing eclipse neon repository plus public eclipse mars update date.  Suitable for use by Eclipse Neon.
    - **ep-rc-windows-virtual** / **ep-rc-macos-virtual** / **ep-rc-linux-virtual** - Contains the local event processing release candidates plus public Maven repositories.  Used by release process only.

Other TIBCO groups are welcome to create their own repositories.

Container images that are meant for TIBCO cloud usage should be pushed to reldocker.tibco.com; credentials for pushes are via TIBCO user id and password; a stored Jenkins credential for `nightly` is available.

If you run a new or personal Artifactory server, you need either to proxy or to cache these repositories by using the Artifactory remote repository feature. You can use the Artifactory web interface or the `sbx artifactory` command to create a remote repository cache; run `sbx artifactory --help` for more details.

Sometimes, Jenkins will lose Artifactory credentials (perhaps on a plugin upgrade). If deployments start reporting mysterious 401 errors after successful Maven steps, go the **Artifactory** configuration section of the Jenkins system management page and re-enter our usual deployment credentials.

## How is the Docker repository set up ?

Docker doesn't have the concept of repositories, so ports are used to map to the correct repository :

- port 2000 – ep-docker-virtual (including `ep-docker-bos` ,which gets pushes, and `docker-hub` cache)
- port 2001 – ep-docker

So, if using the Maven plugin (see http://ep-artifacts-us-east.tibco.com/site/latest-linux/ep-maven-plugin/cloud_examples.html), to push an image the following can be used :

```
$ sw_mvn docker:push -Ddocker.push.registry=ep-artifacts-us-east.tibco.com:2001 -Ddocker.username=deployment -
Ddocker.password=deployment

...
[INFO] --- docker-maven-plugin:0.19.1:push (default-cli) @ hellolvweb ---
[INFO] DOCKER> The push refers to a repository [ep-artifacts-us-east.tibco.com:2001/ep/hellolvweb]
[INFO] DOCKER> 1.2.0: digest: sha256:a5a93f7eb70b950c92f96118990bed62fbccd8a1301c2970490ca902d4da5b18 size: 955
[INFO] DOCKER> Pushed ep/hellolvweb:1.2.0 in 21 minutes and 2 seconds
...
```

and to pull and run an image outside of Maven :

```
$ docker pull ep-artifacts-us-east.tibco.com:2001/ep/hellolvweb:1.2.0
...
1.2.0: Pulling from ep/hellolvweb
...
$ docker run -h A --name A.hellolvweb -p 10080:10080 -e NODENAME=A.hellolvweb ep-artifacts-us-east.tibco.com:
2001/ep/hellolvweb:1.2.0
```

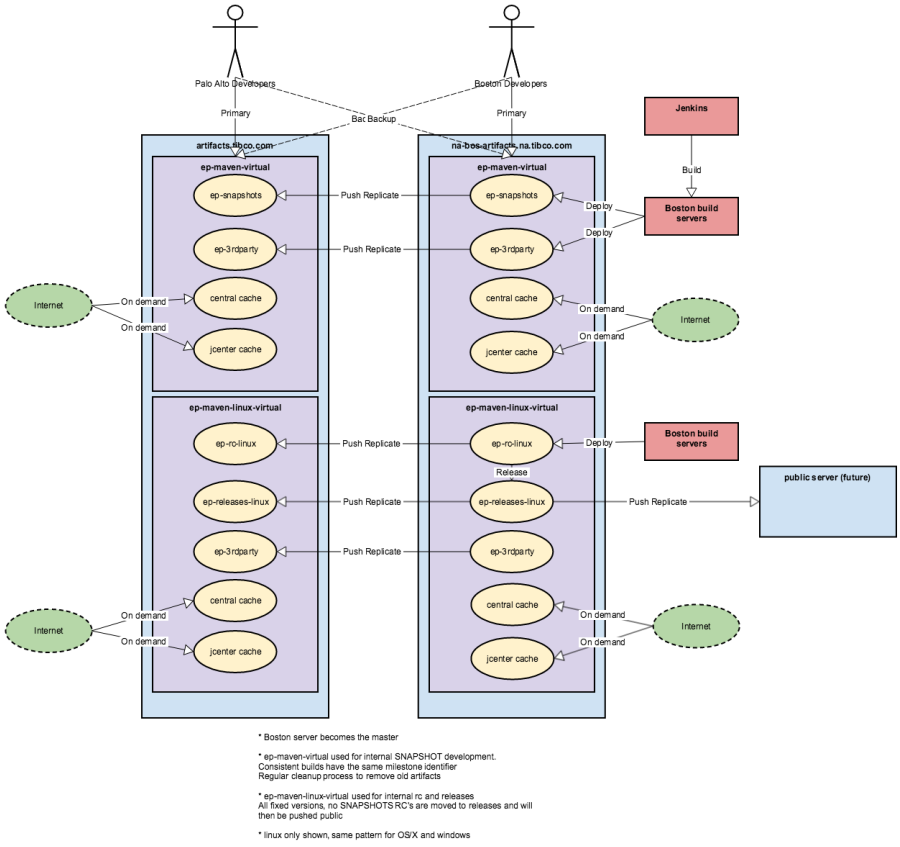
Note that Artifactory API tokens can be used as passwords for these Docker registries (and its other repositories in general), so they are a good choice for both Jenkins and local use cases - if you use an API token as your password, you'll still be in business when your password changes.

## How does replication work ?

Replicating artifacts to other servers is desirable for the following reasons :

1. Reduce the time to download artifacts.  For example when working in Boston, download from a local Boston Artifactory server instead of a remote server in Palo Alto.
2. More reliable deploys.  If the wide area network is less than 100% reliable, build servers can deploy to the local Artifactory server and allow Artifactory to synchronise asynchronously.
3. Resilience.  Users can use an alternative service should one fail.

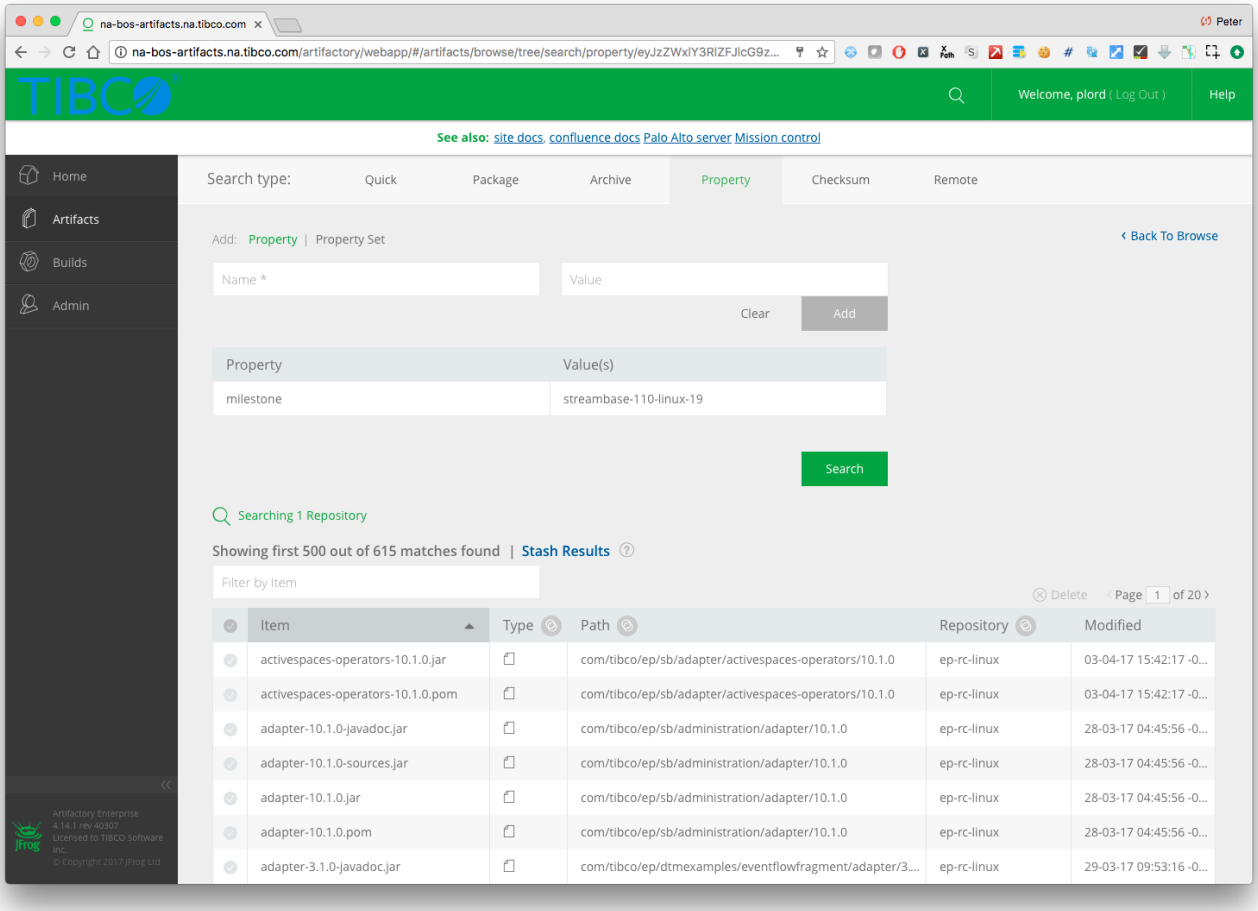
We use Repository replication so that artifacts are push immediately to other servers.




## How can I move artifacts ?

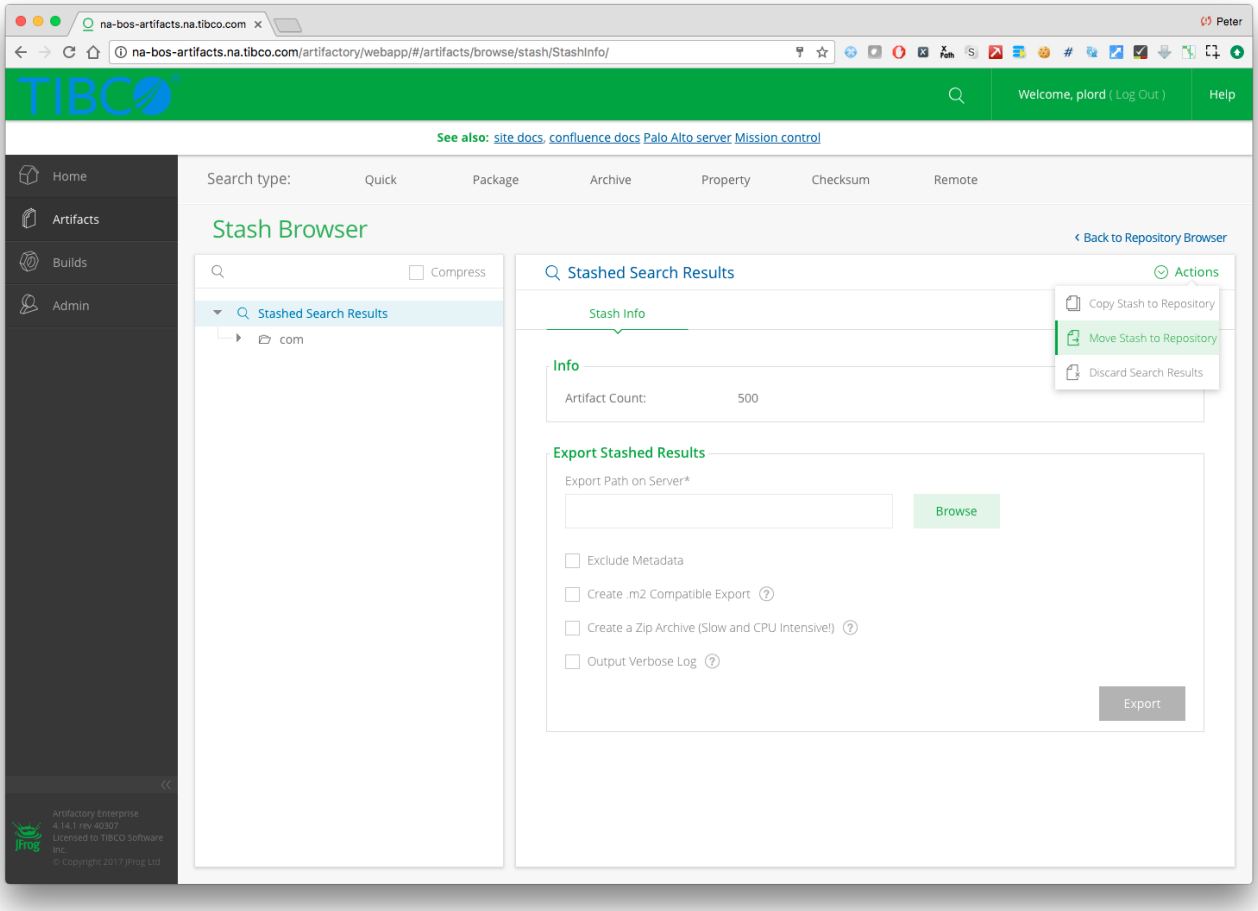
Sometimes we need to move a set of artifacts from one repository to another, main example is promoting rc artifacts to releases.

First artifacts need to be found, suggest search by property and limit search to a repository :




Stash the search results.

Then click on the stashed results and use Actions  move stash to repository.




Repeat if search results is greater than the limit of 500.

### What About Administration?

See Artifactory And Jenkins Management for how this is done.

# Administrating Jenkins

# How does pipeline linking work ?

The pipeline artifacts need to be linked - for example the ams pipeline needs to use the artifacts deployed by the runtime pipeline. To achieve this, when a pipeline completes, properties are set in artifactory which can be found by the next pipeline. For example :

1. runtime pipeline completes, old property runtime.latest is removed and properties runtime.latest=linux and runtime.complete=true is set for this milestone
2. ams pipeline starts, milestone is found be querying artifactory where properties runtime.latest=linux and runtime.complete=true. Also parent job name is verified
3. ams pipeline completes, old property ams.latest is removed and properties ams.latest=linux and ams.complete=true is set for this milestone
4. ...

A default milestone "None" is set if no milestone can be found - this happens only if the previous pipeline has never completed.

In this way, updates checked in for a milestone can still be tested even if the previous pipeline fails - simply the last dependency is used.

# What pipeline support functions

Needs updating: use of new functions that avoid mention of versions or branches, `workflowLibs` moving to `ep-jenkins-workflow`.

To support the pipeline script we have some utilities in https://git.tibco.com/git/?p=product/ep/eppoc.git;a=tree;f=workflowLibs/src/com/tibco/workflow - this allows simplified pipeline script :

```
@Library('workflowLibs')
import com.tibco.workflow.Alert
import com.tibco.workflow.Artifactory

// globals
//
def alert = new Alert()
def artifactory = new Artifactory()


// Send a build job alert
//
alert.jobAlert(buildResult, currentBuild)

// get latest milestone from that most recent runtime build
//
milestone = artifactory.getMilestone(artifactoryServer, "runtime", platform, "EventProcessing :: runtime-master-
"+buildLabel+"-pipeline")

// Set pipeline description
//
alert.setDescription(currentBuild, milestone)

// Send a pipeline job alert and update pipeline description
//
alert.pipelineAlert("SUCCESS", currentBuild, milestone)
```

## How do no-deploy pipelines work ?

- Once a day, the workspace is completely removed before a fresh checkout and build.  This is shown in the build history as **Clean** builds.
- Every 2 hours, the source code is checked for changes - if there are any changes, the workspace is updated from source code and updated.  This is shown in the build history as **Incremental** builds.  The changes that triggered the builds are shown in the **Stage View** as a number of commits.  These incremental builds are faster than the clean builds.
- The latest successful pipeline build artifacts are kept and made available for download under **Last Successful Artifacts.**  This is also shown by the blue circle to the left of the green build steps.
- **No tests are run - using these builds is at your own risk.** If you find any issues with these builds, please re-test with official builds before raising issues.
- Nothings is deployed to artifactory
- We have setup these 3 pipelines for the master branches - if you want to run builds with personal branches :
    - Take a copy of the jenkins pipeline job - give it a suitable name
    - Edit the copied pipeline job and change the default branch parameters
- These "no deploy" pipelines can be triggered by :
    - Specific time of day - in which case the workspace is removed first. Schedule is: `H 0 * * 0-5`
    - Poll for source code changes. Schedule is: `H 5-20/5 * * 0-5`
    - Manually

Note that of the 3 platforms, windows is the slowest and most fragile.  At time of writing, more work is needed to improve the windows builds.

Cloning the nd job of your choice, call it, say deend.<platform>

## What is in a Jenkins build job ?

Build jobs take a given source repository, build the component, run test cases and if tests pass, deploy built artifacts to artifactory.

Build jobs should be able to run on any platform - ie platform agnostic.

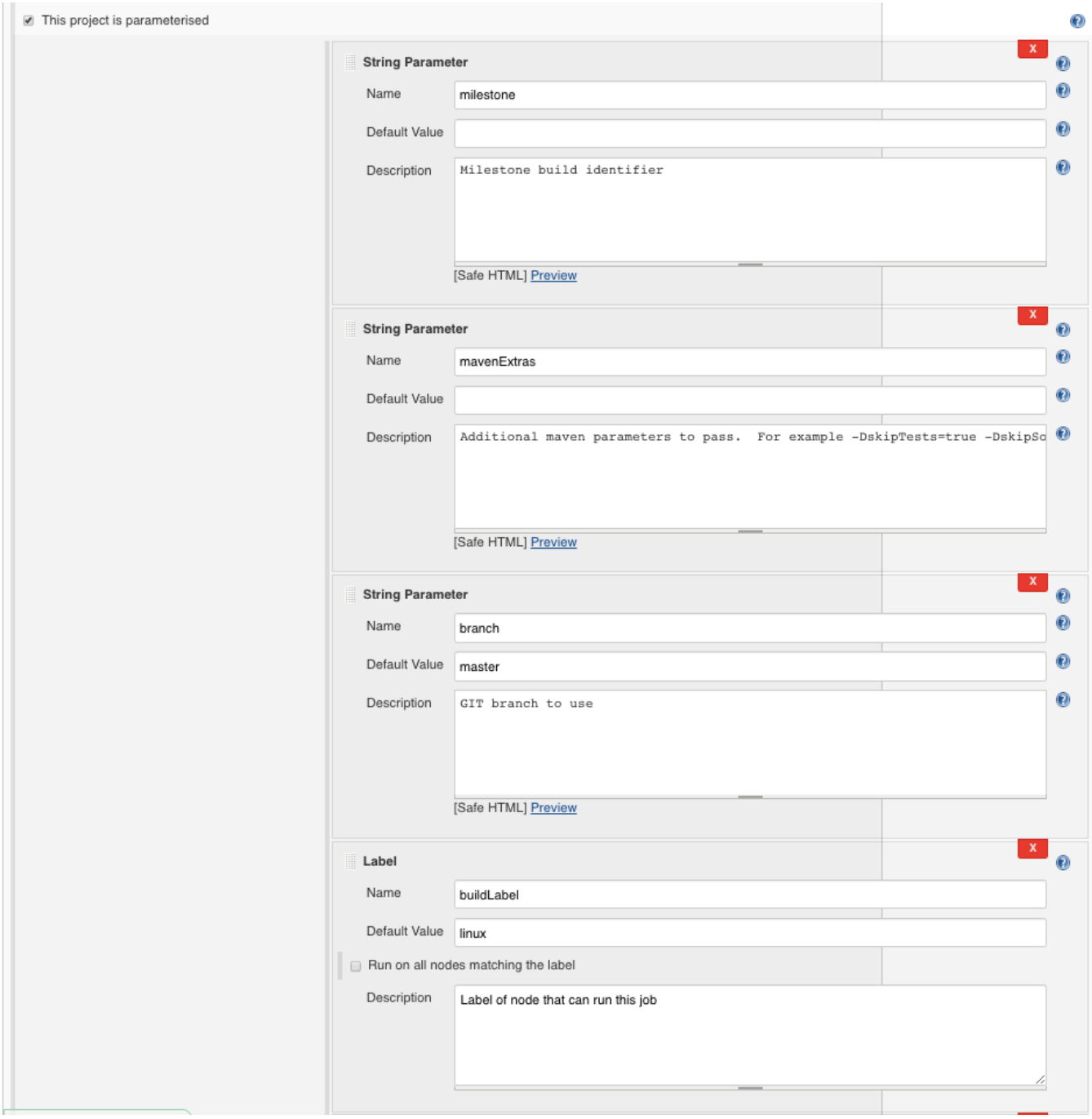
### Parameters

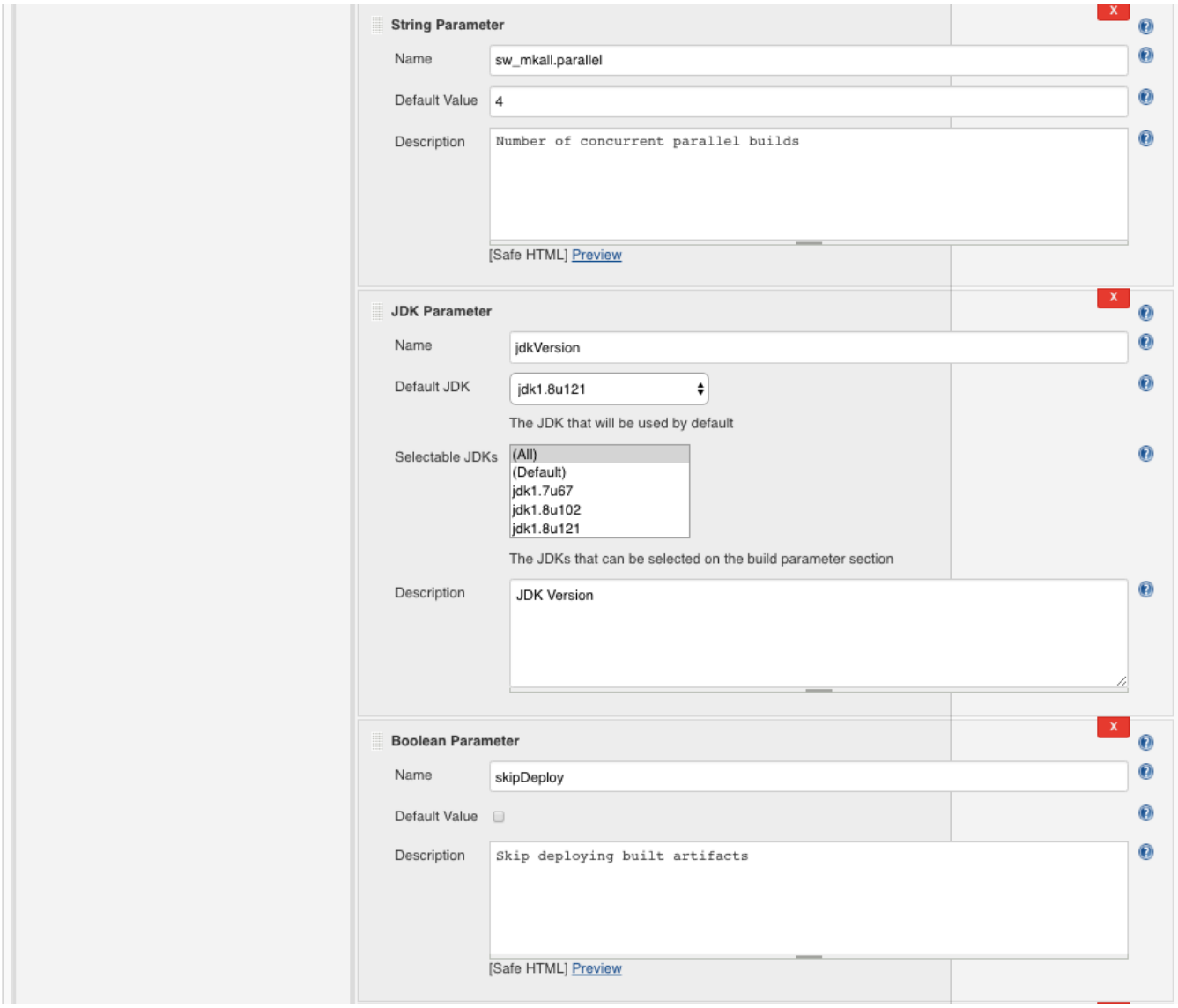
Each build contains at least the following parameters :

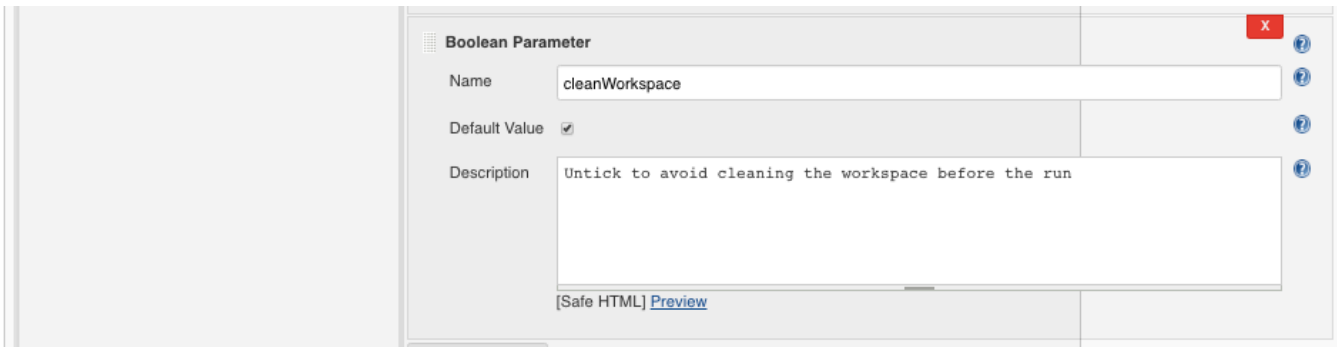
- **milestone** - milestone identifier
- **mavenExtras** - Any additional flags to pass to maven.  For example -X enables detailed tracing
- **branch** - GIT branch to use

- **buildLabel** - jenkins build server label.  May be a specific build server name or a Jenkins label (normally an expression composed of operators and labels)
- **sw_mkall.parallel** - Number of concurrent parallel builds for native
- **jdkVersion** - JDK version to use.  The pipeline jobs determin the JDK version, and so the JDK version in the job should be set to (System)
- **skipDeploy** - If set, skip deploying to Artifactory
- **cleanWorkspace** - Untick to avoid cleaning the workspace before the run (used by the **Deploy and continue** action)

These parameters are set by the pipline job, but also have suitable defaults.



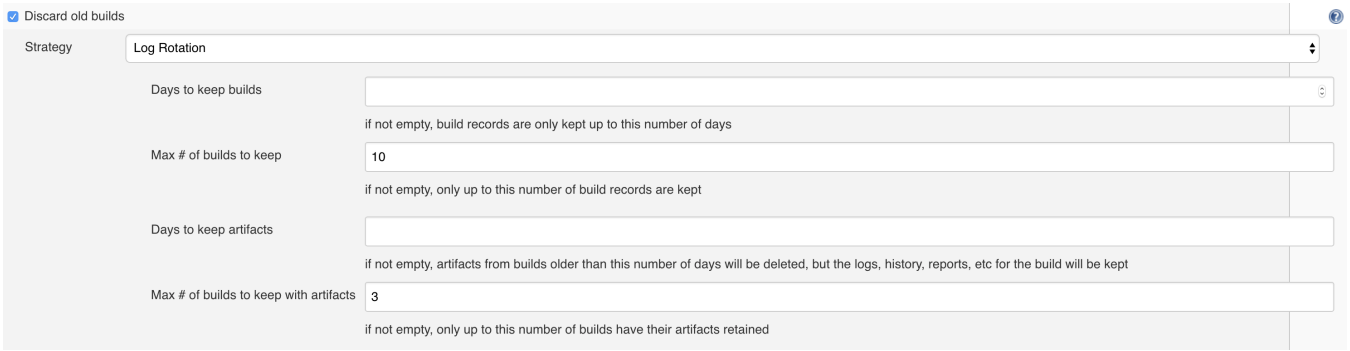



Note that the Selectable JDK's should be set to (ALL).

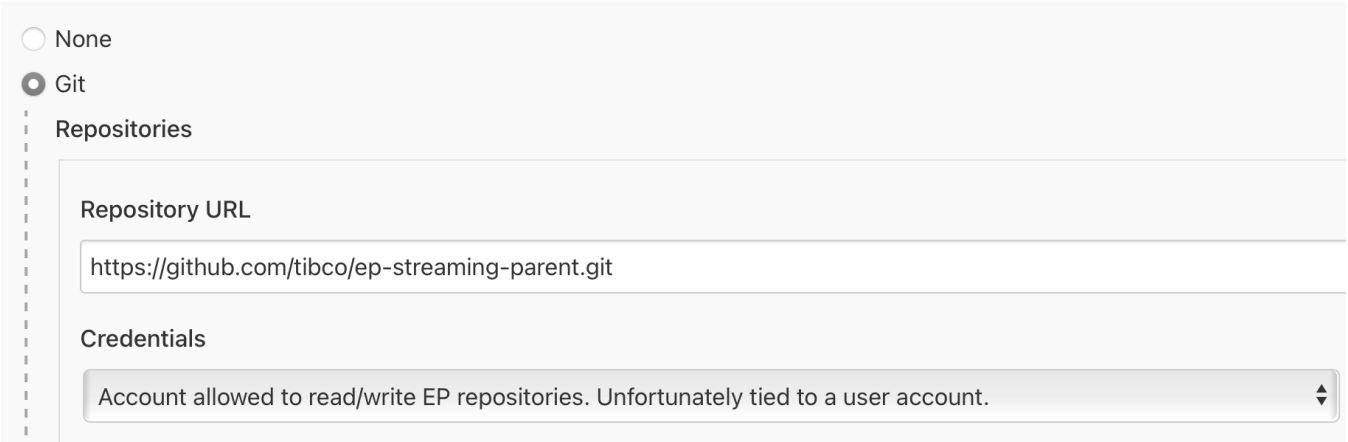
## Discard old builds

To avoid Jenkins filling up with unnecessary log files we discard old builds :



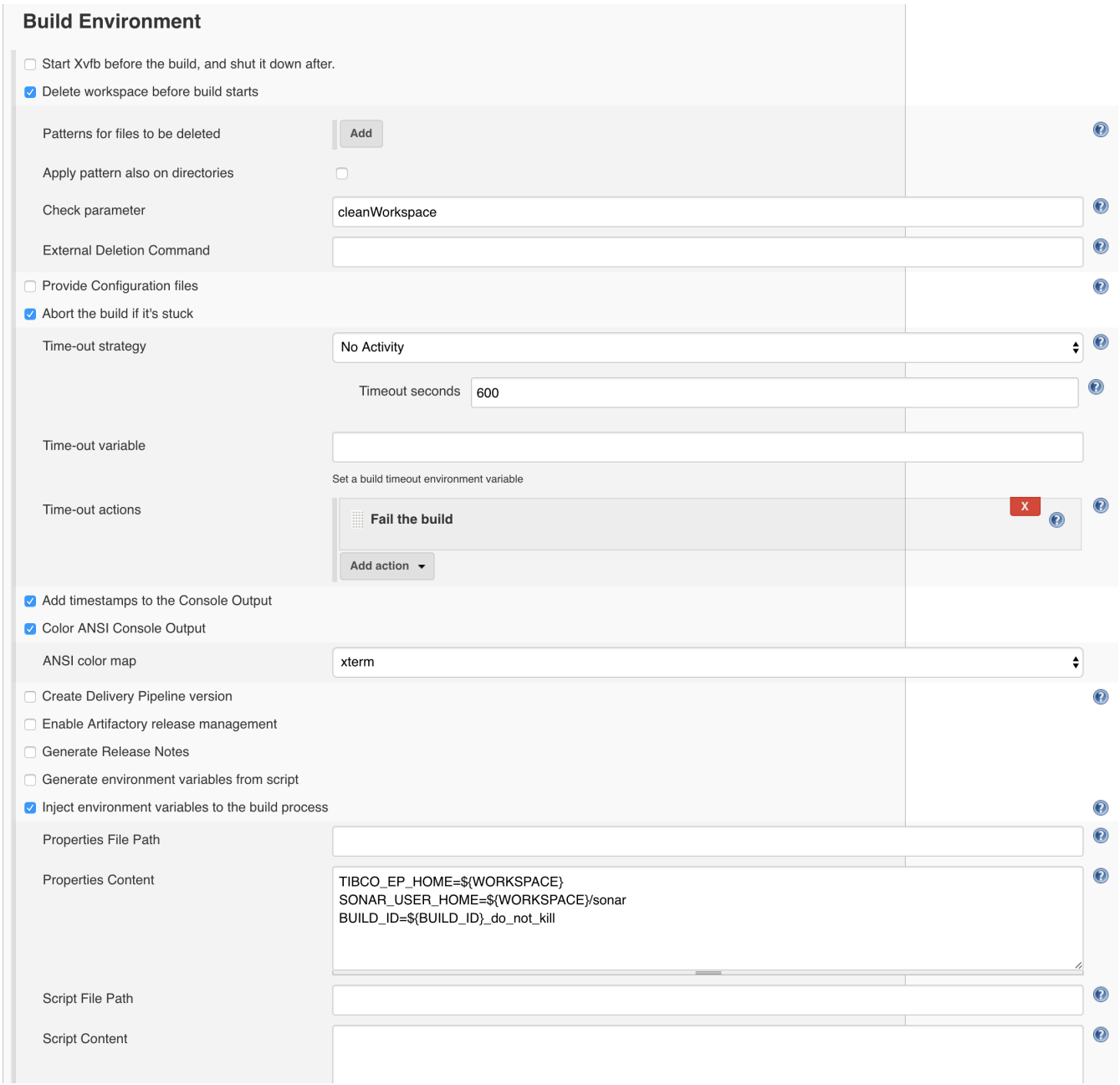

## Git

The Source Code Management section uses a placeholder GitHub user with PAT as a password, and variables `${CODELINE}` and `${branch}`.




## Build environment

The build environment is used to remove any previous builds, set time-out options, set properties and set the output formatting :



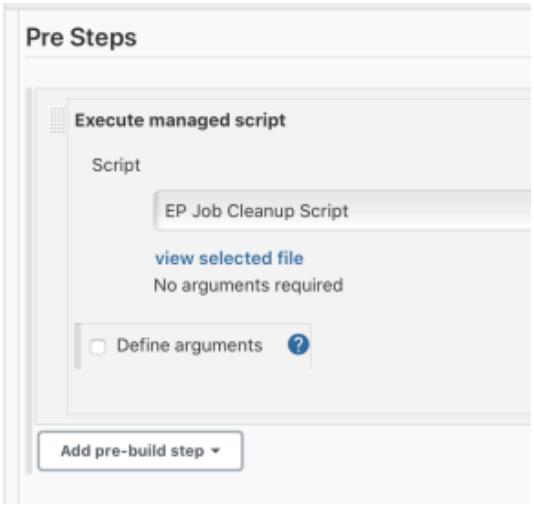

Note that for windows/npm builds ( such as ep-ams ) we need to set **USERPROFILE=${HOME}** as well. Because dtm processes can take up CPU time and add to traffic on the lab network, the BUILD_ID hack that works around Jenkins cleanup should be used only for runtime jobs that DTM experts will be around to debug.

A Groovy script can be used to set the description of the build job :




## Kill old jobs

On successful build, we would expect no left over processes, however on failure there may be some still running that the developers want to attach to to diagnose the failure.  Hence, we don't kill any left over processes at the end of each run but before the next run :




The clean script, which calls out to `sbx jenkins-janitor`, identifies old processes in two ways :

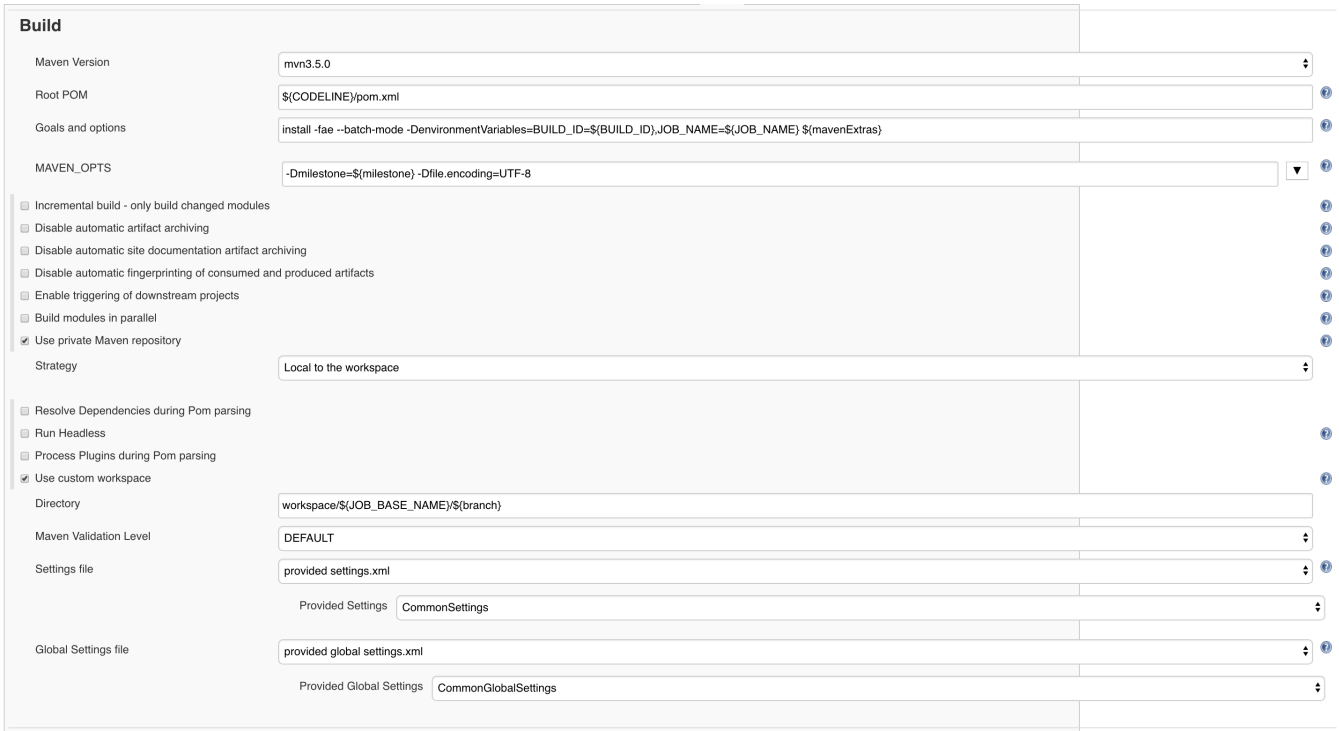
1. If the BUILD_ID from the previous run can be identified, this BUILD_ID is used to locate the right set of processes ( these have BUILD_ID set in the process environment )
2. Failing that, TIBCO_EP_HOME is used in a similar way

Note that this follows the same pattern as used by Jenkins by default ( see https://wiki.jenkins.io/display/JENKINS/ProcessTreeKiller )

Notice in the Maven build step that this environment variable is specifically set so that the epmaven maven plugin will pass it through to all spawned processes ( see also https://github.com/TIBCOSoftware/tibco-streaming-maven-plugin/blob/master/ep-maven/src/site/markdown/contint_examples.md#user-content-build-step ).

## Maven build step

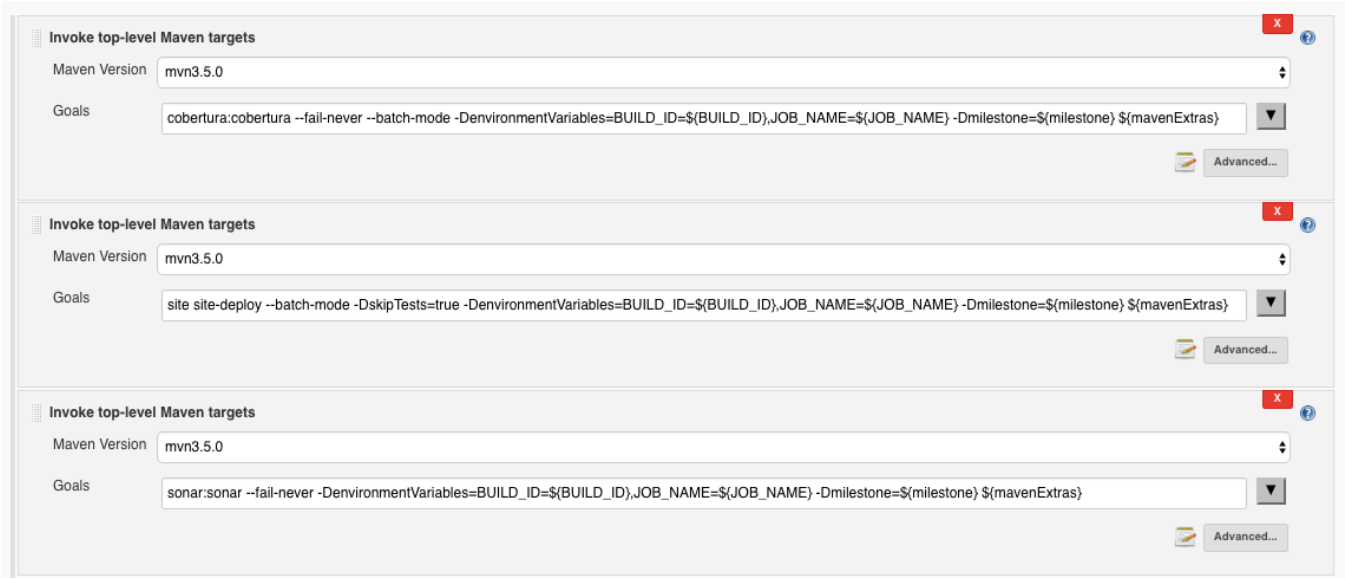
The maven build step calls the maven install target along with additional parameters required.



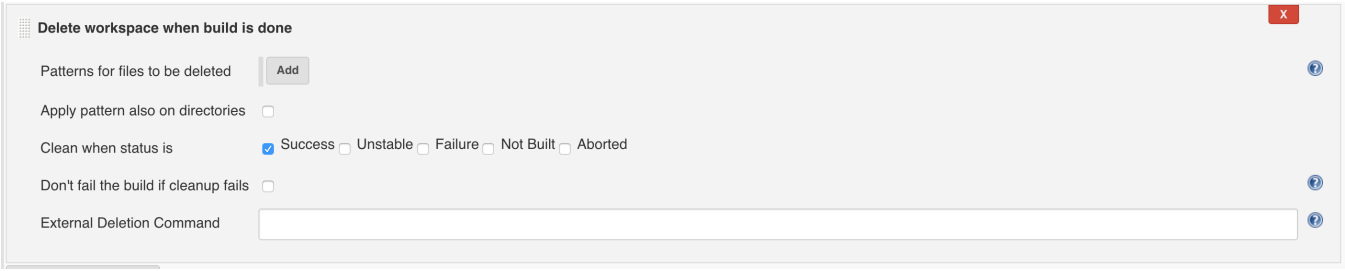

Note that this builds refers to a settings.xml file stored in jenkins - this is to configure the artifactory URL to download artifacts.  The download URL includes the milestone identifier so that the build job will use previously deployed artifacts from the same milestone.

We specify a per-branch custom workspace to avoid clashes between branch builds.

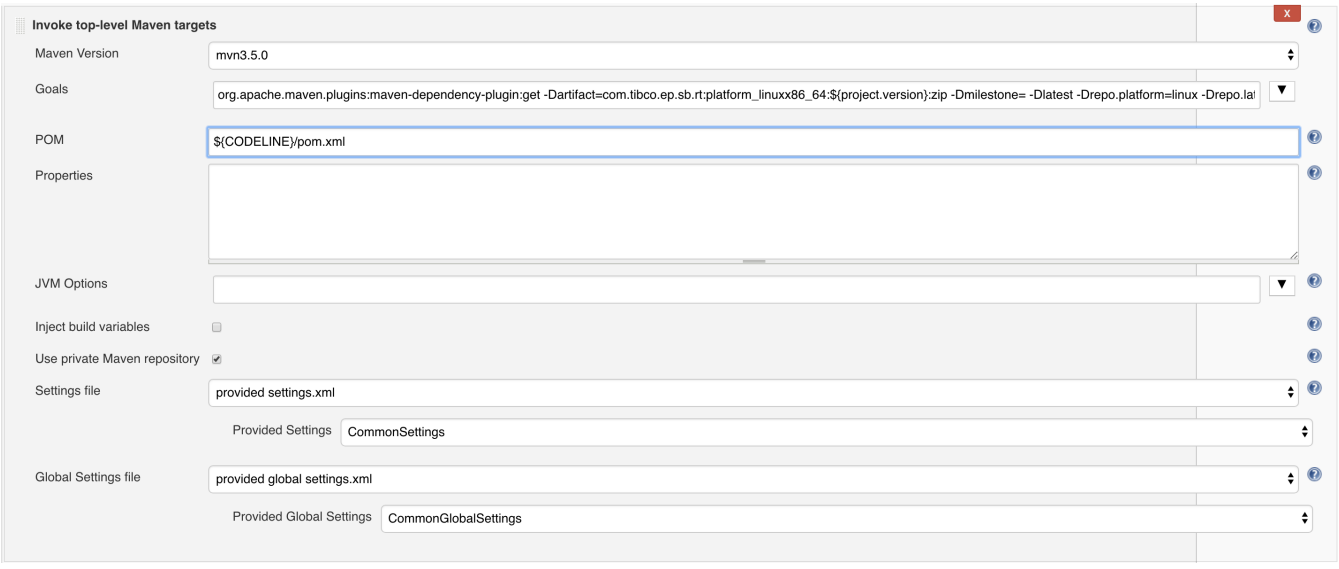
Further post steps complete the maven runs :




Workspace should be deleted on success to avoid possible use of hard-coded paths in downstream builds :

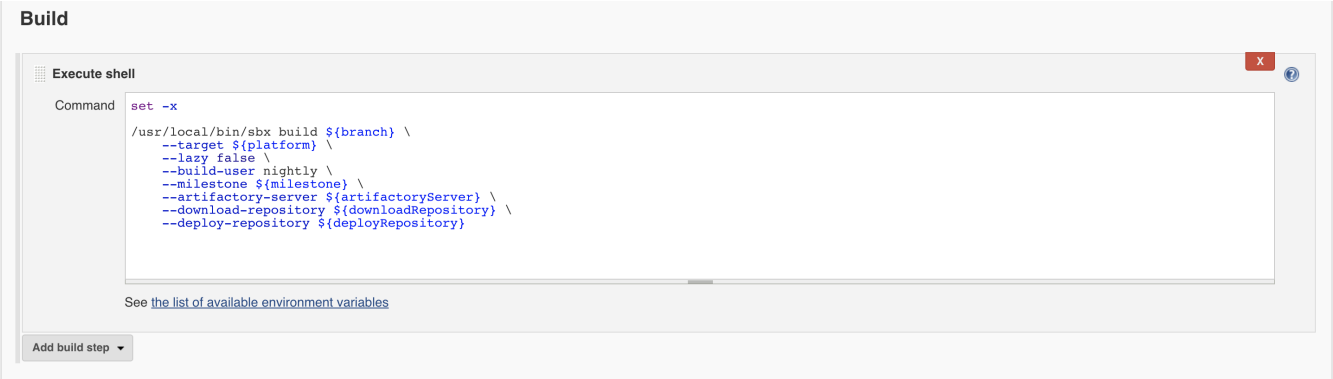



In the special case of Docker, builds on mac & windows will require the runtime from linux. This can be achieved by using the org.apache.maven.plugins: maven-dependency-plugin:get goal :



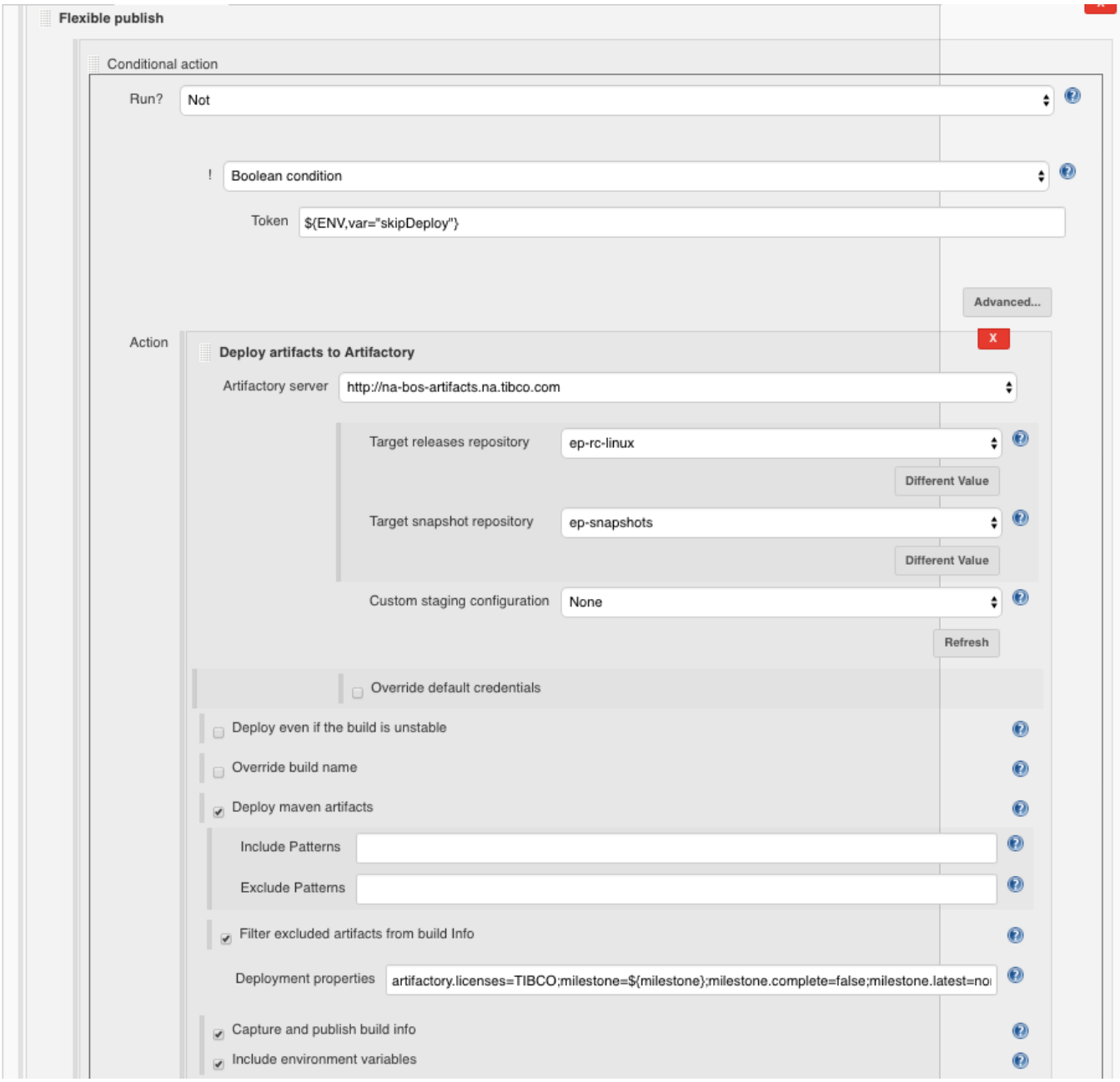

## sbx build step

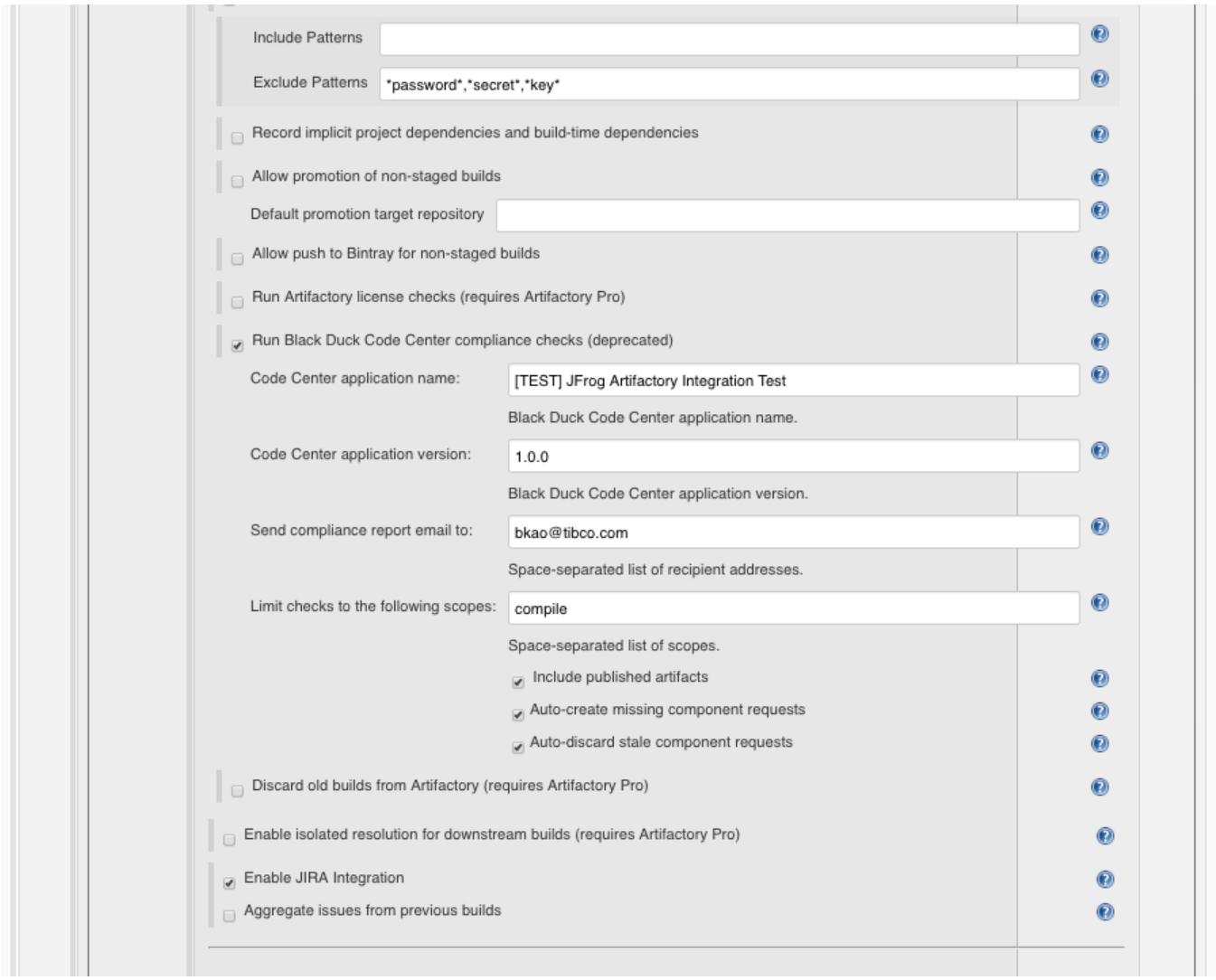
The streambase build step calls out to sbx to do installer builds.




## Deploy to artifactory

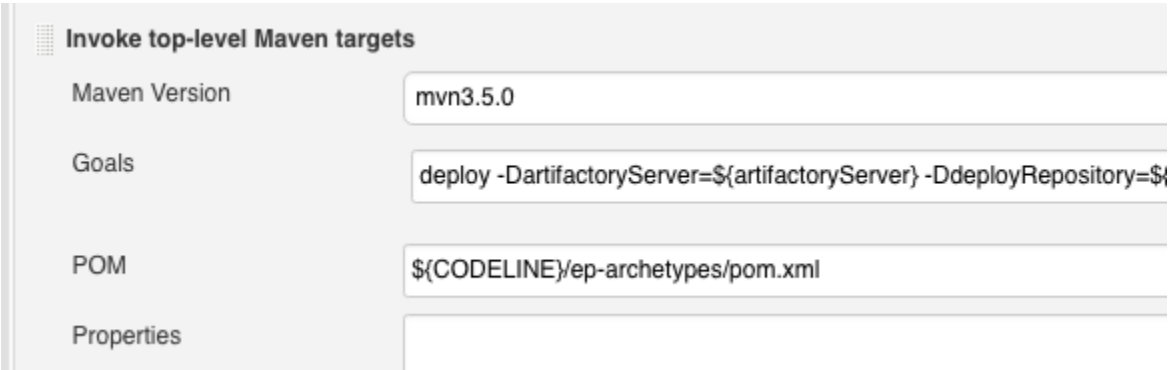
Instead of using either the native maven deploy plugin or the artifactory maven plugin, we use the jenkins plugin to deploy artifacts to artifactory.  This gives us some benefits including always deploy at the end of the build and link jenkins builds to artifactory.  We trigger this optionally so deploy can be skipped :


Flexible publish
Conditional action
Run?
Not
!
Boolean condition
Token
${ENV,var="skipDeploy"}
Advanced...
X
Action
Deploy artifacts to Artifactory
Artifactory server
http://na-bos-artifacts.na.tibco.com
Target releases repository
ep-rc-linux
Different Value
Target snapshot repository
ep-snapshots
Different Value
Custom staging configuration
None
Refresh
Override default credentials
Deploy even if the build is unstable
Override build name
Deploy maven artifacts
Include Patterns
Exclude Patterns
Filter excluded artifacts from build Info
Deployment properties
artifactory.licenses=TIBCO;milestone=${milestone};milestone.complete=false;milestone.latest=no
Capture and publish build info
Include environment variables




Notice that the repository includes the milestone name - this tells artifactory to add the milestone parameter to the deployed artifacts - see https://www.jfrog.com/confluence/display/RTF/Using+Properties+in+Deployment+and+Resolution for further details.

One case where the artifactory deploy mechanism doesn't work is with archetypes ( located in the sbrt repository ).  So in this case we add in a specific deploy step :




We've found that enabling **Record implicit project dependencies and build-time dependencies** can bring in 100's of non essential dependencies to artifactory which causes load issues with Black Duck.

## Team email notification

Team email and slack notifications are in the pipeline script.

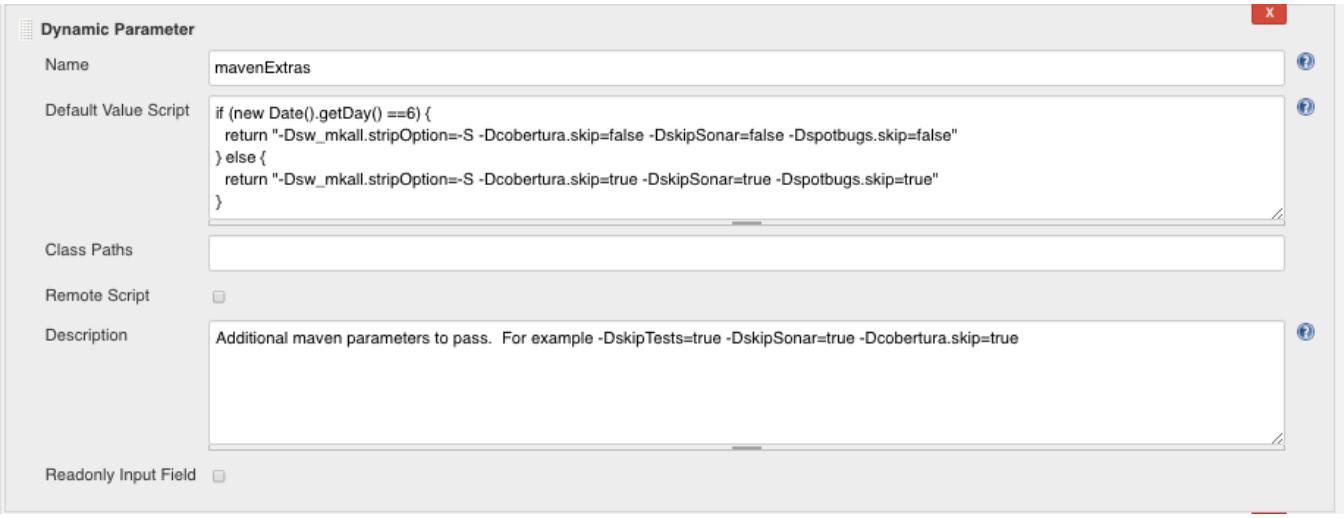
## Executing

Runtime build worker jobs are executed by pipeline jobs.

Manually running of individual jobs is not normal, but may be useful to repeat a build after a check fix for verification - Jenkins has a **retry** button for this purpose.

## Special runs on Saturday (Disabled)

On Linux only and on Saturdays, extra runs are enabled for coverage, spotbugs and sonar reporting.  This is triggered in the pipeline parameters :




The linux build jobs can take notice of these parameters :





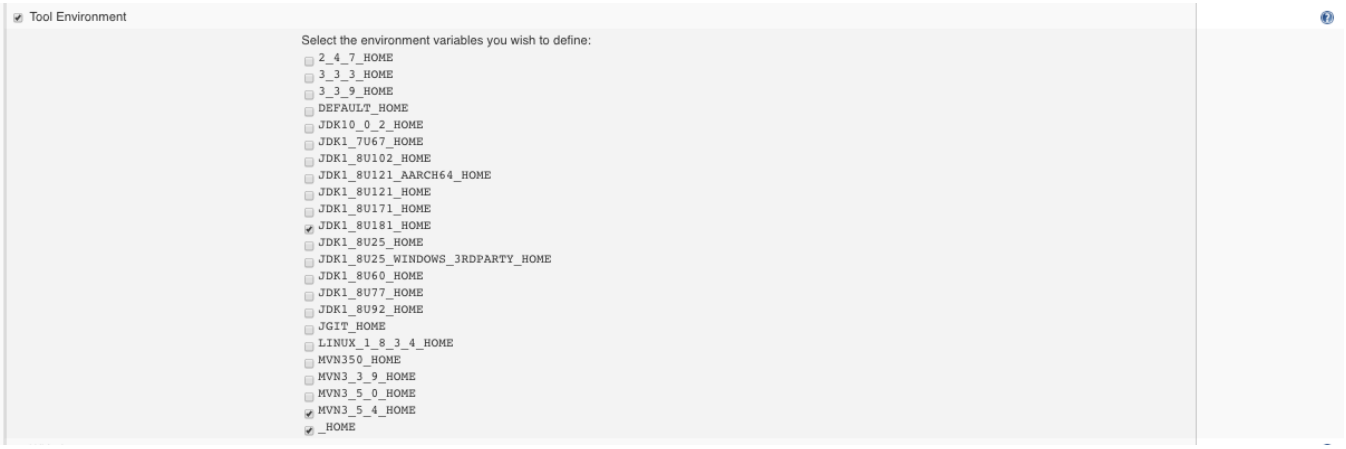




Note that the Maven targets are only run when actually needed.

The script to upload the Spotbugs report to http://sadm.tibco.com does require access to the Maven settings.  So we also need :






## What administration jenkins jobs do we have ?

The following non-build jobs are available :

- **cleanup-builds** - remove old artifacts from artifactory – the old cleanup job was run from the runtime-master-linux-pipeline.
- **streaming-thirdparty** - deploy non-TIBCO components to ep-3rdparty on Artifactory.
- **publishdevzone** - enter build milestones for the platform and push the docs to devzone
- janitor-sweep – force the Jenkins janitor to run an all qualified executors to free up more disk space; run on the weekends.
- mirrorowasp – mirror OWASP data locally.

## How do we use Jenkins in the release process ?

Release process with start with a JIRA issue - release plan added to the issue and comments added to track build milestones and steps completed.

1. Code freeze
2. Obtain branch go-ahead from runtime, ams, qa and studio teams
3. Create release branch - see http://jenkins-bos.na.tibco.com:8080/job/EventProcessing/job/create.release.branch/
4. Version bump master - see http://jenkins-bos.na.tibco.com:8080/job/EventProcessing/job/update.master/
5. Build release branch, verify with QA
6. Remove SNAPSHOT from release branch - see http://jenkins-bos.na.tibco.com:8080/job/EventProcessing/job/create.release.branch/
7. Build release branch into artifactory rc repositories
8. Verify with QA
9. Copy artifacts from artifactory rc repositories to release repositories - see Movingartifacts
    a. Note that archetype-catalog.xml might need to be copied as well
10. Tag release branches - see http://jenkins-bos.na.tibco.com:8080/job/EventProcessing/job/tag.release.branch/
11. Copy site docs to devzone - see http://jenkins-bos.na.tibco.com:8080/job/EventProcessing/job/publishdevzone/
12. Version bump release branch ready for maintenance - see http://jenkins-bos.na.tibco.com:8080/job/EventProcessing/job/update.release.branch/

The runtime pipeline schedules, which effectively control the build for the branch, should be disabled (unticked) during version bumps. If unticking the schedule option loses the schedule details, you can get our schedule options in the "How are the main pipeline jobs scheduled ?" section on this page.

## How do we use Jenkins for version bumping ?

Here we describe some of the mechanical steps required during the release process.

Create a new release branch from master and update maven versions

The jenkins job create.release.branch will :

- For each codeline
    - git clone
    - create a new branch & checkout
    - use maven to remove SNAPSHOT from versions
    - run some work-arounds where the above isn't sufficient
    - build (although no tests for speed)
    - verify dependency contains no snapshots
    - commit
    - optionally push

Note that the **projecturl** in the jenkins jobs will need updating so that public docs have the right references.

### Promote snapshot to next major snapshot

The goal here is to update SNAPSHOTs the next major version.  Eg 1.1.0-SNAPSHOT to 1.2.0-SNAPSHOT

The jenkins job update.master will :

- For each codeline
  - git clone
  - use maven to update the maven version
  - run some work-arounds where the above isn't sufficient
  - build (although no tests for speed)
  - commit
  - optionally push

## Promote release to next minor snapshot

The goal here is to update release the next minor SNAPSHOT version.  Eg 1.1.0 to 1.1.1-SNAPSHOT

The jenkins job update.release.branch using the first option for nextVersionArg :

- For each codeline
  - git clone / checkout
  - use maven to update the maven version
  - run some work-arounds where the above isn't sufficient
  - build (although no tests for speed)
  - commit
  - optionally push

## Promote release to next minor release

The goal here is to increase the minor version.  Eg 1.0.1 to 1.0.2

The jenkins job update.release.branch using the second option for nextVersionArg :

- For each codeline
  - git clone / checkout
  - use maven to update the maven version
  - run some work-arounds where the above isn't sufficient
  - build (although no tests for speed)
  - commit
  - optionally push

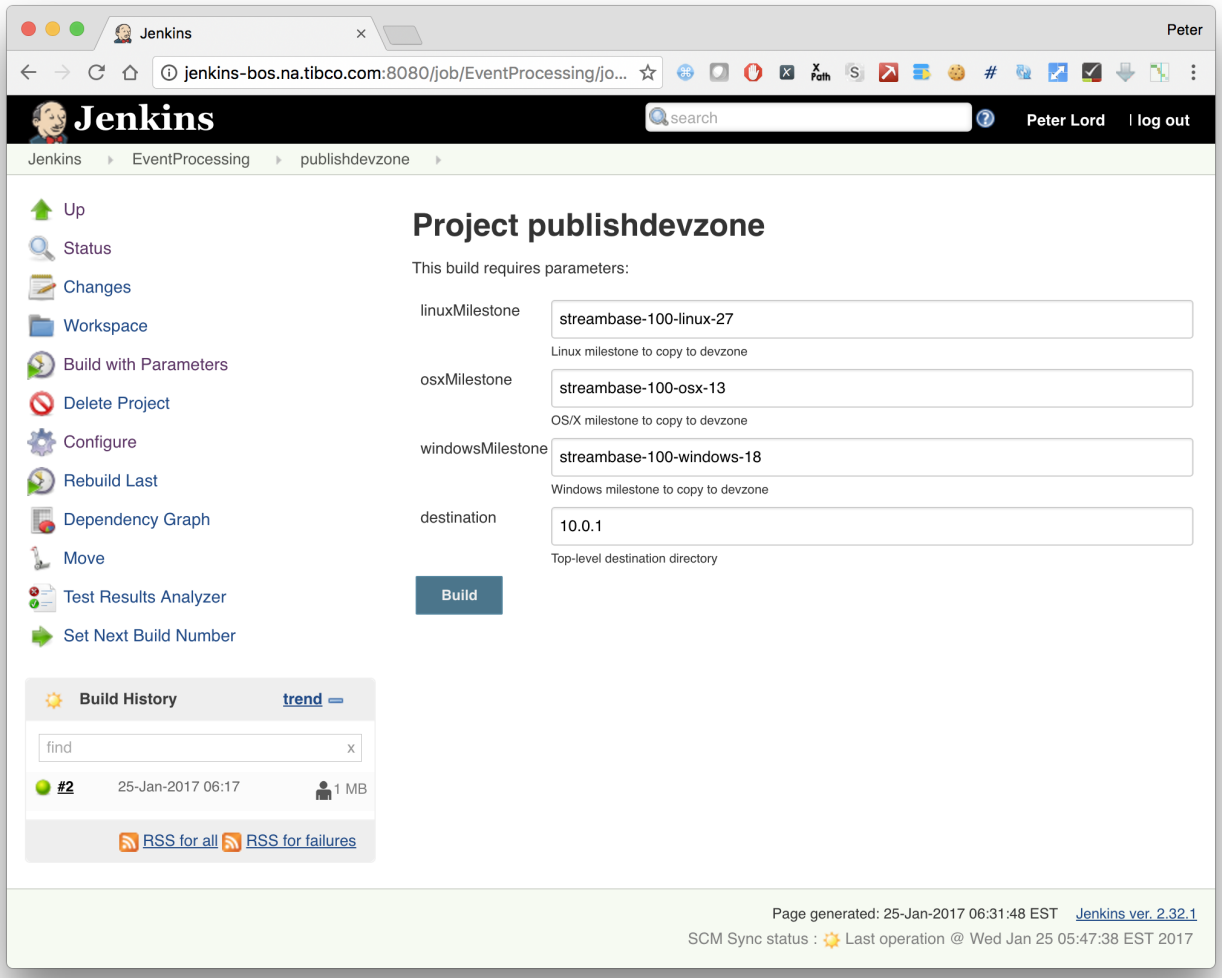
## Exceptions

The above jenkins jobs have various work-arounds in them to handle exceptions.  We should consider updating the codelines to avoid these cases.  Open issues are :

- parent
  - epmaven.version not updated since it doesn't exist yet - https://devzone.tibco.com/jira/browse/FLUENCY-10048
    - sed -i -e "/epmaven.version/s+${thisVersion}+${nextVersion}+" pom.xml
- dtm
  - product/assembly/pom*xml not updated - https://devzone.tibco.com/jira/browse/FLUENCY-10050
    - sed -i -e "s+${thisVersion}+${nextVersion}+" product/assembly/pom-*.xml
- sbrt
  - sb-eclipse-runtime/META-INF/MANIFEST.MF needs update - https://devzone.tibco.com/jira/browse/FLUENCY-10051
    - sed -i -e "s+${eclipseThisVersion}+${eclipseNextVersion}+" sb-eclipse-runtime/META-INF/MANIFEST.MF
  - sb-product/assembly/pom*xml not updated - https://devzone.tibco.com/jira/browse/FLUENCY-10050
    - sed -i -e "s+${thisVersion}+${nextVersion}+" sb-product/assembly/pom-*.xml
  - sb-client-core/src/main/java/com/streambase/sb/util/Version.java - https://devzone.tibco.com/jira/browse/FLUENCY-10051
    - New major releases need to be hand-added
- dtmexamples
  - JNI version is encoded in java (CallCpp.java) - should be able to use nar to generate file - https://devzone.tibco.com/jira/browse/FLUENCY-10052
  - sdk in pom.xml not updated - https://devzone.tibco.com/jira/browse/FLUENCY-10053
  - examples hierarchy not updated since they now use the shipped parent pom - https://devzone.tibco.com/jira/browse/FLUENCY-10053
- rms
  - test-api, dt-manager and management versions in sb-rms/pom.xml not updated - https://devzone.tibco.com/jira/browse/FLUENCY-10054
- sbproductsmoke
  - Its not possible to skip the product installation so tooling can fail - https://devzone.tibco.com/jira/browse/FLUENCY-10056
- qa
  - fails to download ca-devtest - https://devzone.tibco.com/jira/browse/FLUENCY-10055
  - pom.xml uses profiles to reference modules and so version bump fails for sub-directories - https://devzone.tibco.com/jira/browse/FLUENCY-10055
  - dtmperformance contains 2 poms - https://devzone.tibco.com/jira/browse/FLUENCY-10055
    - mvn versions:use-latest-snapshots -Dincludes=com.kabira*,com.tibco*
  - dtmperformance depends on platform zip and uses a lot of properties - can this use shipped parent pom ? - https://devzone.tibco.com/jira/browse/FLUENCY-10055
  - versions encoded in zip filenames - https://devzone.tibco.com/jira/browse/FLUENCY-10055
  - devtest poms not updated - https://devzone.tibco.com/jira/browse/FLUENCY-10055

# How are internal site docs published to DevZone ? (Obsolete from Streaming 10.6 on)

The jenkins job **publishdevzone** pushes the internal site docs to the devzone.  In the future this jenkins job will also update the public maven repository.  The milestones and version need to be specified :

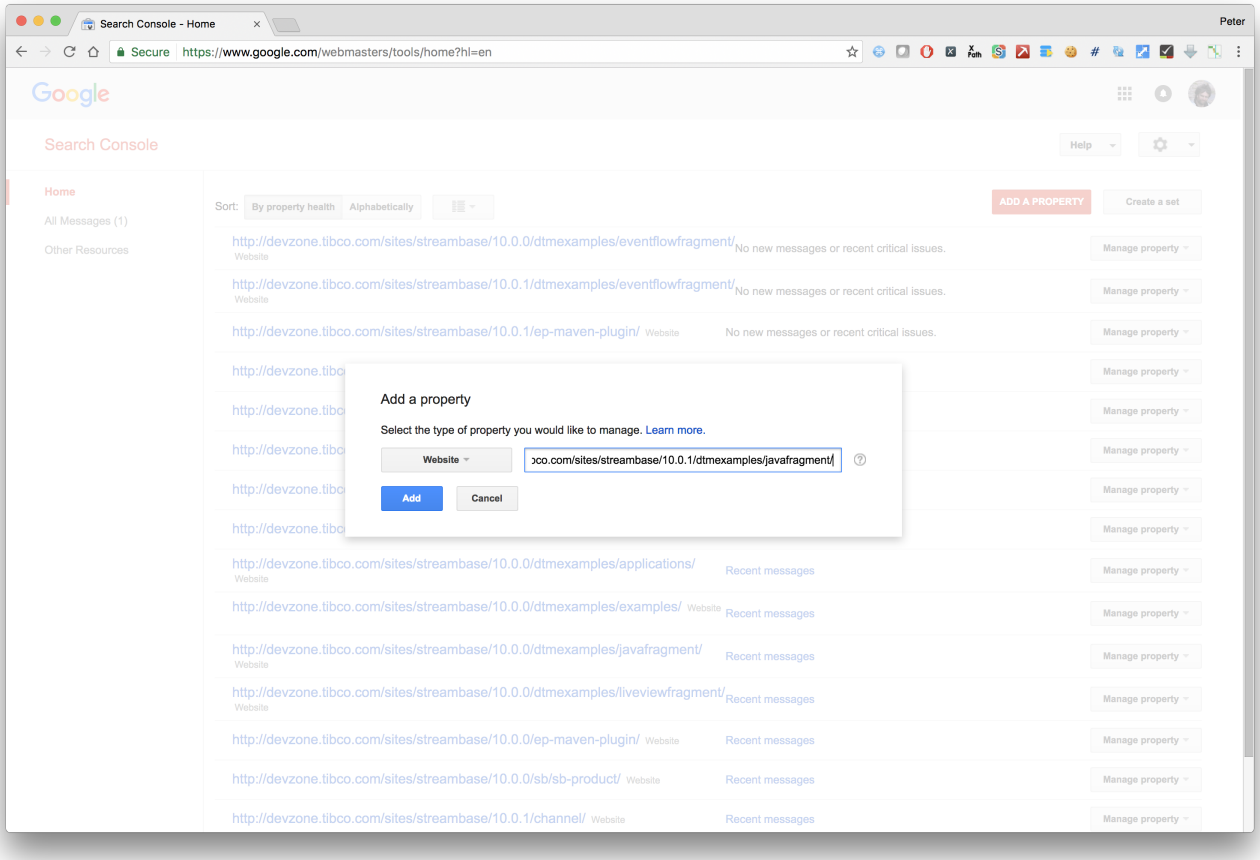



This job is bound to comet.tibco.com; the `nightly` account must have password-free SSH access using the standard nightly SSH key to both Artifactory and tibco-app02.managed.contegix.com (IPV4 is 199.102.167.22), which is a companion Linux machine to `devzone.tibco.com` which has access to the same file systems that are used for devzone. Both machines are administered by Contegix and root-level operations must be run through them.

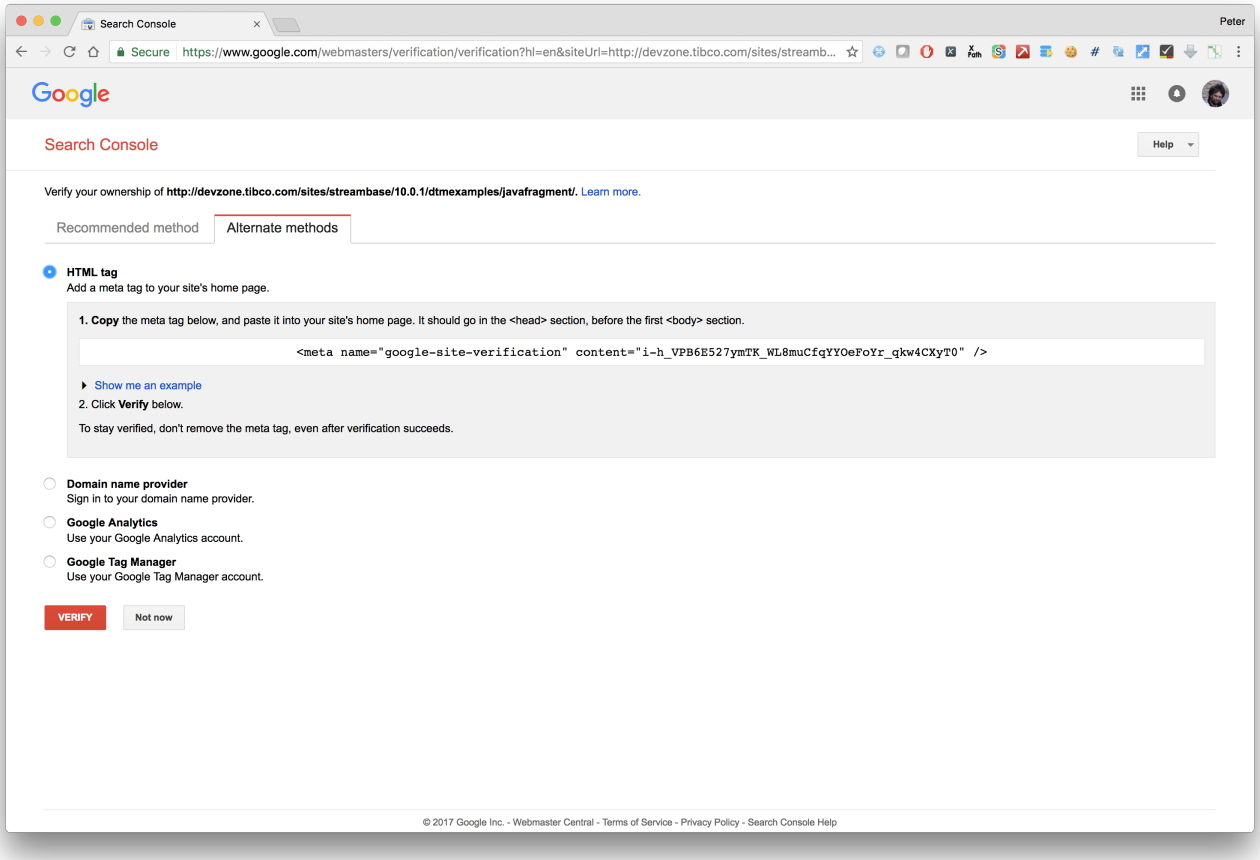
The site docs do include google sitemap.xml and ownership tags so can be added to google webmaster console to manage searches.

To add a new site to the webmaster console the following sequence can be followed :

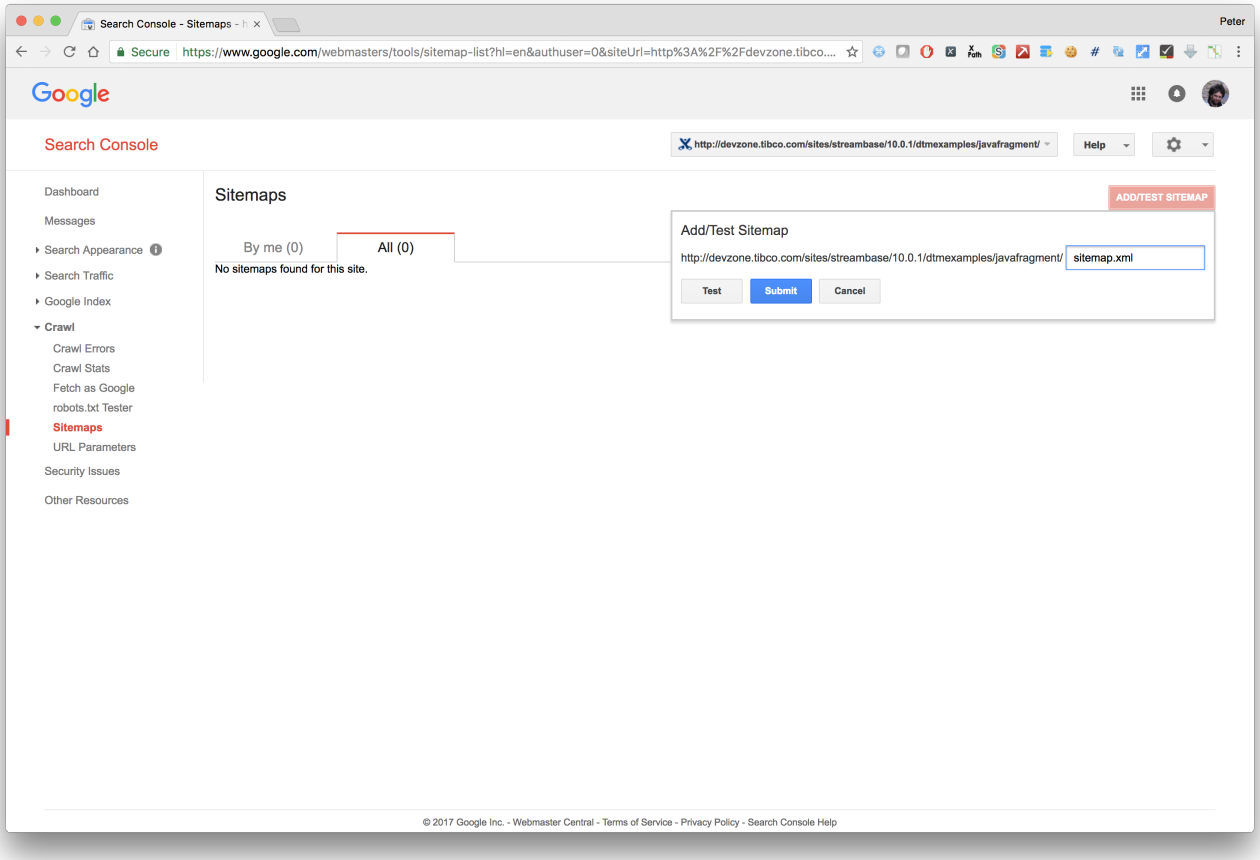
Add the new property :



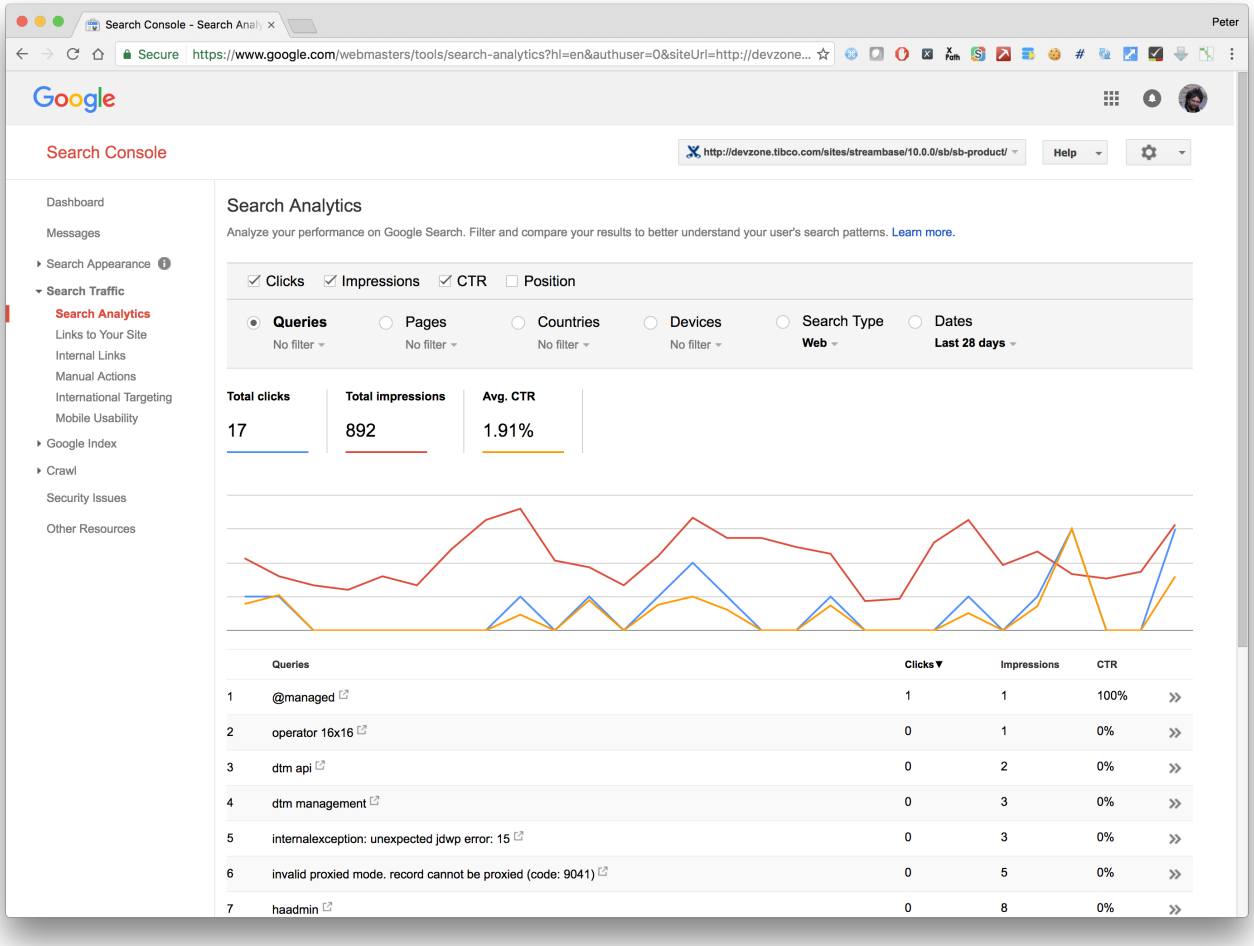

Verify ownership via the alternative method - since the tag is already added, pressing verify is sufficient :




Submit the sitemap.xml :



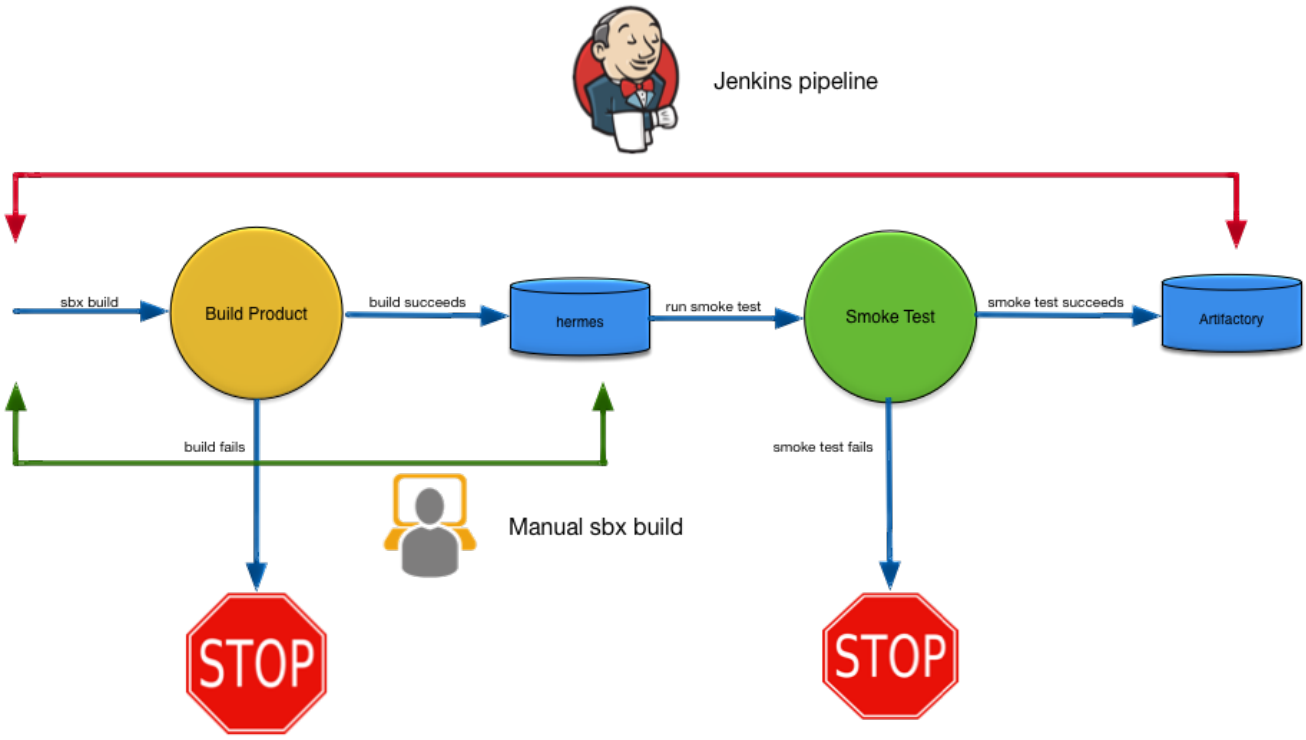

Over time google will index the pages and search reports are available :




## How do sbx builds work ?

**sbx** builds always stage installers to **ep-dist.tibco.com**. This is true for both Jenkins and **sbx** command line builds initiated manually. The Jenkins pipeline builds migrate the build to Artifactory if the smoke test succeeds.

> Accessing builds on hermes.streambase.com requires a StreamBase Active Directory user account.




SBX Builds.graffle.zip

sbx builds use these rules to resolve required dependencies.

- if the reactor POM in the SVN repository has **SNAPSHOT** in the version, the dependencies are resolved from the **ep-snapshots** repository with **milestone.latest=$***platform*
- if the reactor POM in the SVN repository does not have **SNAPSHOT** in version, the dependencies are resolved from **ep-rc-$***platform*.

## How can I copy Jenkins jobs ?

Its not obvious how jobs are copied, the process is :

- Select New Item link in EventProcessing
- Add the new project name in **Enter an item name** at the top
- Add the old project name in **Copy from** at the bottom - this auto completes

## How can I issue job events for analysis?

This is part of an experimental effort (JIRA Issue SB-47203) to measure how are builds are behaving without requiring a lot of state to be managed on the Jenkins master server, using a Jenkins plugin that publishes to our lab's Kafka instance, which can be read and displayed by our LVDucky server.

Most of the configuration for this plugin is already taken of by global defaults, but each job that should contribute data needs to be configured by itself. Add a "Publish to Kafka" step to the Post Build Actions. The important parameter are:

- Bootstrap Servers: **lvtest.lab.streambase.com:9092**
- Metadata Broker List: **lvtest.lab.streambase.com:9092**
- Topic Name: **BuildEvents**
- Under Advanced options, *check* **Include Unit Test Summary**.

The intention is to avoid having all details configured globally so that these fields and left blank. A secondary issue is to get pipelines involved.